

that was also requested by many readers of the first two editions): namely, how to use the AWS Provider with the Kubernetes provider to deploy Dockerized apps. Kubernetes is, in many ways, a cloud of its own—it can run applications, networks, data stores, load balancers, secret stores, and much more—so, in a sense, this is both a multiprovider and multicloud example. And because Kubernetes is a cloud, that means there is a lot to learn, so I’m going to have to build up to it one step at a time, starting with mini crash courses on Docker and Kubernetes, before finally moving on to the full multiprovider example that uses both AWS and Kubernetes:

- A crash course on Docker
- A crash course on Kubernetes
- Deploying Docker containers in AWS using Elastic Kubernetes Service (EKS)

A Crash Course on Docker

As you may remember from [Chapter 1](#), Docker images are like self-contained “snapshots” of the operating system (OS), the software, the files, and all other relevant details. Let’s now see Docker in action.

First, if you don’t have Docker installed already, follow the instructions on the [Docker website](#) to install Docker Desktop for your operating system. Once it’s installed, you should have the `docker` command available on your command line. You can use the `docker run` command to run Docker images locally:

```
$ docker run <IMAGE> [COMMAND]
```

where `IMAGE` is the Docker image to run and `COMMAND` is an optional command to execute. For example, here’s how you can run a Bash shell in an Ubuntu 20.04 Docker image (note that the following command includes the `-it` flag so you get an interactive shell where you can type):

```
$ docker run -it ubuntu:20.04 bash
```

```
Unable to find image 'ubuntu:20.04' locally
20.04: Pulling from library/ubuntu
Digest: sha256:669e010b58baf5beb2836b253c1fd5768333f0d1dbcb834f7c07a4dc93f474be
Status: Downloaded newer image for ubuntu:20.04
```

```
root@d96ad3779966:/#
```

And voilà, you’re now in Ubuntu! If you’ve never used Docker before, this can seem fairly magical. Try running some commands. For example, you can look at the contents of `/etc/os-release` to verify you really are in Ubuntu:

```
root@d96ad3779966:/# cat /etc/os-release
NAME="Ubuntu"
VERSION="20.04.3 LTS (Focal Fossa)"
```

```
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04.3 LTS"
VERSION_ID="20.04"
VERSION_CODENAME=focal
```

How did this happen? Well, first, Docker searches your local filesystem for the `ubuntu:20.04` image. If you don't have that image downloaded already, Docker downloads it automatically from Docker Hub, which is a *Docker Registry* that contains shared Docker images. The `ubuntu:20.04` image happens to be a public Docker image—an official one maintained by the Docker team—so you're able to download it without any authentication. However, it's also possible to create private Docker images that only certain authenticated users can use.

Once the image is downloaded, Docker runs the image, executing the `bash` command, which starts an interactive Bash prompt, where you can type. Try running the `ls` command to see the list of files:

```
root@d96ad3779966:/# ls -al
total 56
drwxr-xr-x  1 root root 4096 Feb 22 14:22 .
drwxr-xr-x  1 root root 4096 Feb 22 14:22 ..
lrwxrwxrwx  1 root root    7 Jan 13 16:59 bin -> usr/bin
drwxr-xr-x  2 root root 4096 Apr 15  2020 boot
drwxr-xr-x  5 root root  360 Feb 22 14:22 dev
drwxr-xr-x  1 root root 4096 Feb 22 14:22 etc
drwxr-xr-x  2 root root 4096 Apr 15  2020 home
lrwxrwxrwx  1 root root    7 Jan 13 16:59 lib -> usr/lib
drwxr-xr-x  2 root root 4096 Jan 13 16:59 media
(...)
```

You might notice that's not your filesystem. That's because Docker images run in containers that are isolated at the userspace level: when you're in a container, you can only see the filesystem, memory, networking, etc., in that container. Any data in other containers, or on the underlying host operating system, is not accessible to you, and any data in your container is not visible to those other containers or the underlying host operating system. This is one of the things that makes Docker useful for running applications: the image format is self-contained, so Docker images run the same way no matter where you run them, and no matter what else is running there.

To see this in action, write some text to a `test.txt` file as follows:

```
root@d96ad3779966:/# echo "Hello, World!" > test.txt
```

Next, exit the container by hitting Ctrl-D on Windows and Linux or Cmd-D on macOS, and you should be back in your original command prompt on your underlying host OS. If you try to look for the *test.txt* file you just wrote, you'll see that it doesn't exist: the container's filesystem is totally isolated from your host OS.

Now, try running the same Docker image again:

```
$ docker run -it ubuntu:20.04 bash
root@3e0081565a5d:/#
```

Notice that this time, since the *ubuntu:20.04* image is already downloaded, the container starts almost instantly. This is another reason Docker is useful for running applications: unlike virtual machines, containers are lightweight, boot up quickly, and incur little CPU or memory overhead.

You may also notice that the second time you fired up the container, the command prompt looked different. That's because you're now in a totally new container; any data you wrote in the previous one is no longer accessible to you. Run `ls -al` and you'll see that the *test.txt* file does not exist. Containers are isolated not only from the host OS but also from each other.

Hit Ctrl-D or Cmd-D again to exit the container, and back on your host OS, run the `docker ps -a` command:

```
$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
3e0081565a5d	ubuntu:20.04	"bash"	5 min ago	Exited (0) 16 sec ago
d96ad3779966	ubuntu:20.04	"bash"	14 min ago	Exited (0) 5 min ago

This will show you all the containers on your system, including the stopped ones (the ones you exited). You can start a stopped container again by using the `docker start <ID>` command, setting ID to an ID from the `CONTAINER ID` column of the `docker ps` output. For example, here is how you can start the first container up again (and attach an interactive prompt to it via the `-ia` flags):

```
$ docker start -ia d96ad3779966
root@d96ad3779966:/#
```

You can confirm this is really the first container by outputting the contents of *test.txt*:

```
root@d96ad3779966:/# cat test.txt
Hello, World!
```

Let's now see how a container can be used to run a web app. Hit Ctrl-D or Cmd-D again to exit the container, and back on your host OS, run a new container:

```
$ docker run training/webapp
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
```

The **training/webapp** image contains a simple Python "Hello, World" web app for testing. When you run the image, it fires up the web app, listening on port 5000 by

default. However, if you open a new terminal on your host operating system and try to access the web app, it won't work:

```
$ curl localhost:5000
curl: (7) Failed to connect to localhost port 5000: Connection refused
```

What's the problem? Actually, it's not a problem but a feature! Docker containers are isolated from the host operating system and other containers, not only at the filesystem level but also in terms of networking. So while the container really is listening on port 5000, that is only on a port *inside* the container, which isn't accessible on the host OS. If you want to expose a port from the container on the host OS, you have to do it via the `-p` flag.

First, hit Ctrl-C to shut down the `training/webapp` container: note that it's C this time, not D, and it's Ctrl regardless of OS, as you're shutting down a process, rather than exiting an interactive prompt. Now rerun the container but this time with the `-p` flag as follows:

```
$ docker run -p 5000:5000 training/webapp
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
```

Adding `-p 5000:5000` to the command tells Docker to expose port 5000 inside the container on port 5000 of the host OS. In another terminal on your host OS, you should now be able to see the web app working:

```
$ curl localhost:5000
Hello world!
```



Cleaning Up Containers

Every time you run `docker run` and exit, you are leaving behind containers, which take up disk space. You may wish to clean them up with the `docker rm <CONTAINER_ID>` command, where `CONTAINER_ID` is the ID of the container from the `docker ps` output. Alternatively, you could include the `--rm` flag in your `docker run` command to have Docker automatically clean up when you exit the container.

A Crash Course on Kubernetes

Kubernetes is an orchestration tool for Docker, which means it's a platform for running and managing Docker containers on your servers, including scheduling (picking which servers should run a given container workload), auto healing (automatically redeploying containers that failed), auto scaling (scaling the number of containers up and down in response to load), load balancing (distributing traffic across containers), and much more.

Under the hood, Kubernetes consists of two main pieces:

Control plane

The control plane is responsible for managing the Kubernetes cluster. It is the “brains” of the operation, responsible for storing the state of the cluster, monitoring containers, and coordinating actions across the cluster. It also runs the API server, which provides an API you can use from command-line tools (e.g., `kubectl`), web UIs (e.g., the Kubernetes Dashboard), and IaC tools (e.g., Terraform) to control what’s happening in the cluster.

Worker nodes

The worker nodes are the servers used to actually run your containers. The worker nodes are entirely managed by the control plane, which tells each worker node what containers it should run.

Kubernetes is open source, and one of its strengths is that you can run it anywhere: in any public cloud (e.g., AWS, Azure, Google Cloud), in your own datacenter, and even on your own developer workstation. A little later in this chapter, I’ll show you how you can run Kubernetes in the cloud (in AWS), but for now, let’s start small and run it locally. This is easy to do if you installed a relatively recent version of Docker Desktop, as it has the ability to fire up a Kubernetes cluster locally with just a few clicks.

If you open Docker Desktop’s preferences on your computer, you should see Kubernetes in the nav, as shown in [Figure 7-7](#).

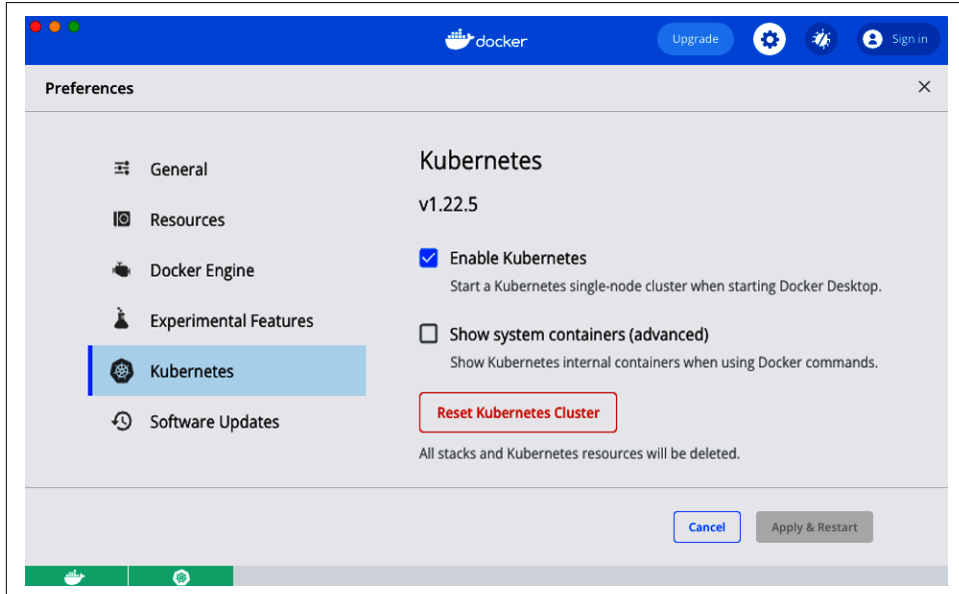


Figure 7-7. Enable Kubernetes on Docker Desktop.

If it's not enabled already, check the Enable Kubernetes checkbox, click Apply & Restart, and wait a few minutes for that to complete. In the meantime, follow the instructions on the [Kubernetes website](#) to install `kubectl`, which is the command-line tool for interacting with Kubernetes.

To use `kubectl`, you must first update its configuration file, which lives in `$HOME/.kube/config` (that is, the `.kube` folder of your home directory), to tell it what Kubernetes cluster to connect to. Conveniently, when you enable Kubernetes in Docker Desktop, it updates this config file for you, adding a `docker-desktop` entry to it, so all you need to do is tell `kubectl` to use this configuration as follows:

```
$ kubectl config use-context docker-desktop
Switched to context "docker-desktop".
```

Now you can check if your Kubernetes cluster is working with the `get nodes` command:

```
$ kubectl get nodes
NAME                STATUS    ROLES                  AGE   VERSION
docker-desktop      Ready    control-plane,master   95m   v1.22.5
```

The `get nodes` command shows you information about all the nodes in your cluster. Since you're running Kubernetes locally, your computer is the only node, and it's running both the control plane and acting as a worker node. You're now ready to run some Docker containers!

To deploy something in Kubernetes, you create Kubernetes *objects*, which are persistent entities you write to the Kubernetes cluster (via the API server) that record your intent: e.g., your intent to have specific Docker images running. The cluster runs a *reconciliation loop*, which continuously checks the objects you stored in it and works to make the state of the cluster match your intent.

There are many different types of Kubernetes objects available. For the examples in this book, let's use the following two objects:

Kubernetes Deployment

A *Kubernetes Deployment* is a declarative way to manage an application in Kubernetes. You declare what Docker images to run, how many copies of them to run (called *replicas*), a variety of settings for those images (e.g., CPU, memory, port numbers, environment variables), and the strategy to roll out updates to those images, and the Kubernetes Deployment will then work to ensure that the requirements you declared are always met. For example, if you specified you wanted three replicas, but one of the worker nodes went down so only two replicas are left, the Deployment will automatically spin up a third replica on one of the other worker nodes.

Kubernetes Service

A *Kubernetes Service* is a way to expose a web app running in Kubernetes as a networked service. For example, you can use a Kubernetes Service to configure a load balancer that exposes a public endpoint and distributes traffic from that endpoint across the replicas in a Kubernetes Deployment.

The idiomatic way to interact with Kubernetes is to create YAML files describing what you want—e.g., one YAML file that defines the Kubernetes Deployment and another one that defines the Kubernetes Service—and to use the `kubectl` `apply` command to submit those objects to the cluster. However, using raw YAML has drawbacks, such as a lack of support for code reuse (e.g., variables, modules), abstraction (e.g., loops, if-statements), clear standards on how to store and manage the YAML files (e.g., to track changes to the cluster over time), and so on. Therefore, many Kubernetes users turn to alternatives, such as Helm or Terraform. Since this is a book on Terraform, I'm going to show you how to create a Terraform module called `k8s-app` (K8S is an acronym for Kubernetes in the same way that I18N is an acronym for internationalization) that deploys an app in Kubernetes using a Kubernetes Deployment and Kubernetes Service.

Create a new module in the `modules/services/k8s-app` folder. Within that folder, create a `variables.tf` file that defines the module's API via the following input variables:

```
variable "name" {
  description = "The name to use for all resources created by this module"
  type       = string
}

variable "image" {
  description = "The Docker image to run"
  type       = string
}

variable "container_port" {
  description = "The port the Docker image listens on"
  type       = number
}

variable "replicas" {
  description = "How many replicas to run"
  type       = number
}

variable "environment_variables" {
  description = "Environment variables to set for the app"
  type       = map(string)
  default    = {}
}
```

This should give you just about all the inputs you need for creating the Kubernetes Deployment and Service. Next, add a *main.tf* file, and at the top, add the `required_providers` block to it with the Kubernetes provider:

```
terraform {
  required_version = ">= 1.0.0, < 2.0.0"

  required_providers {
    kubernetes = {
      source = "hashicorp/kubernetes"
      version = "~> 2.0"
    }
  }
}
```

Hey, a new provider, neat! OK, let's make use of that provider to create a Kubernetes Deployment by using the `kubernetes_deployment` resource:

```
resource "kubernetes_deployment" "app" {
}
```

There are quite a few settings to configure within the `kubernetes_deployment` resource, so let's go through them one at a time. First, you need to configure the `metadata` block:

```
resource "kubernetes_deployment" "app" {
  metadata {
    name = var.name
  }
}
```

Every Kubernetes object includes metadata that can be used to identify and target that object in API calls. In the preceding code, I'm setting the Deployment name to the `name` input variable.

The rest of the configuration for the `kubernetes_deployment` resource goes into the `spec` block:

```
resource "kubernetes_deployment" "app" {
  metadata {
    name = var.name
  }

  spec {
  }
}
```

The first item to put into the `spec` block is to specify the number of replicas to create:

```
spec {
  replicas = var.replicas
}
```


Next, define the `template` block:

```
spec {
  replicas = var.replicas

  template {
  }
}
```

In Kubernetes, instead of deploying one container at a time, you deploy *Pods*, which are groups of containers that are meant to be deployed together. For example, you could have a Pod with one container to run a web app (e.g., the Python app you saw earlier) and another container that gathers metrics on the web app and sends them to a central service (e.g., Datadog). The `template` block is where you define the *Pod Template*, which specifies what container(s) to run, the ports to use, environment variables to set, and so on.

One important ingredient in the Pod Template will be the labels to apply to the Pod. You'll need to reuse these labels in several places—e.g., the Kubernetes Service uses labels to identify the Pods that need to be load balanced—so let's define those labels in a local variable called `pod_labels`:

```
locals {
  pod_labels = {
    app = var.name
  }
}
```

And now use `pod_labels` in the `metadata` block of the Pod Template:

```
spec {
  replicas = var.replicas

  template {
    metadata {
      labels = local.pod_labels
    }
  }
}
```

Next, add a `spec` block inside of `template`:

```
spec {
  replicas = var.replicas

  template {
    metadata {
      labels = local.pod_labels
    }

    spec {
      container {
```

```

    name = var.name
    image = var.image

    port {
        container_port = var.container_port
    }

    dynamic "env" {
        for_each = var.environment_variables
        content {
            name = env.key
            value = env.value
        }
    }
}
}
}
}
}

```

There's a lot here, so let's go through it one piece at a time:

container

Inside the `spec` block, you can define one or more `container` blocks to specify which Docker containers to run in this Pod. To keep this example simple, there's just one `container` block in the Pod. The rest of these items are all within this `container` block.

name

The name to use for the container. I've set this to the `name` input variable.

image

The Docker image to run in the container. I've set this to the `image` input variable.

port

The ports to expose in the container. To keep the code simple, I'm assuming the container only needs to listen on one port, set to the `container_port` input variable.

env

The environment variables to expose to the container. I'm using a `dynamic` block with `for_each` (two concepts you may remember from [Chapter 5](#)) to set this to the variables in the `environment_variables` input variable.

OK, that wraps up the Pod Template. There's just one thing left to add to the `kubernetes_deployment` resource—a `selector` block:

```

spec {
    replicas = var.replicas

```

```

template {
  metadata {
    labels = local.pod_labels
  }

  spec {
    container {
      name = var.name
      image = var.image

      port {
        container_port = var.container_port
      }

      dynamic "env" {
        for_each = var.environment_variables
        content {
          name = env.key
          value = env.value
        }
      }
    }
  }
}

selector {
  match_labels = local.pod_labels
}
}

```

The selector block tells the Kubernetes Deployment what to target. By setting it to `pod_labels`, you are telling it to manage deployments for the Pod Template you just defined. Why doesn't the Deployment just assume that the Pod Template defined within that Deployment is the one you want to target? Well, Kubernetes tries to be an extremely flexible and decoupled system: e.g., it's possible to define a Deployment for Pods that are defined separately, so you always need to specify a selector to tell the Deployment what to target.

That wraps up the `kubernetes_deployment` resource. The next step is to use the `kubernetes_service` resource to create a Kubernetes Service:

```

resource "kubernetes_service" "app" {
  metadata {
    name = var.name
  }

  spec {
    type = "LoadBalancer"
    port {
      port = 80
    }
  }
}

```

```

        target_port = var.container_port
        protocol     = "TCP"
    }
    selector = local.pod_labels
}
}

```

Let's go through these parameters:

metadata

Just as with the Deployment object, the Service object uses metadata to identify and target that object in API calls. In the preceding code, I've set the Service name to the name input variable.

type

I've configured this Service as type `LoadBalancer`, which, depending on how your Kubernetes cluster is configured, will deploy a different type of load balancer: e.g., in AWS, with EKS, you might get an Elastic Load Balancer, whereas in Google Cloud, with GKE, you might get a Cloud Load Balancer.

port

I'm configuring the load balancer to route traffic on port 80 (the default port for HTTP) to the port the container is listening on.

selector

Just as with the Deployment object, the Service object uses a selector to specify what that Service should be targeting. By setting the selector to `pod_labels`, the Service and the Deployment will both operate on the same Pods.

The final step is to expose the Service endpoint (the load balancer hostname) as an output variable in *outputs.tf*:

```

locals {
    status = kubernetes_service.app.status
}

output "service_endpoint" {
    value = try(
        "http://${local.status[0]["load_balancer"]}[0]["ingress"][0]["hostname"]",
        "(error parsing hostname from status)"
    )
    description = "The K8S Service endpoint"
}

```

This convoluted code needs a bit of explanation. The `kubernetes_service` resource has an output attribute called `status` that returns the latest status of the Service. I've stored this attribute in a local variable called `status`. For a Service of type `LoadBalancer`, `status` will contain a complicated object that looks something like this:

```
[
  {
    load_balancer = [
      {
        ingress = [
          {
            hostname = "<HOSTNAME>"
          }
        ]
      }
    ]
  }
]
```

Buried within this deeply nested object is the hostname for the load balancer that you want. This is why the `service_endpoint` output variable needs to use a complicated sequence of array lookups (e.g., `[0]`) and map lookups (e.g., `["load_balancer"]`) to extract the hostname. But what happens if the `status` attribute returned by the `kubernetes_service` resource happens to look a little different? In that case, any of those array and map lookups could fail, leading to a confusing error.

To handle this error gracefully, I’ve wrapped the entire expression in a function called `try`. The `try` function has the following syntax:

```
try(ARG1, ARG2, ..., ARGN)
```

This function evaluates all the arguments you pass to it and returns the first argument that doesn’t produce any errors. Therefore, the `service_endpoint` output variable will either end up with a hostname in it (the first argument) or, if reading the hostname caused an error, the variable will instead say “error parsing hostname from status” (the second argument).

OK, that wraps up the `k8s-app` module. To use it, add a new example in *examples/kubernetes-local*, and create a *main.tf* file in it with the following contents:

```
module "simple_webapp" {
  source = "../modules/services/k8s-app"

  name           = "simple-webapp"
  image          = "training/webapp"
  replicas       = 2
  container_port = 5000
}
```

This configures the module to deploy the `training/webapp` Docker image you ran earlier, with two replicas listening on port 5000, and to name all the Kubernetes objects (based on their metadata) “simple-webapp”. To have this module deploy into your local Kubernetes cluster, add the following provider block:

```
provider "kubernetes" {
  config_path = "~/.kube/config"
}
```

```
    config_context = "docker-desktop"
}
```

This code tells the Kubernetes provider to authenticate to your local Kubernetes cluster by using the `docker-desktop` context from your `kubectl` config. Run `terraform apply` to see how it works:

```
$ terraform apply

(...)

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

service_endpoint = "http://localhost"
```

Give the app a few seconds to boot and then try out that `service_endpoint`:

```
$ curl http://localhost
Hello world!
```

Success!

That said, this looks nearly identical to the output of the `docker run` command, so was all that extra work worth it? Well, let's look under the hood to see what's going on. You can use `kubectl` to explore your cluster. First, run the `get deployments` command:

```
$ kubectl get deployments
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
simple-webapp  2/2     2            2           3m21s
```

You can see your Kubernetes Deployment, named `simple-webapp`, as that was the name in the `metadata` block. This Deployment is reporting that 2/2 Pods (the two replicas) are ready. To see those Pods, run the `get pods` command:

```
$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
simple-webapp-d45b496fd-7d447        1/1     Running   0           2m36s
simple-webapp-d45b496fd-vl6j7        1/1     Running   0           2m36s
```

So that's one difference from `docker run` already: there are multiple containers running here, not just one. Moreover, those containers are being actively monitored and managed. For example, if one crashed, a replacement will be deployed automatically. You can see this in action by running the `docker ps` command:

```
$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED          STATUS
b60f5147954a   training/webapp   "python app.py"         3 seconds ago   Up 2 seconds
c350ec648185   training/webapp   "python app.py"         12 minutes ago  Up 12 minutes
```

Grab the `CONTAINER ID` of one of those containers, and use the `docker kill` command to shut it down:

```
$ docker kill b60f5147954a
```

If you run `docker ps` again very quickly, you'll see just one container left running:

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
c350ec648185   training/webapp "python app.py"         12 minutes ago Up 12 minutes
```

But just a few seconds later, the Kubernetes Deployment will have detected that there is only one replica instead of the requested two, and it'll launch a replacement container automatically:

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
56a216b8a829   training/webapp "python app.py"         1 second ago   Up 5 seconds
c350ec648185   training/webapp "python app.py"         12 minutes ago Up 12 minutes
```

So Kubernetes is ensuring that you always have the expected number of replicas running. Moreover, it is also running a load balancer to distribute traffic across those replicas, which you can see by running the `kubectl get services` command:

```
$ kubectl get services
NAME            TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes      ClusterIP     10.96.0.1    <none>        443/TCP          4h26m
simple-webapp    LoadBalancer 10.110.25.79 localhost     80:30234/TCP     4m58s
```

The first service in the list is Kubernetes itself, which you can ignore. The second is the Service you created, also with the name `simple-webapp` (based on the metadata block). This service runs a load balancer for your app: you can see the IP it's accessible at (`localhost`) and the port it's listening on (`80`).

Kubernetes Deployments also provide automatic rollout of updates. A fun trick with the `training/webapp` Docker image is that if you set the environment variable `PROVIDER` to some value, it'll use that value instead of the word *world* in the text "Hello, world!" Update `examples/kubernetes-local/main.tf` to set this environment variable as follows:

```
module "simple_webapp" {
  source = "../../modules/services/k8s-app"

  name           = "simple-webapp"
  image          = "training/webapp"
  replicas       = 2
  container_port = 5000

  environment_variables = {
    PROVIDER = "Terraform"
  }
}
```

Run `apply` one more time:

```
$ terraform apply

(...)

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:
```

```
service_endpoint = "http://localhost"
```

After a few seconds, try the endpoint again:

```
$ curl http://localhost
Hello Terraform!
```

And there you go, the Deployment has rolled out your change automatically: under the hood, Deployments do a rolling deployment by default, similar to what you saw with Auto Scaling Groups (note that you can change deployment settings by adding a strategy block to the `kubernetes_deployment` resource).

Deploying Docker Containers in AWS Using Elastic Kubernetes Service

Kubernetes has one more trick up its sleeve: it's fairly portable. That is, you can reuse both the Docker images and the Kubernetes configurations in a totally different cluster and get similar results. To see this in action, let's now deploy a Kubernetes cluster in AWS.

Setting up and managing a secure, highly available, scalable Kubernetes cluster in the cloud from scratch is complicated. Fortunately, most cloud providers offer managed Kubernetes services, where they run the control plane and worker nodes for you: e.g., Elastic Kubernetes Service (EKS) in AWS, Azure Kubernetes Service (AKS) in Azure, and Google Kubernetes Engine (GKE) in Google Cloud. I'm going to show you how to deploy a very basic EKS cluster in AWS.

Create a new module in `modules/services/eks-cluster`, and define the API for the module in a `variables.tf` file with the following input variables:

```
variable "name" {
  description = "The name to use for the EKS cluster"
  type        = string
}

variable "min_size" {
  description = "Minimum number of nodes to have in the EKS cluster"
  type        = number
}

variable "max_size" {
  description = "Maximum number of nodes to have in the EKS cluster"
```



```

    type          = number
}

variable "desired_size" {
  description = "Desired number of nodes to have in the EKS cluster"
  type        = number
}

variable "instance_types" {
  description = "The types of EC2 instances to run in the node group"
  type        = list(string)
}

```

This code exposes input variables to set the EKS cluster's name, size, and the types of instances to use for the worker nodes. Next, in *main.tf*, create an IAM role for the control plane:

```

# Create an IAM role for the control plane
resource "aws_iam_role" "cluster" {
  name           = "${var.name}-cluster-role"
  assume_role_policy = data.aws_iam_policy_document.cluster_assume_role.json
}

# Allow EKS to assume the IAM role
data "aws_iam_policy_document" "cluster_assume_role" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = ["eks.amazonaws.com"]
    }
  }
}

# Attach the permissions the IAM role needs
resource "aws_iam_role_policy_attachment" "AmazonEKSClusterPolicy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSClusterPolicy"
  role       = aws_iam_role.cluster.name
}

```

This IAM role can be assumed by the EKS service, and it has a Managed IAM Policy attached that gives the control plane the permissions it needs. Now, add the `aws_vpc` and `aws_subnets` data sources to fetch information about the Default VPC and its subnets:

```

# Since this code is only for learning, use the Default VPC and subnets.
# For real-world use cases, you should use a custom VPC and private subnets.

data "aws_vpc" "default" {
  default = true
}

```

```

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

```

Now you can create the control plane for the EKS cluster by using the `aws_eks_cluster` resource:

```

resource "aws_eks_cluster" "cluster" {
  name     = var.name
  role_arn = aws_iam_role.cluster.arn
  version  = "1.21"

  vpc_config {
    subnet_ids = data.aws_subnets.default.ids
  }

  # Ensure that IAM Role permissions are created before and deleted after
  # the EKS Cluster. Otherwise, EKS will not be able to properly delete
  # EKS managed EC2 infrastructure such as Security Groups.
  depends_on = [
    aws_iam_role_policy_attachment.AmazonEKSClusterPolicy
  ]
}

```

The preceding code configures the control plane to use the IAM role you just created, and to deploy into the Default VPC and subnets.

Next up are the worker nodes. EKS supports several different types of worker nodes: self-managed EC2 Instances (e.g., in an ASG that you create), AWS-managed EC2 Instances (known as a *managed node group*), and Fargate (serverless).³ The simplest option to use for the examples in this chapter will be the managed node groups.

To deploy a managed node group, you first need to create another IAM role:

```

# Create an IAM role for the node group
resource "aws_iam_role" "node_group" {
  name           = "${var.name}-node-group"
  assume_role_policy = data.aws_iam_policy_document.node_assume_role.json
}

# Allow EC2 instances to assume the IAM role
data "aws_iam_policy_document" "node_assume_role" {
  statement {
    effect = "Allow"
    actions = ["sts:AssumeRole"]
  }
}

```

³ For a comparison of the different types of EKS worker nodes, see [the Gruntwork blog](#).

```

    principals {
      type       = "Service"
      identifiers = ["ec2.amazonaws.com"]
    }
  }
}

# Attach the permissions the node group needs
resource "aws_iam_role_policy_attachment" "AmazonEKSWorkerNodePolicy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKSWorkerNodePolicy"
  role       = aws_iam_role.node_group.name
}

resource "aws_iam_role_policy_attachment" "AmazonEC2ContainerRegistryReadOnly" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEC2ContainerRegistryReadOnly"
  role       = aws_iam_role.node_group.name
}

resource "aws_iam_role_policy_attachment" "AmazonEKS_CNI_Policy" {
  policy_arn = "arn:aws:iam::aws:policy/AmazonEKS_CNI_Policy"
  role       = aws_iam_role.node_group.name
}

```

This IAM role can be assumed by the EC2 service (which makes sense, as managed node groups use EC2 Instances under the hood), and it has several Managed IAM Policies attached that give the managed node group the permissions it needs. Now you can use the `aws_eks_node_group` resource to create the managed node group itself:

```

resource "aws_eks_node_group" "nodes" {
  cluster_name   = aws_eks_cluster.cluster.name
  node_group_name = var.name
  node_role_arn  = aws_iam_role.node_group.arn
  subnet_ids    = data.aws_subnets.default.ids
  instance_types = var.instance_types

  scaling_config {
    min_size     = var.min_size
    max_size     = var.max_size
    desired_size = var.desired_size
  }

  # Ensure that IAM Role permissions are created before and deleted after
  # the EKS Node Group. Otherwise, EKS will not be able to properly
  # delete EC2 Instances and Elastic Network Interfaces.
  depends_on = [
    aws_iam_role_policy_attachment.AmazonEKSWorkerNodePolicy,
    aws_iam_role_policy_attachment.AmazonEC2ContainerRegistryReadOnly,
    aws_iam_role_policy_attachment.AmazonEKS_CNI_Policy,
  ]
}

```

This code configures the managed node group to use the control plane and IAM role you just created, to deploy into the Default VPC, and to use the name, size, and instance type parameters passed in as input variables.

In *outputs.tf*, add the following output variables:

```
output "cluster_name" {
  value     = aws_eks_cluster.cluster.name
  description = "Name of the EKS cluster"
}

output "cluster_arn" {
  value     = aws_eks_cluster.cluster.arn
  description = "ARN of the EKS cluster"
}

output "cluster_endpoint" {
  value     = aws_eks_cluster.cluster.endpoint
  description = "Endpoint of the EKS cluster"
}

output "cluster_certificate_authority" {
  value     = aws_eks_cluster.cluster.certificate_authority
  description = "Certificate authority of the EKS cluster"
}
```

OK, the `eks-cluster` module is now ready to roll. Let's use it and the `k8s-app` module from earlier to deploy an EKS cluster and to deploy the training/webapp Docker image into that cluster. Create *examples/kubernetes-eks/main.tf*, and configure the `eks-cluster` module as follows:

```
provider "aws" {
  region = "us-east-2"
}

module "eks_cluster" {
  source = "../modules/services/eks-cluster"

  name           = "example-eks-cluster"
  min_size       = 1
  max_size       = 2
  desired_size    = 1

  # Due to the way EKS works with ENIs, t3.small is the smallest
  # instance type that can be used for worker nodes. If you try
  # something smaller like t2.micro, which only has 4 ENIs,
  # they'll all be used up by system services (e.g., kube-proxy)
  # and you won't be able to deploy your own Pods.
  instance_types = ["t3.small"]
}
```

Next, configure the `k8s-app` module as follows:

```

provider "kubernetes" {
  host = module.eks_cluster.cluster_endpoint
  cluster_ca_certificate = base64decode(
    module.eks_cluster.cluster_certificate_authority[0].data
  )
  token = data.aws_eks_cluster_auth.cluster.token
}

data "aws_eks_cluster_auth" "cluster" {
  name = module.eks_cluster.cluster_name
}

module "simple_webapp" {
  source = "../../modules/services/k8s-app"

  name           = "simple-webapp"
  image          = "training/webapp"
  replicas       = 2
  container_port = 5000

  environment_variables = {
    PROVIDER = "Terraform"
  }

  # Only deploy the app after the cluster has been deployed
  depends_on = [module.eks_cluster]
}

```

The preceding code configures the Kubernetes provider to authenticate to the EKS cluster, rather than your local Kubernetes cluster (from Docker Desktop). It then uses the `k8s-app` module to deploy the `training/webapp` Docker image exactly the same way as you did when deploying it to Docker Desktop; the only difference is the addition of the `depends_on` parameter to ensure that Terraform only tries to deploy the Docker image after the EKS cluster has been deployed.

Next, pass through the service endpoint as an output variable:

```

output "service_endpoint" {
  value       = module.simple_webapp.service_endpoint
  description = "The K8S Service endpoint"
}

```

OK, now you're ready to deploy! Run `terraform apply` as usual (note that EKS clusters can take 10–20 minutes to deploy, so be patient):

```
$ terraform apply
```

```
(...)
```

```
Apply complete! Resources: 10 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
service_endpoint = "http://774696355.us-east-2.elb.amazonaws.com"
```

Wait a little while for the web app to spin up and pass health checks, and then test out the `service_endpoint`:

```
$ curl http://774696355.us-east-2.elb.amazonaws.com
Hello Terraform!
```

And there you have it! The same Docker image and Kubernetes code is now running in an EKS cluster in AWS, just the way it ran on your local computer. All the same features work here too. For example, try updating `environment_variables` to a different `PROVIDER` value, such as “Readers”:

```
module "simple_webapp" {
  source = "../../modules/services/k8s-app"

  name          = "simple-webapp"
  image         = "training/webapp"
  replicas      = 2
  container_port = 5000

  environment_variables = {
    PROVIDER = "Readers"
  }

  # Only deploy the app after the cluster has been deployed
  depends_on = [module.eks_cluster]
}
```

Rerun `apply`, and just a few seconds later, the Kubernetes Deployment will have deployed the changes:

```
$ curl http://774696355.us-east-2.elb.amazonaws.com
Hello Readers!
```

This is one of the advantages of using Docker: changes can be deployed very quickly.

You can use `kubectl` again to see what’s happening in your cluster. To authenticate `kubectl` to the EKS cluster, you can use the `aws eks update-kubeconfig` command to automatically update your `$HOME/.kube/config` file:

```
$ aws eks update-kubeconfig --region <REGION> --name <EKS_CLUSTER_NAME>
```

where `REGION` is the AWS region and `EKS_CLUSTER_NAME` is the name of your EKS cluster. In the Terraform module, you deployed to the `us-east-2` region and named the cluster `kubernetes-example`, so the command will look like this:

```
$ aws eks update-kubeconfig --region us-east-2 --name kubernetes-example
```

Now, just as before, you can use the `get nodes` command to inspect the worker nodes in your cluster, but this time, add the `-o wide` flag to get a bit more info:

```
$ kubectl get nodes
```

NAME	STATUS	AGE	EXTERNAL-IP	OS-IMAGE
xxx.us-east-2.compute.internal	Ready	22m	3.134.78.187	Amazon Linux 2

The preceding snippet is highly truncated to fit into the book, but in the real output, you should be able to see the one worker node, its internal and external IP, version information, OS information, and much more.

You can use the `get deployments` command to inspect your Deployments:

```
$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
simple-webapp	2/2	2	2	19m

Next, run `get pods` to see the Pods:

```
$ kubectl get pods
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
simple-webapp	2/2	2	2	19m

And finally, run `get services` to see the Services:

```
$ kubectl get services
```

NAME	TYPE	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	<none>	443/TCP
simple-webapp	LoadBalancer	774696355.us-east-2.elb.amazonaws.com	80/TCP

You should be able to see your load balancer and the URL you used to test it.

So there you have it: two different providers, both working in the same cloud, helping you to deploy containerized workloads.

That said, just as in previous sections, I want to leave you with a few warnings:

Warning 1: These Kubernetes examples are very simplified!

Kubernetes is complicated, and it's rapidly evolving and changing; trying to explain all the details can easily fill a book all by itself. Since this is a book about Terraform, and not Kubernetes, my goal with the Kubernetes examples in this chapter was to keep them as simple and minimal as possible. Therefore, while I hope the code examples you've seen have been useful from a learning and experimentation perspective, if you are going to use Kubernetes for real-world, production use cases, you'll need to change many aspects of this code, such as configuring a number of additional services and settings in the `eks-cluster` module (e.g., ingress controllers, secret envelope encryption, security groups, OIDC authentication, Role-Based Access Control (RBAC) mapping, VPC CNI, kube-proxy, CoreDNS), exposing many other settings in the `k8s-app` module (e.g., secrets management, volumes, liveness probes, readiness probes, labels, annotations, multiple ports, multiple containers), and using a custom VPC with

private subnets for your EKS cluster instead of the Default VPC and public subnets.⁴

Warning 2: Use multiple providers sparingly

Although you certainly can use multiple providers in a single module, I don't recommend doing it too often, for similar reasons to why I don't recommend using provider aliases too often: in most cases, you want each provider to be isolated in its own module so that you can manage it separately and limit the blast radius from mistakes or attackers.

Moreover, Terraform doesn't have great support for dependency ordering between providers. For example, in the Kubernetes example, you had a single module that deployed both the EKS cluster, using the AWS Provider, and a Kubernetes app into that cluster, using the Kubernetes provider. As it turns out, the [Kubernetes provider documentation](#) explicitly recommends *against* this pattern:

When using interpolation to pass credentials to the Kubernetes provider from other resources, these resources SHOULD NOT be created in the same Terraform module where Kubernetes provider resources are also used. This will lead to intermittent and unpredictable errors which are hard to debug and diagnose. The root issue lies with the order in which Terraform itself evaluates the provider blocks vs. actual resources.

The example code in this book is able to work around these issues by depending on the `aws_eks_cluster_auth` data source, but that's a bit of a hack. Therefore, in production code, I always recommend deploying the EKS cluster in one module and deploying Kubernetes apps in separate modules, after the cluster has been deployed.

Conclusion

At this point, you hopefully understand how to work with multiple providers in Terraform code, and you can answer the three questions from the beginning of this chapter:

What if you need to deploy to multiple AWS regions?

Use multiple provider blocks, each configured with a different `region` and `alias` parameter.

⁴ Alternatively, you can use off-the-shelf production-grade Kubernetes modules, such as the ones in the [Gruntwork Infrastructure as Code Library](#).

What if you need to deploy to multiple AWS accounts?

Use multiple `provider` blocks, each configured with a different `assume_role` block and an `alias` parameter.

What if you need to deploy to other clouds, such as Azure or GCP or Kubernetes?

Use multiple `provider` blocks, each configured for its respective cloud.

However, you’ve also seen that using multiple providers in one module is typically an antipattern. So the real answer to these questions, especially in real-world, production use cases, is to use each provider in a separate module to keep different regions, accounts, and clouds isolated from one another, and to limit your blast radius.

Let’s now move on to [Chapter 8](#), where I’ll go over several other patterns for how to build Terraform modules for real-world, production use cases—the kind of modules you could bet your company on.

Production-Grade Terraform Code

Building production-grade infrastructure is difficult. And stressful. And time consuming. By *production-grade infrastructure*, I mean the kind of infrastructure you'd bet your company on. You're betting that your infrastructure won't fall over if traffic goes up, or lose your data if there's an outage, or allow that data to be compromised when hackers try to break in—and if that bet doesn't work out, your company might go out of business. That's what's at stake when I refer to production-grade infrastructure in this chapter.

I've had the opportunity to work with hundreds of companies, and based on all of these experiences, here's roughly how long you should expect your next production-grade infrastructure project to take:

- If you want to deploy a service fully managed by a third party, such as running MySQL using the AWS Relational Database Service (RDS), you can expect it to take you one to two weeks to get that service ready for production.
- If you want to run your own stateless distributed app, such as a cluster of Node.js apps that don't store any data locally (e.g., they store all their data in RDS) running on top of an AWS Auto Scaling Group (ASG), that will take roughly twice as long, or about two to four weeks to get ready for production.
- If you want to run your own stateful distributed app, such as an Elasticsearch cluster that runs on top of an ASG and stores data on local disks, that will be another order-of-magnitude increase, or about two to four months to get ready for production.
- If you want to build out your entire architecture, including all of your apps, data stores, load balancers, monitoring, alerting, security, and so on, that's another order-of-magnitude (or two) increase, or about 6 to 36 months of work, with

small companies typically being closer to six months and larger companies typically taking several years.

Table 8-1 shows a summary of this data.

Table 8-1. How long it takes to build production-grade infrastructure from scratch

Type of infrastructure	Example	Time estimate
Managed service	Amazon RDS	1–2 weeks
Self-managed distributed system (stateless)	A cluster of Node.js apps in an ASG	2–4 weeks
Self-managed distributed system (stateful)	Elasticsearch cluster	2–4 months
Entire architecture	Apps, data stores, load balancers, monitoring, etc.	6–36 months

If you haven't gone through the process of building out production-grade infrastructure, you may be surprised by these numbers. I often hear reactions like, "How can it possibly take that long?" or "I can deploy a server on <cloud> in a few minutes. Surely it can't take months to get the rest done!" And all too often, from many an overconfident engineer, "I'm sure those numbers apply to other people, but *I* will be able to get this done in a few days."

And yet, anyone who has gone through a major cloud migration or assembled a brand-new infrastructure from scratch knows that these numbers, if anything, are optimistic—a best-case scenario, really. If you don't have people on your team with deep expertise in building production-grade infrastructure, or if your team is being pulled in a dozen different directions and you can't find the time to focus on it, it might take you significantly longer.

In this chapter, I'll go over why it takes so long to build production-grade infrastructure, what production grade really means, and what patterns work best for creating reusable, production-grade modules:

- Why it takes so long to build production-grade infrastructure
- The production-grade infrastructure checklist
- Production-grade infrastructure modules
 - Small modules
 - Composable modules
 - Testable modules
 - Versioned modules
 - Beyond Terraform modules



Example Code

As a reminder, you can find all of the code examples in the book on [GitHub](#).

Why It Takes So Long to Build Production-Grade Infrastructure

Time estimates for software projects are notoriously inaccurate. Time estimates for DevOps projects, doubly so. That quick tweak that you thought would take five minutes takes up the entire day; the minor new feature that you estimated at a day of work takes two weeks; the app that you thought would be in production in two weeks is still not quite there six months later. Infrastructure and DevOps projects, perhaps more than any other type of software, are the ultimate examples of Hofstadter’s Law:¹

Hofstadter’s Law: It always takes longer than you expect, even when you take into account Hofstadter’s Law.

I think there are three major reasons for this. The first reason is that DevOps, as an industry, is still in the Stone Age. I don’t mean that as an insult but rather in the sense that the industry is still in its infancy. The terms “cloud computing,” “infrastructure as code,” and “DevOps” only appeared in the mid- to late-2000s, and tools like Terraform, Docker, Packer, and Kubernetes were all initially released in the mid- to late-2010s. All of these tools and techniques are relatively new, and all of them are changing rapidly. This means that they are not particularly mature and few people have deep experience with them, so it’s no surprise that projects take longer than expected.

The second reason is that DevOps seems to be particularly susceptible to *yak shaving*. If you haven’t heard of “yak shaving” before, I assure you, this is a term that you will grow to love (and hate). The best definition I’ve seen of this term comes from a blog post by Seth Godin:²

“I want to wax the car today.”

“Oops, the hose is still broken from the winter. I’ll need to buy a new one at Home Depot.”

“But Home Depot is on the other side of the Tappan Zee bridge and getting there without my EZPass is miserable because of the tolls.”

“But, wait! I could borrow my neighbor’s EZPass...”

1 Douglas R. Hofstadter, *Gödel, Escher, Bach: An Eternal Golden Braid*, 20th anniversary ed. (New York: Basic Books, 1999).

2 Seth Godin, “Don’t Shave That Yak!” Seth’s Blog, March 5, 2005, <https://bit.ly/2OK45uL>.

“Bob won’t lend me his EZPass until I return the mooshi pillow my son borrowed, though.”

“And we haven’t returned it because some of the stuffing fell out and we need to get some yak hair to restuff it.”

And the next thing you know, you’re at the zoo, shaving a yak, all so you can wax your car.

Yak shaving consists of all the tiny, seemingly unrelated tasks you must do before you can do the task you originally wanted to do. If you develop software, and especially if you work in the DevOps industry, you’ve probably seen this sort of thing a thousand times. You go to deploy a fix for a small typo, only to uncover a bug in your app configuration. You try to deploy a fix for the app configuration, but that’s blocked by a TLS certificate issue. After spending hours on Stack Overflow, you try to roll out a fix for the TLS issue, but that fails due to a problem with your deployment system. You spend hours digging into that problem and find out it’s due to an out-of-date Linux version. The next thing you know, you’re updating the operating system on your entire fleet of servers, all so you can deploy a “quick” one-character typo fix.

DevOps seems to be especially prone to these sorts of yak-shaving incidents. In part, this is a consequence of the immaturity of DevOps technologies and modern system design, which often involves lots of tight coupling and duplication in the infrastructure. Every change you make in the DevOps world is a bit like trying to pull out one wire from a box of tangled wires—it just tends to pull up everything else in the box with it. In part, this is because the term “DevOps” covers an astonishingly broad set of topics: everything from build to deployment to security and so on.

This brings us to the third reason why DevOps work takes so long. The first two reasons—DevOps is in the Stone Age and yak shaving—can be classified as accidental complexity. *Accidental complexity* refers to the problems imposed by the particular tools and processes you’ve chosen, as opposed to *essential complexity*, which refers to the problems inherent in whatever it is that you’re working on.³ For example, if you’re using C++ to write stock-trading algorithms, dealing with memory allocation bugs is accidental complexity: had you picked a different programming language with automatic memory management, you wouldn’t have this as a problem at all. On the other hand, figuring out an algorithm that can beat the market is essential complexity: you’d have to solve this problem no matter what programming language you picked.

³ Frederick P. Brooks Jr., *The Mythical Man-Month: Essays on Software Engineering*, anniversary ed. (Reading, MA: Addison-Wesley Professional, 1995).

The third reason why DevOps takes so long—the essential complexity of this problem—is that there is a genuinely long checklist of tasks that you must do to prepare infrastructure for production. The problem is that the vast majority of developers don’t know about most of the items on the checklist, so when they estimate a project, they forget about a huge number of critical and time-consuming details. This checklist is the focus of the next section.

The Production-Grade Infrastructure Checklist

Here’s a fun experiment: go around your company and ask, “What are the requirements for going to production?” In most companies, if you ask this question to five people, you’ll get five different answers. One person will mention the need for metrics and alerts; another will talk about capacity planning and high availability; someone else will go on a rant about automated tests and code reviews; yet another person will bring up encryption, authentication, and server hardening; and if you’re lucky, someone might remember to bring up data backups and log aggregation. Most companies do not have a clear definition of the requirements for going to production, which means each piece of infrastructure is deployed a little differently and can be missing some critical functionality.

To help improve this situation, I’d like to share with you the *Production-Grade Infrastructure Checklist*, as shown in [Table 8-2](#). This list covers most of the key items that you need to consider to deploy infrastructure to production.

Table 8-2. The Production-Grade Infrastructure Checklist

Task	Description	Example tools
Install	Install the software binaries and all dependencies.	Bash, Ansible, Docker, Packer
Configure	Configure the software at runtime. Includes port settings, TLS certs, service discovery, leaders, followers, replication, etc.	Chef, Ansible, Kubernetes
Provision	Provision the infrastructure. Includes servers, load balancers, network configuration, firewall settings, IAM permissions, etc.	Terraform, CloudFormation
Deploy	Deploy the service on top of the infrastructure. Roll out updates with no downtime. Includes blue-green, rolling, and canary deployments.	ASG, Kubernetes, ECS
High availability	Withstand outages of individual processes, servers, services, datacenters, and regions.	Multi-datacenter, multi-region
Scalability	Scale up and down in response to load. Scale horizontally (more servers) and/or vertically (bigger servers).	Auto scaling, replication
Performance	Optimize CPU, memory, disk, network, and GPU usage. Includes query tuning, benchmarking, load testing, and profiling.	Dynatrace, Valgrind, VisualVM
Networking	Configure static and dynamic IPs, ports, service discovery, firewalls, DNS, SSH access, and VPN access.	VPCs, firewalls, Route 53

Task	Description	Example tools
Security	Encryption in transit (TLS) and on disk, authentication, authorization, secrets management, server hardening.	ACM, Let's Encrypt, KMS, Vault
Metrics	Availability metrics, business metrics, app metrics, server metrics, events, observability, tracing, and alerting.	CloudWatch, Datadog
Logs	Rotate logs on disk. Aggregate log data to a central location.	Elastic Stack, Sumo Logic
Data backup	Make backups of DBs, caches, and other data on a scheduled basis. Replicate to separate region/account.	AWS Backup, RDS snapshots
Cost optimization	Pick proper Instance types, use spot and reserved Instances, use auto scaling, and clean up unused resources.	Auto scaling, Infracost
Documentation	Document your code, architecture, and practices. Create playbooks to respond to incidents.	READMEs, wikis, Slack, IaC
Tests	Write automated tests for your infrastructure code. Run tests after every commit and nightly.	Terratest, tfint, OPA, InSpec

Most developers are aware of the first few tasks: install, configure, provision, and deploy. It's all the ones that come after them that catch people off guard. For example, did you think through the resilience of your service and what happens if a server goes down? Or a load balancer goes down? Or an entire datacenter goes dark? Networking tasks are also notoriously tricky: setting up VPCs, VPNs, service discovery, and SSH access are all essential tasks that can take months and yet are often entirely left out of many project plans and time estimates. Security tasks, such as encrypting data in transit using TLS, dealing with authentication, and figuring out how to store secrets, are also often forgotten until the last minute.

Every time you're working on a new piece of infrastructure, go through this checklist. Not every single piece of infrastructure needs every single item on the list, but you should consciously and explicitly document which items you've implemented, which ones you've decided to skip, and why.

Production-Grade Infrastructure Modules

Now that you know the list of tasks that you need to do for each piece of infrastructure, let's talk about the best practices for building reusable modules to implement these tasks. Here are the topics I'll cover:

- Small modules
- Composable modules
- Testable modules
- Versioned modules
- Beyond Terraform modules

Small Modules

Developers who are new to Terraform, and IaC in general, often define all of their infrastructure for all of their environments (dev, stage, prod, etc.) in a single file or single module. As discussed in “[State File Isolation](#)” on [page 93](#), this is a bad idea. In fact, I’ll go even further and make the following claim: large modules—modules that contain more than a few hundred lines of code or that deploy more than a few closely related pieces of infrastructure—should be considered harmful.

Here are just a few of the downsides of large modules:

Large modules are slow

If all of your infrastructure is defined in one Terraform module, running any command will take a long time. I’ve seen modules grow so large that `terraform plan` takes 20 minutes to run!

Large modules are insecure

If all your infrastructure is managed in a single large module, to change anything, you need permissions to access everything. This means that almost every user must be an admin, which goes against the *principle of least privilege*.

Large modules are risky

If all your eggs are in one basket, a mistake anywhere could break everything. You might be making a minor change to a frontend app in staging, but due to a typo or running the wrong command, you delete the production database.

Large modules are difficult to understand

The more code you have in one place, the more difficult it is for any one person to understand it all. And when you don’t understand the infrastructure you’re dealing with, you end up making costly mistakes.

Large modules are difficult to review

Reviewing a module that consists of several dozen lines of code is easy; reviewing a module that consists of several thousand lines of code is nearly impossible. Moreover, `terraform plan` not only takes longer to run, but if the output of the `plan` command is several thousand lines, no one will bother to read it. And that means no one will notice that one little red line that means your database is being deleted.

Large modules are difficult to test

Testing infrastructure code is hard; testing a large amount of infrastructure code is nearly impossible. I’ll come back to this point in [Chapter 9](#).

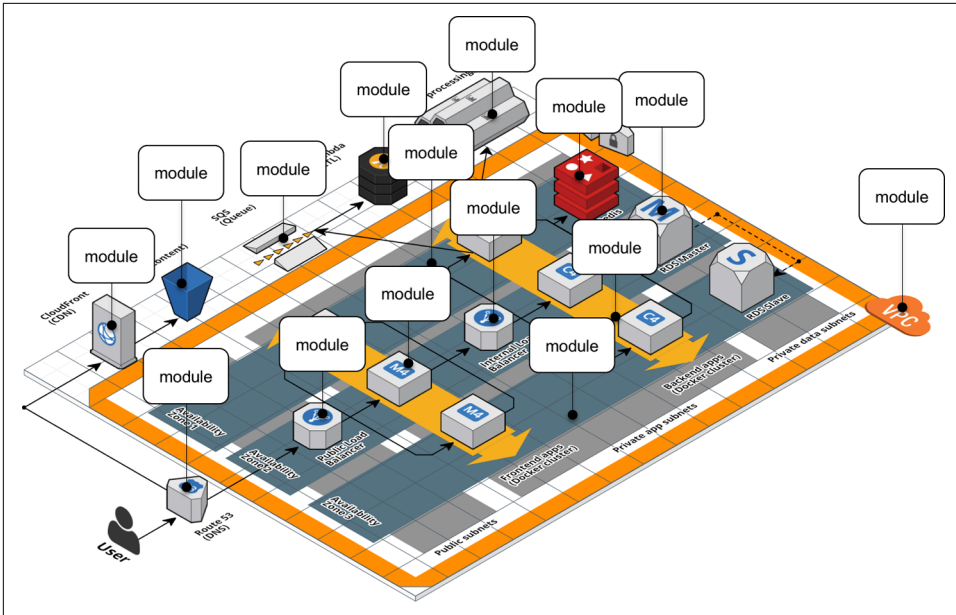


Figure 8-2. A relatively complicated AWS architecture refactored into many small modules.

For example, consider the `webserver-cluster` module, which you last worked on in [Chapter 5](#). This module has become fairly large, as it is handling three somewhat unrelated tasks:

Auto Scaling Group (ASG)

The `webserver-cluster` module deploys an ASG that can do a zero-downtime, rolling deployment.

Application Load Balancer (ALB)

The `webserver-cluster` deploys an ALB.

Hello, World app

The `webserver-cluster` module also deploys a simple “Hello, World” app.

Let’s refactor the code accordingly into three smaller modules:

`modules/cluster/asg-rolling-deploy`

A generic, reusable, standalone module for deploying an ASG that can do a zero-downtime, rolling deployment.

`modules/networking/alb`

A generic, reusable, standalone module for deploying an ALB.

`modules/services/hello-world-app`

A module specifically for deploying the “Hello, World” app, which uses the `asg-rolling-deploy` and `alb` modules under the hood.

Before getting started, make sure to run `terraform destroy` on any `webserver-cluster` deployments you have from previous chapters. After you do that, you can start putting together the `asg-rolling-deploy` and `alb` modules. Create a new folder at `modules/cluster/asg-rolling-deploy`, and move the following resources from `module/services/webserver-cluster/main.tf` to `modules/cluster/asg-rolling-deploy/main.tf`:

- `aws_launch_configuration`
- `aws_autoscaling_group`
- `aws_autoscaling_schedule` (both of them)
- `aws_security_group` (for the Instances but not for the ALB)
- `aws_security_group_rule` (just the one rule for the Instances but not those for the ALB)
- `aws_cloudwatch_metric_alarm` (both of them)

Next, move the following variables from `module/services/webserver-cluster/variables.tf` to `modules/cluster/asg-rolling-deploy/variables.tf`:

- `cluster_name`
- `ami`
- `instance_type`
- `min_size`
- `max_size`
- `enable_autoscaling`
- `custom_tags`
- `server_port`

Let’s now move on to the ALB module. Create a new folder at `modules/networking/alb`, and move the following resources from `module/services/webserver-cluster/main.tf` to `modules/networking/alb/main.tf`:

- `aws_lb`
- `aws_lb_listener`
- `aws_security_group` (the one for the ALB but not for the Instances)

- `aws_security_group_rule` (both of the rules for the ALB but not the one for the Instances)

Create `modules/networking/alb/variables.tf`, and define a single variable within:

```
variable "alb_name" {
  description = "The name to use for this ALB"
  type        = string
}
```

Use this variable as the name argument of the `aws_lb` resource:

```
resource "aws_lb" "example" {
  name                = var.alb_name
  load_balancer_type = "application"
  subnets            = data.aws_subnets.default.ids
  security_groups     = [aws_security_group.alb.id]
}
```

And the name argument of the `aws_security_group` resource:

```
resource "aws_security_group" "alb" {
  name = var.alb_name
}
```

This is a lot of code to shuffle around, so feel free to use the code examples for this chapter from [GitHub](#).

Composable Modules

You now have two small modules—`asg-rolling-deploy` and `alb`—that each do one thing and do it well. How do you make them work together? How do you build modules that are reusable and composable? This question is not unique to Terraform but is something programmers have been thinking about for decades. To quote Doug McIlroy,⁵ the original developer of Unix pipes and a number of other Unix tools, including `diff`, `sort`, `join`, and `tr`:

This is the Unix philosophy: Write programs that do one thing and do it well. Write programs to work together.

One way to do this is through *function composition*, in which you can take the outputs of one function and pass them as the inputs to another. For example, if you had the following small functions in Ruby:

```
# Simple function to do addition
def add(x, y)
  return x + y
end
```

⁵ Peter H. Salus, *A Quarter-Century of Unix* (New York: Addison-Wesley Professional, 1994).

```

# Simple function to do subtraction
def sub(x, y)
    return x - y
end

# Simple function to do multiplication
def multiply(x, y)
    return x * y
end

```

you can use function composition to put them together by taking the outputs from add and sub and passing them as the inputs to multiply:

```

# Complex function that composes several simpler functions
def do_calculation(x, y)
    return multiply(add(x, y), sub(x, y))
end

```

One of the main ways to make functions composable is to minimize *side effects*: that is, where possible, avoid reading state from the outside world and instead have it passed in via input parameters, and avoid writing state to the outside world and instead return the result of your computations via output parameters. Minimizing side effects is one of the core tenets of functional programming because it makes the code easier to reason about, easier to test, and easier to reuse. The reuse story is particularly compelling, since function composition allows you to gradually build up more complicated functions by combining simpler functions.

Although you can't avoid side effects when working with infrastructure code, you can still follow the same basic principles in your Terraform modules: pass everything in through input variables, return everything through output variables, and build more complicated modules by combining simpler modules.

Open up `modules/cluster/asg-rolling-deploy/variables.tf`, and add four new input variables:

```

variable "subnet_ids" {
    description = "The subnet IDs to deploy to"
    type        = list(string)
}

variable "target_group_arns" {
    description = "The ARNs of ELB target groups in which to register Instances"
    type        = list(string)
    default     = []
}

variable "health_check_type" {
    description = "The type of health check to perform. Must be one of: EC2, ELB."
    type        = string
    default     = "EC2"
}

```

```

}

variable "user_data" {
  description = "The User Data script to run in each Instance at boot"
  type        = string
  default     = null
}

```

The first variable, `subnet_ids`, tells the `asg-rolling-deploy` module what subnets to deploy into. Whereas the `webserver-cluster` module was hardcoded to deploy into the Default VPC and subnets, by exposing the `subnet_ids` variable, you allow this module to be used with any VPC or subnets. The next two variables, `target_group_arns` and `health_check_type`, configure how the ASG integrates with load balancers. Whereas the `webserver-cluster` module had a built-in ALB, the `asg-rolling-deploy` module is meant to be a generic module, so exposing the load-balancer settings as input variables allows you to use the ASG with a wide variety of use cases; e.g., no load balancer, one ALB, multiple NLBs, and so on.

Take these three new input variables and pass them through to the `aws_autoscaling_group` resource in `modules/cluster/asg-rolling-deploy/main.tf`, replacing the previously hardcoded settings that were referencing resources (e.g., the ALB) and data sources (e.g., `aws_subnets`) that we didn't copy into the `asg-rolling-deploy` module:

```

resource "aws_autoscaling_group" "example" {
  name                = var.cluster_name
  launch_configuration = aws_launch_configuration.example.name
  vpc_zone_identifier = var.subnet_ids

  # Configure integrations with a load balancer
  target_group_arns = var.target_group_arns
  health_check_type = var.health_check_type

  min_size = var.min_size
  max_size = var.max_size

  # (...)
}

```

The fourth variable, `user_data`, is for passing in a User Data script. Whereas the `webserver-cluster` module had a hardcoded User Data script that could only be used to deploy a “Hello, World” app, by taking in a User Data script as an input variable, the `asg-rolling-deploy` module can be used to deploy any app across an ASG. Pass this `user_data` variable through to the `aws_launch_configuration` resource:

```

resource "aws_launch_configuration" "example" {
  image_id        = var.ami
  instance_type   = var.instance_type

```

```

security_groups = [aws_security_group.instance.id]
user_data       = var.user_data

# Required when using a launch configuration with an auto scaling group.
lifecycle {
  create_before_destroy = true
}
}

```

You'll also want to add a couple of useful output variables to *modules/cluster/asg-rolling-deploy/outputs.tf*:

```

output "asg_name" {
  value     = aws_autoscaling_group.example.name
  description = "The name of the Auto Scaling Group"
}

output "instance_security_group_id" {
  value     = aws_security_group.instance.id
  description = "The ID of the EC2 Instance Security Group"
}

```

Outputting this data makes the *asg-rolling-deploy* module even more reusable, since consumers of the module can use these outputs to add new behaviors, such as attaching custom rules to the security group.

For similar reasons, you should add several output variables to *modules/networking/alb/outputs.tf*:

```

output "alb_dns_name" {
  value     = aws_lb.example.dns_name
  description = "The domain name of the load balancer"
}

output "alb_http_listener_arn" {
  value     = aws_lb_listener.http.arn
  description = "The ARN of the HTTP listener"
}

output "alb_security_group_id" {
  value     = aws_security_group.alb.id
  description = "The ALB Security Group ID"
}

```

You'll see how to use these shortly.

The last step is to convert the *webserver-cluster* module into a *hello-world-app* module that can deploy a “Hello, World” app using the *asg-rolling-deploy* and *alb* modules. To do this, rename *module/services/webserver-cluster* to *module/services/hello-world-app*. After all the changes in the previous steps, you should have only the following resources and data sources left in *module/services/hello-world-app/main.tf*:

- `aws_lb_target_group`
- `aws_lb_listener_rule`
- `terraform_remote_state` (for the DB)
- `aws_vpc`
- `aws_subnets`

Add the following variable to `modules/services/hello-world-app/variables.tf`:

```
variable "environment" {
  description = "The name of the environment we're deploying to"
  type        = string
}
```

Now, add the `asg-rolling-deploy` module that you created earlier to the `hello-world-app` module to deploy an ASG:

```
module "asg" {
  source = "../../cluster/asg-rolling-deploy"

  cluster_name = "hello-world-${var.environment}"
  ami          = var.ami
  instance_type = var.instance_type

  user_data      = templatefile("${path.module}/user-data.sh", {
    server_port = var.server_port
    db_address  = data.terraform_remote_state.db.outputs.address
    db_port     = data.terraform_remote_state.db.outputs.port
    server_text = var.server_text
  })

  min_size      = var.min_size
  max_size      = var.max_size
  enable_autoscaling = var.enable_autoscaling

  subnet_ids      = data.aws_subnets.default.ids
  target_group_arns = [aws_lb_target_group.asg.arn]
  health_check_type = "ELB"

  custom_tags = var.custom_tags
}
```

And add the `alb` module, also that you created earlier, to the `hello-world-app` module to deploy an ALB:

```
module "alb" {
  source = "../../networking/alb"

  alb_name      = "hello-world-${var.environment}"
  subnet_ids    = data.aws_subnets.default.ids
}
```

Note the use of the input variable `environment` as a way to enforce a naming convention, so all of your resources will be namespaced based on the environment (e.g., `hello-world-stage`, `hello-world-prod`). This code also sets the new `subnet_ids`, `target_group_arns`, `health_check_type`, and `user_data` variables you added earlier to appropriate values.

Next, you need to configure the ALB target group and listener rule for this app. Update the `aws_lb_target_group` resource in `modules/services/hello-world-app/main.tf` to use `environment` in its name:

```
resource "aws_lb_target_group" "asg" {
  name      = "hello-world-${var.environment}"
  port      = var.server_port
  protocol  = "HTTP"
  vpc_id    = data.aws_vpc.default.id

  health_check {
    path            = "/"
    protocol        = "HTTP"
    matcher         = "200"
    interval        = 15
    timeout         = 3
    healthy_threshold = 2
    unhealthy_threshold = 2
  }
}
```

Now, update the `listener_arn` parameter of the `aws_lb_listener_rule` resource to point at the `alb_http_listener_arn` output of the ALB module:

```
resource "aws_lb_listener_rule" "asg" {
  listener_arn = module.alb.alb_http_listener_arn
  priority     = 100

  condition {
    path_pattern {
      values = ["*"]
    }
  }

  action {
    type = "forward"
    target_group_arn = aws_lb_target_group.asg.arn
  }
}
```

Finally, pass through the important outputs from the `asg-rolling-deploy` and `alb` modules as outputs of the `hello-world-app` module:

```
output "alb_dns_name" {
  value = module.alb.alb_dns_name
}
```

```

    description = "The domain name of the load balancer"
}

output "asg_name" {
    value     = module.asg.asg_name
    description = "The name of the Auto Scaling Group"
}

output "instance_security_group_id" {
    value     = module.asg.instance_security_group_id
    description = "The ID of the EC2 Instance Security Group"
}

```

This is function composition at work: you’re building up more complicated behavior (a “Hello, World” app) from simpler parts (ASG and ALB modules).

Testable Modules

At this stage, you’ve written a whole lot of code in the form of three modules: `asg-rolling-deploy`, `alb`, and `hello-world-app`. The next step is to check that your code actually works.

The modules you’ve created aren’t root modules meant to be deployed directly. To deploy them, you need to write some Terraform code to plug in the arguments you want, set up the provider, configure the backend, and so on. A great way to do this is to create an *examples* folder that, as the name suggests, shows examples of how to use your modules. Let’s try it out.

Create `examples/asg/main.tf` with the following contents:

```

provider "aws" {
    region = "us-east-2"
}

module "asg" {
    source = "../modules/cluster/asg-rolling-deploy"

    cluster_name = var.cluster_name
    ami           = data.aws_ami.ubuntu.id
    instance_type = "t2.micro"

    min_size     = 1
    max_size     = 1
    enable_autoscaling = false

    subnet_ids = data.aws_subnets.default.ids
}

data "aws_vpc" "default" {
    default = true
}

```

```

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
  }
}

```

This bit of code uses the `asg-rolling-deploy` module to deploy an ASG of size 1. Try it out by running `terraform init` and `terraform apply` and checking to see that it runs without errors and actually spins up an ASG. Now, add in a *README.md* file with these instructions, and suddenly this tiny little example takes on a whole lot of power. In just several files and lines of code, you now have the following:

A manual test harness

You can use this example code while working on the `asg-rolling-deploy` module to repeatedly deploy and undeploy it by manually running `terraform apply` and `terraform destroy` to check that it works as you expect.

An automated test harness

As you will see in [Chapter 9](#), this example code is also how you create automated tests for your modules. I typically recommend that tests go into the *test* folder.

Executable documentation

If you commit this example (including *README.md*) into version control, other members of your team can find it, use it to understand how your module works, and take the module for a spin without writing a line of code. It's both a way to teach the rest of your team and, if you add automated tests around it, a way to ensure that your teaching materials always work as expected.

Every Terraform module you have in the *modules* folder should have a corresponding example in the *examples* folder. And every example in the *examples* folder should have a corresponding test in the *test* folder. In fact, you'll most likely have multiple examples (and therefore multiple tests) for each module, with each example showing different configurations and permutations of how that module can be used. For example, you might want to add other examples for the `asg-rolling-deploy` module that show how to use it with auto scaling policies, how to hook up load balancers to it, how to set custom tags, and so on.

Putting this all together, the folder structure for a typical *modules* repo will look something like this:

```
modules
├── examples
│   ├── alb
│   ├── asg-rolling-deploy
│   │   ├── one-instance
│   │   ├── auto-scaling
│   │   ├── with-load-balancer
│   │   └── custom-tags
│   ├── hello-world-app
│   └── mysql
├── modules
│   ├── alb
│   ├── asg-rolling-deploy
│   ├── hello-world-app
│   └── mysql
├── test
│   ├── alb
│   ├── asg-rolling-deploy
│   ├── hello-world-app
│   └── mysql
```

As an exercise for the reader, I leave it up to you to add lots of examples for the alb, asg-rolling-deploy, mysql, and hello-world-app modules.

A great practice to follow when developing a new module is to write the example code *first*, before you write even a line of module code. If you begin with the implementation, it's too easy to become lost in the implementation details, and by the time you resurface and make it back to the API, you end up with a module that is unintuitive and difficult to use. On the other hand, if you begin with the example code, you're free to think through the ideal user experience and come up with a clean API for your module and then work backward to the implementation. Because the example code is the primary way of testing modules anyway, this is a form of *Test-Driven Development* (TDD); I'll dive more into this topic in [Chapter 9](#), which is entirely dedicated to testing.

In this section, I'll focus on creating *self-validating modules*: that is, modules that can check their own behavior to prevent certain types of bugs. Terraform has two ways of doing this built in:

- Validations
- Preconditions and postconditions

Validations

As of Terraform 0.13, you can add *validation blocks* to any input variable to perform checks that go beyond basic type constraints. For example, you can add a validation block to the `instance_type` variable to ensure not only that the value the user passes in is a string (which is enforced by the type constraint) but that the string has one of two allowed values from the AWS Free Tier:

```
variable "instance_type" {
  description = "The type of EC2 Instances to run (e.g. t2.micro)"
  type        = string

  validation {
    condition     = contains(["t2.micro", "t3.micro"], var.instance_type)
    error_message = "Only free tier is allowed: t2.micro | t3.micro."
  }
}
```

The way a validation block works is that the `condition` parameter should evaluate to true if the value is valid and false otherwise. The `error_message` parameter allows you to specify the message to show the user if they pass in an invalid value. For example, here's what happens if you try to set `instance_type` to `m4.large`, which is not in the AWS Free Tier:

```
$ terraform apply -var instance_type="m4.large"
| Error: Invalid value for variable
|
|   on main.tf line 17:
|   1: variable "instance_type" {
|       |_____
|       | var.instance_type is "m4.large"
|
| Only free tier is allowed: t2.micro | t3.micro.
|
| This was checked by the validation rule at main.tf:21,3-13.
```

You can have multiple validation blocks in each variable to check multiple conditions:

```
variable "min_size" {
  description = "The minimum number of EC2 Instances in the ASG"
  type        = number

  validation {
    condition     = var.min_size > 0
    error_message = "ASGs can't be empty or we'll have an outage!"
  }

  validation {
    condition     = var.min_size <= 10
    error_message = "ASGs must have 10 or fewer instances to keep costs down."
  }
}
```

```

    }
}

```

Note that validation blocks have a major limitation: the condition in a validation block can *only* reference the surrounding input variable. If you try to reference any other input variables, local variables, resources, or data sources, you will get an error. So while validation blocks are useful for basic input sanitization, they can't be used for anything more complicated: for example, you can't use them to do checks across multiple variables (such as "exactly one of these two input variables must be set") or any kind of dynamic checks (such as checking that the AMI the user requested uses the x86_64 architecture). To do these sorts of more dynamic checks, you'll need to use precondition and postcondition blocks, as described next.

Preconditions and postconditions

As of Terraform 1.2, you can add precondition and postcondition blocks to resources, data sources, and output variables to perform more dynamic checks. The precondition blocks are for catching errors before you run `apply`. For example, you could use a precondition block to do a more robust check that the `instance_type` the user passes in is in the AWS Free Tier. In the previous section, you did this check using a validation block and a hardcoded list of instance types, but these sorts of lists quickly go out of date. You can instead use the `instance_type_data` data source to always get up-to-date information from AWS:

```

data "aws_ec2_instance_type" "instance" {
  instance_type = var.instance_type
}

```

And then you can add a precondition block to the `aws_launch_configuration` resource to check that this instance type is eligible for the AWS Free Tier:

```

resource "aws_launch_configuration" "example" {
  image_id      = var.ami
  instance_type = var.instance_type
  security_groups = [aws_security_group.instance.id]
  user_data     = var.user_data

  # Required when using a launch configuration with an auto scaling group.
  lifecycle {
    create_before_destroy = true
    precondition {
      condition = data.aws_ec2_instance_type.instance.free_tier_eligible
      error_message = "${var.instance_type} is not part of the AWS Free Tier!"
    }
  }
}

```

Just like validation blocks, precondition blocks (and postcondition blocks, as you'll see shortly) include a condition that must evaluate to true or false and an `error_message` to show the user if the condition evaluates to false. If you now try to run `apply` with an instance type not in the AWS Free Tier, you'll see your error message:

```
$ terraform apply -var instance_type="m4.large"
| Error: Resource precondition failed
|
|   on main.tf line 25, in resource "aws_launch_configuration" "example":
|   18:     condition = data.aws_ec2_instance_type.instance.free_tier_eligible
|       |_____
|       | data.aws_ec2_instance_type.instance.free_tier_eligible is false
|
|   m4.large is not part of the AWS Free Tier!
```

The postcondition blocks are for catching errors after you run `apply`. For example, you can add a postcondition block to the `aws_autoscaling_group` resource to check that the ASG was deployed across more than one Availability Zone (AZ), thereby ensuring you can tolerate the failure of at least one AZ:

```
resource "aws_autoscaling_group" "example" {
  name                = var.cluster_name
  launch_configuration = aws_launch_configuration.example.name
  vpc_zone_identifier = var.subnet_ids

  lifecycle {
    postcondition {
      condition     = length(self.availability_zones) > 1
      error_message = "You must use more than one AZ for high availability!"
    }
  }

  # (...)
}
```

Note the use of the `self` keyword in the condition parameter. *Self expressions* use the following syntax:

```
self.<ATTRIBUTE>
```

You can use this special syntax solely in postcondition, connection, and provisioner blocks (you'll see examples of the latter two later in this chapter) to refer to an output `ATTRIBUTE` of the surrounding resource. If you tried to use the standard `aws_autoscaling_group.example.<ATTRIBUTE>` syntax, you'd get a circular dependency error, as resources can't have references to themselves, so the `self` expression is a workaround added specifically for this sort of use case.

If you run `apply` on this module, Terraform will deploy the module, but after, if it turns out that the subnets the user passed in via the `subnet_ids` input variable were

all in the same AZ, the postcondition block will show an error. This way, you'll always be warned if your ASG isn't configured for high availability.

When to use validations, preconditions, and postconditions

As you can see, validation, precondition, and postcondition blocks are all similar, so when should you use each one?

Use validation blocks for basic input sanitization

Use validation blocks in all of your production-grade modules to prevent users from passing invalid variables into your modules. The goal is to catch basic input errors *before* any changes have been deployed. Although precondition blocks are more powerful, you should still use validation blocks for checking variables whenever possible, as validation blocks are defined with the variables they validate, which leads to a more readable and maintainable API.

Use precondition blocks for checking basic assumptions

Use precondition blocks in all of your production-grade modules to check assumptions that must be true *before* any changes have been deployed. This includes any checks on variables you can't do with validation blocks (such as checks that reference multiple variables or data sources) as well as checks on resources and data sources. The goal is to catch as many errors as early as you can, before those errors can do any damage.

Use postcondition blocks for enforcing basic guarantees

Use postcondition blocks in all of your production-grade modules to check guarantees about how your module behaves *after* changes have been deployed. The goal is to give users of your module confidence that your module will either do what it says when they run `apply` or exit with an error. It also gives maintainers of that module a clearer signal of what behaviors you want this module to enforce, so those aren't accidentally lost during a refactor.

Use automated testing tools for enforcing more advanced assumptions and guarantees

validation, precondition, and postcondition blocks are all useful tools, but they can only do basic checks. This is because you can only use data sources, resources, and language constructs built into Terraform to do these checks, and those are often not enough for more advanced behavior. For example, if you built a module to deploy a web service, you might want to add a check after deployment that the web service is able to respond to HTTP requests. You could try to do this in a postcondition block by making HTTP requests to the service using Terraform's [http provider](#), but most deployments happen asynchronously, so you may need to retry the HTTP request multiple times, and there is no retry mechanism built into that provider. Moreover, if you deployed an internal web service, it might not be accessible over the public internet, so you'd need

to connect to some internal network or VPN first, which is also tricky to do in pure Terraform code. Therefore, to do more robust checks, you'll want to use automated testing tools such as OPA and Terratest, both of which you'll see in [Chapter 9](#).

Versioned Modules

There are two types of versioning you'll want to think through with modules:

- Versioning of the module's dependencies
- Versioning of the module itself

Let's start with versioning of the module's dependencies. Your Terraform code has three types of dependencies:

Terraform core

The version of the `terraform` binary you depend on

Providers

The version of each provider your code depends on, such as the `aws` provider

Modules

The version of each module you depend on that are pulled in via `module` blocks

As a general rule, you'll want to practice *versioning pinning* with all of your dependencies. That means that you pin each of these three types of dependencies to a specific, fixed, known version. Deployments should be predictable and repeatable: if the code didn't change, then running `apply` should always produce the same result, whether you run it today or three months from now or three years from now. To make that happen, you need to avoid pulling in new versions of dependencies accidentally. Instead, version upgrades should always be an explicit, deliberate action that is visible in the code you check into version control.

Let's go through how to do version pinning for the three types of Terraform dependencies.

To pin the version of the first type of dependency, your Terraform core version, you can use the `required_version` argument in your code. At a bare minimum, you should require a specific major version of Terraform:

```
terraform {  
  # Require any 1.x version of Terraform  
  required_version = ">= 1.0.0, < 2.0.0"  
}
```

This is critical, because each major release of Terraform is backward incompatible: e.g., the upgrade from 1.0.0 to 2.0.0 will likely include breaking changes, so you don't

want to do it by accident. The preceding code will allow you to use only 1.x.x versions of Terraform with that module, so 1.0.0 and 1.2.3 will work, but if you try to use, perhaps accidentally, 0.14.3 or 2.0.0, and run `terraform apply`, you immediately get an error:

```
$ terraform apply
```

```
Error: Unsupported Terraform Core version
```

```
This configuration does not support Terraform version 0.14.3. To proceed,
either choose another supported Terraform version or update the root module's
version constraint. Version constraints are normally set for good reason, so
updating the constraint may lead to other errors or unexpected behavior.
```

For production-grade code, you may want to pin not only the major version but the minor and patch version too:

```
terraform {
  # Require Terraform at exactly version 1.2.3
  required_version = "1.2.3"
}
```

In the past, before the Terraform 1.0.0 release, this was absolutely required, as every release of Terraform potentially included backward-incompatible changes, including to the state file format: e.g., a state file written by Terraform version 0.12.1 could not be read by Terraform version 0.12.0. Fortunately, after the 1.0.0 release, this is no longer the case: as per the officially published [Terraform v1.0 Compatibility Promises](#), upgrades between v1.x releases should require no changes to your code or workflows.

That said, you might still not want to upgrade to a new version of Terraform *accidentally*. New versions introduce new features, and if some of your computers (developer workstations and CI servers) start using those features but others are still on the old versions, you'll run into issues. Moreover, new versions of Terraform may have bugs, and you'll want to test that out in pre-production environments before trying it in production. Therefore, while pinning the major version is the bare minimum, I also recommend pinning the minor and patch version and applying Terraform upgrades intentionally, carefully, and consistently throughout each environment.

Note that, occasionally, you may have to use different versions of Terraform within a single codebase. For example, perhaps you are testing out Terraform 1.2.3 in the stage environment, while the prod environment is still on Terraform 1.0.0. Having to deal with multiple Terraform versions, whether on your own computer or on your CI servers, can be tricky. Fortunately, the open source tool `tfenv`, the Terraform version manager, makes this much easier.

At its most basic level, you can use `tfenv` to install and switch between multiple versions of Terraform. For example, you can use the `tfenv install` command to install a specific version of Terraform:

```
$ tfenv install 1.2.3
Installing Terraform v1.2.3
Downloading release tarball from
https://releases.hashicorp.com/terraform/1.2.3/terraform_1.2.3_darwin_amd64.zip
Archive:  tfenv_download.ZUS3Qn/terraform_1.2.3_darwin_amd64.zip
  inflating: /opt/homebrew/Cellar/tfenv/2.2.2/versions/1.2.3/terraform
Installation of terraform v1.2.3 successful.
```



tfenv on Apple Silicon (M1, M2)

As of June 2022, tfenv did not install the proper version of Terraform on Apple Silicon, such as Macs running M1 or M2 processors (see [this open issue for details](#)). The workaround is to set the `TFENV_ARCH` environment variable to `arm64`:

```
$ export TFENV_ARCH=arm64
$ tfenv install 1.2.3
```

You can list the versions you have installed using the `list` command:

```
$ tfenv list
1.2.3
1.1.4
1.1.0
* 1.0.0 (set by /opt/homebrew/Cellar/tfenv/2.2.2/version)
```

And you can select the version of Terraform to use from that list using the `use` command:

```
$ tfenv use 1.2.3
Switching default version to v1.2.3
Switching completed
```

These commands are all handy for working with multiple versions of Terraform, but the real power of tfenv is its support for *.terraform-version* files. tfenv will automatically look for a *.terraform-version* file in the current folder, as well as all the parent folders, all the way up to the project root—that is, the version control root (e.g., the folder with a *.git* folder in it)—and if it finds that file, any terraform command you run will automatically use the version defined in that file.

For example, if you wanted to try out Terraform 1.2.3 in the stage environment, while sticking with Terraform 1.0.0 in the prod environment, you could use the following folder structure:

```
live
├─ stage
│   └─ vpc
│       └─ mysql
│           └─ frontend-app
│               └─ .terraform-version
└─ prod
```

```
L vpc
L mysql
L frontend-app
L .terraform-version
```

Inside of *live/stage/.terraform-version*, you would have the following:

```
1.2.3
```

And inside of *live/prod/.terraform-version*, you would have the following:

```
1.0.0
```

Now, any `terraform` command you run in *stage* or any subfolder will automatically use Terraform 1.2.3. You can check this by running the `terraform version` command:

```
$ cd stage/vpc
$ terraform version
Terraform v1.2.3
```

And similarly, any `terraform` command you run in *prod* will automatically use Terraform 1.0.0:

```
$ cd prod/vpc
$ terraform version
Terraform v1.0.0
```

This works automatically on any developer workstation and in your CI server so long as everyone has `tfenv` installed. If you're a Terragrunt user, `tgswitch` offers similar functionality to automatically pick the Terragrunt version based on a *.terragrunt-version* file.

Let's now turn our attention to the second type of dependency in your Terraform code: providers. As you saw in [Chapter 7](#), to pin provider versions, you can use the `required_providers` block:

```
terraform {
  required_version = ">= 1.0.0, < 2.0.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 4.0"
    }
  }
}
```

This code pins the AWS Provider code to any 4.x version (the `~> 4.0` syntax is equivalent to `>= 4.0, < 5.0`). Again, the bare minimum is to pin to a specific major version number to avoid accidentally pulling in backward-incompatible changes. With Terraform 0.14.0 and above, you don't need to pin minor or patch versions

for providers, as this happens automatically due to the *lock file*. The first time you run `terraform init`, Terraform creates a `.terraform.lock.hcl` file, which records the following information:

The exact version of each provider you used

If you check the `.terraform.lock.hcl` file into version control (which you should!), then in the future, if you run `terraform init` again, on this computer or any other, Terraform will download the *exact* same version of each provider. That's why you don't need to pin the minor and patch version number in the `required_providers` block, as that's the default behavior anyway. If you want to explicitly upgrade a provider version, you can update the version constraint in the `required_providers` block and run `terraform init -upgrade`. Terraform will download new providers that match your version constraints and update the `.terraform.lock.hcl` file; you should review those updates and commit them to version control.

The checksums for each provider

Terraform records the checksum of each provider it downloads, and on subsequent runs of `terraform init`, it will show an error if the checksum changed. This is a security measure to ensure someone can't swap out provider code with malicious code in the future. If the provider is cryptographically signed (most official HashiCorp providers are), Terraform will also validate the signature as an additional check that the code can be trusted.



Lock Files with Multiple Operating Systems

By default, Terraform only records checksums for the platform you ran `init` on: for example, if you ran `init` on Linux, then Terraform will only record the checksums for Linux provider binaries in `.terraform.lock.hcl`. If you check that file in and, later on, you run `init` on that code on a Mac, you'll get an error, as the Mac checksums won't be in the `.terraform.lock.hcl` file. If your team works across multiple operating systems, you'll need to run the `terraform providers lock` command to record the checksums for every platform you use:

```
terraform providers lock \  
-platform=windows_amd64 \  
-platform=darwin_amd64 \  
-platform=darwin_arm64 \  
-platform=linux_amd64
```

64-bit Windows
64-bit macOS
64-bit macOS (ARM)
64-bit Linux

Finally, let's now look at the third type of dependencies: modules. As discussed in [“Module Versioning” on page 133](#), I strongly recommend pinning module versions

by using source URLs (rather than local file paths) with the `ref` parameter set to a Git tag:

```
source = "git@github.com:foo/modules.git//services/hello-world-app?ref=v0.0.5"
```

If you use these sorts of URLs, Terraform will always download the exact same code for the module every time you run `terraform init`.

Now that you’ve seen how to version your code’s dependencies, let’s talk about how to version the code itself. As you saw in “[Module Versioning](#)” on page 133, you can version your code by using Git tags with semantic versioning:

```
$ git tag -a "v0.0.5" -m "Create new hello-world-app module"
$ git push --follow-tags
```

For example, to deploy version `v0.0.5` of your `hello-world-app` module in the staging environment, put the following code into `live/stage/services/hello-world-app/main.tf`:

```
provider "aws" {
    region = "us-east-2"
}

module "hello_world_app" {
    # TODO: replace this with your own module URL and version!!
    source = "git@github.com:foo/modules.git//services/hello-world-app?ref=v0.0.5"

    server_text      = "New server text"
    environment      = "stage"
    db_remote_state_bucket = "(YOUR_BUCKET_NAME)"
    db_remote_state_key   = "stage/data-stores/mysql/terraform.tfstate"

    instance_type    = "t2.micro"
    min_size         = 2
    max_size         = 2
    enable_autoscaling = false
    ami              = data.aws_ami.ubuntu.id
}

data "aws_ami" "ubuntu" {
    most_recent = true
    owners      = ["099720109477"] # Canonical

    filter {
        name   = "name"
        values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
    }
}
```

Next, pass through the ALB DNS name as an output in `live/stage/services/hello-world-app/outputs.tf`:

```
output "alb_dns_name" {  
  value      = module.hello_world_app.alb_dns_name  
  description = "The domain name of the load balancer"  
}
```

Now you can deploy your versioned module by running `terraform init` and `terraform apply`:

```
$ terraform apply
```

```
(...)
```

```
Apply complete! Resources: 13 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
alb_dns_name = "hello-world-stage-477699288.us-east-2.elb.amazonaws.com"
```

If that works well, you can then deploy the exact same version—and therefore the exact same code—to other environments, including production. If you ever encounter an issue, versioning also gives you the option to roll back by deploying an older version.

Another option for releasing modules is to publish them in the Terraform Registry. The **Public Terraform Registry** includes hundreds of reusable, community-maintained, open source modules for AWS, Google Cloud, Azure, and many other providers. There are a few requirements to publish a module to the Public Terraform Registry:⁶

- The module must live in a public GitHub repo.
- The repo must be named `terraform-<PROVIDER>-<NAME>`, where PROVIDER is the provider the module is targeting (e.g., `aws`) and NAME is the name of the module (e.g., `rds`).
- The module must follow a specific file structure, including defining Terraform code in the root of the repo, providing a *README.md*, and using the convention of *main.tf*, *variables.tf*, and *outputs.tf* as filenames.
- The repo must use Git tags with semantic versioning (`x.y.z`) for releases.

If your module meets those requirements, you can share it with the world by logging in to the Terraform Registry with your GitHub account and using the web UI to publish the module. Once your modules are in the Registry, your team can use a web UI to discover modules and learn how to use them.

⁶ You can find the full details on publishing modules [on the Terraform website](#).

Terraform even supports a special syntax for consuming modules from the Terraform Registry. Instead of long Git URLs with hard-to-spot ref parameters, you can use a special shorter registry URL in the source argument and specify the version via a separate version argument using the following syntax:

```
module "<NAME>" {  
  source = "<OWNER>/<REPO>/<PROVIDER>"  
  version = "<VERSION>"  
  
  # (...)  
}
```

where NAME is the identifier to use for the module in your Terraform code, OWNER is the owner of the GitHub repo (e.g., in `github.com/foo/bar`, the owner is `foo`), REPO is the name of the GitHub repo (e.g., in `github.com/foo/bar`, the repo is `bar`), PROVIDER is the provider you're targeting (e.g., `aws`), and VERSION is the version of the module to use. Here's an example of how to use an open source RDS module from the Terraform Registry:

```
module "rds" {  
  source = "terraform-aws-modules/rds/aws"  
  version = "4.4.0"  
  
  # (...)  
}
```

If you are a customer of HashiCorp's Terraform Cloud or Terraform Enterprise, you can have this same experience with a Private Terraform Registry—that is, a registry that lives in your private Git repos and is only accessible to your team. This can be a great way to share modules within your company.

Beyond Terraform Modules

Although this book is all about Terraform, to build out your entire production-grade infrastructure, you'll need to use other tools, too, such as Docker, Packer, Chef, Puppet, and, of course, the duct tape, glue, and work horse of the DevOps world, the trusty Bash script.

Most of this code can reside in the *modules* folder directly alongside your Terraform code: e.g., you might have a *modules/packer* folder that contains a Packer template and some Bash scripts you use to configure an AMI right next to the *modules/asg-rolling-deploy* Terraform module you use to deploy that AMI.

However, occasionally, you need to go further and run some non-Terraform code (e.g., a script) directly from a Terraform module. Sometimes, this is to integrate Terraform with another system (e.g., you've already used Terraform to configure User Data scripts for execution on EC2 Instances); other times, it's to work around a limitation of Terraform, such as a missing provider API, or the inability to implement

complicated logic due to Terraform’s declarative nature. If you search around, you can find a few “escape hatches” within Terraform that make this possible:

- Provisioners
- Provisioners with `null_resource`
- External data source

Let’s go through these one a time.

Provisioners

Terraform *provisioners* are used to execute scripts either on the local machine or a remote machine when you run Terraform, typically to do the work of bootstrapping, configuration management, or cleanup. There are several different kinds of provisioners, including `local-exec` (execute a script on the local machine), `remote-exec` (execute a script on a remote resource), and `file` (copy files to a remote resource).⁷

You can add provisioners to a resource by using a `provisioner` block. For example, here is how you can use the `local-exec` provisioner to execute a script on your local machine:

```
resource "aws_instance" "example" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  provisioner "local-exec" {
    command = "echo \"Hello, World from $(uname -smp)\""
  }
}
```

When you run `terraform apply` on this code, it prints “Hello, World from” and then the local operating system details using the `uname` command:

```
$ terraform apply

(...)

aws_instance.example (local-exec): Hello, World from Darwin x86_64 i386

(...)

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

⁷ You can find the full list of provisioners [on the Terraform website](#).

Trying out a `remote-exec` provisioner is a little more complicated. To execute code on a remote resource, such as an EC2 Instance, your Terraform client must be able to do the following:

Communicate with the EC2 Instance over the network

You already know how to allow this with a security group.

Authenticate to the EC2 Instance

The `remote-exec` provisioner supports SSH and WinRM connections.

Since the examples in this book have you launch Linux (Ubuntu) EC2 Instances, you'll want to use SSH authentication. And that means you'll need to configure SSH keys. Let's begin by creating a security group that allows inbound connections to port 22, the default port for SSH:

```
resource "aws_security_group" "instance" {
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"

    # To make this example easy to try out, we allow all SSH connections.
    # In real world usage, you should lock this down to solely trusted IPs.
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

With SSH keys, the normal process would be for you to generate an SSH key pair on your computer, upload the public key to AWS, and store the private key somewhere secure where your Terraform code can access it. However, to make it easier for you to try out this code, you can use a resource called `tls_private_key` to automatically generate a private key:

```
# To make this example easy to try out, we generate a private key in Terraform.
# In real-world usage, you should manage SSH keys outside of Terraform.
resource "tls_private_key" "example" {
  algorithm = "RSA"
  rsa_bits  = 4096
}
```

This private key is stored in Terraform state, which is not great for production use cases but is fine for this learning exercise. Next, upload the public key to AWS using the `aws_key_pair` resource:

```
resource "aws_key_pair" "generated_key" {
  public_key = tls_private_key.example.public_key_openssh
}
```

Finally, let's begin writing the code for the EC2 Instance:

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
  }
}

resource "aws_instance" "example" {
  ami               = data.aws_ami.ubuntu.id
  instance_type     = "t2.micro"
  vpc_security_group_ids = [aws_security_group.instance.id]
  key_name          = aws_key_pair.generated_key.key_name
}
```

Just about all of this code should be familiar to you: it's using the `aws_ami` data source to find Ubuntu AMI and using the `aws_instance` resource to deploy that AMI on a `t2.micro` instance, associating that instance with the security group you created earlier. The only new item is the use of the `key_name` attribute in the `aws_instance` resource to instruct AWS to associate your public key with this EC2 Instance. AWS will add that public key to the server's *authorized_keys* file, which will allow you to SSH to that server with the corresponding private key.

Next, add the `remote-exec` provisioner to the `aws_instance` resource:

```
resource "aws_instance" "example" {
  ami               = data.aws_ami.ubuntu.id
  instance_type     = "t2.micro"
  vpc_security_group_ids = [aws_security_group.instance.id]
  key_name          = aws_key_pair.generated_key.key_name

  provisioner "remote-exec" {
    inline = ["echo \"Hello, World from $(uname -smp)\""]
  }
}
```

This looks nearly identical to the `local-exec` provisioner, except you use an `inline` argument to pass a list of commands to execute, instead of a single command argument. Finally, you need to configure Terraform to use SSH to connect to this EC2 Instance when running the `remote-exec` provisioner. You do this by using a `connection` block:

```
resource "aws_instance" "example" {
  ami               = data.aws_ami.ubuntu.id
  instance_type     = "t2.micro"
  vpc_security_group_ids = [aws_security_group.instance.id]
  key_name          = aws_key_pair.generated_key.key_name

  connection {
    type = "ssh"
    host = aws_instance.example.public_ip
    user = "ubuntu"
    private_key = aws_key_pair.generated_key.private_key_path
  }
}
```

```

provisioner "remote-exec" {
  inline = ["echo \"Hello, World from $(uname -smp)\""]
}

connection {
  type      = "ssh"
  host      = self.public_ip
  user      = "ubuntu"
  private_key = tls_private_key.example.private_key_pem
}
}

```

This connection block tells Terraform to connect to the EC2 Instance's public IP address using SSH with "ubuntu" as the username (this is the default username for the root user on Ubuntu AMIs) and the autogenerated private key. If you run `terraform apply` on this code, you'll see the following:

```

$ terraform apply

(...)

aws_instance.example: Creating...
aws_instance.example: Still creating... [10s elapsed]
aws_instance.example: Still creating... [20s elapsed]
aws_instance.example: Provisioning with 'remote-exec'...
aws_instance.example (remote-exec): Connecting to remote host via SSH...
aws_instance.example (remote-exec): Connecting to remote host via SSH...
aws_instance.example (remote-exec): Connecting to remote host via SSH...

(... repeats a few more times ...)

aws_instance.example (remote-exec): Connecting to remote host via SSH...
aws_instance.example (remote-exec): Connected!
aws_instance.example (remote-exec): Hello, World from Linux x86_64 x86_64

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

```

The `remote-exec` provisioner doesn't know exactly when the EC2 Instance will be booted and ready to accept connections, so it will retry the SSH connection multiple times until it succeeds or hits a timeout (the default timeout is five minutes, but you can configure it). Eventually, the connection succeeds, and you get a "Hello, World" from the server.

Note that, by default, when you specify a provisioner, it is a *creation-time provisioner*, which means that it runs (a) during `terraform apply`, and (b) only during the initial creation of a resource. The provisioner will *not* run on any subsequent calls to `terraform apply`, so creation-time provisioners are mainly useful for running initial bootstrap code. If you set the `when = destroy` argument on a provisioner, it will be a

destroy-time provisioner, which will run after you run `terraform destroy`, just before the resource is deleted.

You can specify multiple provisioners on the same resource and Terraform will run them one at a time, in order, from top to bottom. You can use the `on_failure` argument to instruct Terraform how to handle errors from the provisioner: if set to `"continue"`, Terraform will ignore the error and continue with resource creation or destruction; if set to `"abort"`, Terraform will abort the creation or destruction.

Provisioners Versus User Data

You've now seen two different ways to execute scripts on a server using Terraform: one is to use a `remote-exec` provisioner, and the other is to use a User Data script. I've generally found User Data to be the more useful tool for the following reasons:

- A `remote-exec` provisioner requires that you open up SSH or WinRM access to your servers, which is more complicated to manage (as you saw earlier with all the security group and SSH key work) and less secure than User Data, which solely requires AWS API access (which you must have anyway when using Terraform to deploy to AWS).
- You can use User Data scripts with ASGs, ensuring that all servers in that ASG execute the script during boot, including servers launched due to an auto scaling or auto recovery event. Provisioners take effect only while Terraform is running and don't work with ASGs at all.
- The User Data script can be seen in the EC2 console (select an Instance, click Actions → Instance Settings → View/Change User Data), and you can find its execution log on the EC2 Instance itself (typically in `/var/log/cloud-init*.log`), both of which are useful for debugging and neither of which is available with provisioners.

The only real advantage to using a provisioner to execute code on an EC2 Instance is that User Data scripts are limited to a length of 16 KB, whereas provisioner scripts can be arbitrarily long.

Provisioners with `null_resource`

Provisioners can be defined only within a resource, but sometimes, you want to execute a provisioner without tying it to a specific resource. You can do this using something called the `null_resource`, which acts just like a normal Terraform resource, except that it doesn't create anything. By defining provisioners on the `null_resource`, you can run your scripts as part of the Terraform lifecycle but without being attached to any "real" resource:

```
resource "null_resource" "example" {
  provisioner "local-exec" {
    command = "echo \"Hello, World from $(uname -smp)\""
  }
}
```

The `null_resource` even has a handy argument called `triggers`, which takes in a map of keys and values. Whenever the values change, the `null_resource` will be re-created, therefore forcing any provisioners within it to be reexecuted. For example, if you want to execute a provisioner within a `null_resource` every single time you run `terraform apply`, you could use the `uuid()` built-in function, which returns a new, randomly generated UUID each time it's called, within the `triggers` argument:

```
resource "null_resource" "example" {
  # Use UUID to force this null_resource to be recreated on every
  # call to 'terraform apply'
  triggers = {
    uuid = uuid()
  }

  provisioner "local-exec" {
    command = "echo \"Hello, World from $(uname -smp)\""
  }
}
```

Now, every time you call `terraform apply`, the `local-exec` provisioner will execute:

```
$ terraform apply

(...)

null_resource.example (local-exec): Hello, World from Darwin x86_64 i386

$ terraform apply

null_resource.example (local-exec): Hello, World from Darwin x86_64 i386
```

External data source

Provisioners will typically be your go-to for executing scripts from Terraform, but they aren't always the correct fit. Sometimes, what you're really looking to do is execute a script to fetch some data and make that data available within the Terraform code itself. To do this, you can use the external data source, which allows an external command that implements a specific protocol to act as a data source.

The protocol is as follows:

- You can pass data from Terraform to the external program using the `query` argument of the external data source. The external program can read in these arguments as JSON from `stdin`.

- The external program can pass data back to Terraform by writing JSON to stdout. The rest of your Terraform code can then pull data out of this JSON by using the `result` output attribute of the external data source.

Here's an example:

```
data "external" "echo" {
  program = ["bash", "-c", "cat /dev/stdin"]

  query = {
    foo = "bar"
  }
}

output "echo" {
  value = data.external.echo.result
}

output "echo_foo" {
  value = data.external.echo.result.foo
}
```

This example uses the external data source to execute a Bash script that echoes back to stdout any data it receives on stdin. Therefore, any data you pass in via the `query` argument should come back as is via the `result` output attribute. Here's what happens when you run `terraform apply` on this code:

```
$ terraform apply

(...)

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

echo = {
  "foo" = "bar"
}
echo_foo = "bar"
```

You can see that `data.external.<NAME>.result` contains the JSON returned by the external program and that you can navigate within that JSON using the syntax `data.external.<NAME>.result.<PATH>` (e.g., `data.external.echo.result.foo`).

The external data source is a lovely escape hatch if you need to access data in your Terraform code and there's no existing data source that knows how to retrieve that data. However, be conservative with your use of external data sources and all of the other Terraform “escape hatches,” since they make your code less portable and more brittle. For example, the external data source code you just saw relies on Bash, which means you won't be able to deploy that Terraform module from Windows.

Conclusion

Now that you've seen all of the ingredients of creating production-grade Terraform code, it's time to put them together. The next time you begin to work on a new module, use the following process:

1. Go through the production-grade infrastructure checklist in [Table 8-2](#), and explicitly identify the items you'll be implementing and the items you'll be skipping. Use the results of this checklist, plus [Table 8-1](#), to come up with a time estimate for your boss.
2. Create an *examples* folder, and write the example code first, using it to define the best user experience and cleanest API you can think of for your modules. Create an example for each important permutation of your module, and include enough documentation and reasonable defaults to make the example as easy to deploy as possible.
3. Create a *modules* folder, and implement the API you came up with as a collection of small, reusable, composable modules. Use a combination of Terraform and other tools like Docker, Packer, and Bash to implement these modules. Make sure to pin the versions for all your dependencies, including Terraform core, your Terraform providers, and Terraform modules you depend on.
4. Create a *test* folder, and write automated tests for each example.

That last bullet point—writing automated tests for your infrastructure code—is what we'll focus on next, as we move on to [Chapter 9](#).

How to Test Terraform Code

The DevOps world is full of fear: fear of downtime, fear of data loss, fear of security breaches. Every time you go to make a change, you're always wondering, what will this affect? Will it work the same way in every environment? Will this cause another outage? And if there is an outage, how late into the night will you need to stay up to fix it this time? As companies grow, there is more and more at stake, which makes the deployment process even scarier, and even more error prone. Many companies try to mitigate this risk by doing deployments less frequently, but the result is that each deployment is larger and actually more prone to breakage.

If you manage your infrastructure as code, you have a better way to mitigate risk: tests. The goal of testing is to give you the confidence to make changes. The key word here is *confidence*: no form of testing can guarantee that your code is free of bugs, so it's more of a game of probability. If you can capture all of your infrastructure and deployment processes as code, you can test that code in a pre-production environment, and if it works there, there's a high probability that when you use the exact same code in production, it will work there, too. And in a world of fear and uncertainty, high probability and high confidence go a long way.

In this chapter, I'll go over the process of testing infrastructure code, including both manual testing and automated testing, with the bulk of the chapter spent on the latter:

- Manual tests
 - Manual testing basics
 - Cleaning up after tests
- Automated tests
 - Unit tests
 - Integration tests

- End-to-end tests
- Other testing approaches



Example Code

As a reminder, you can find all of the code examples in the book on [GitHub](#).

Manual Tests

When thinking about how to test Terraform code, it can be helpful to draw some parallels with how you would test code written in a general-purpose programming language such as Ruby. Let's say you were writing a simple web server in Ruby in *web-server.rb*:

```
class WebServer < WEBrick::HTTPServlet::AbstractServlet
  def do_GET(request, response)
    case request.path
    when "/"
      response.status = 200
      response['Content-Type'] = 'text/plain'
      response.body = 'Hello, World'
    when "/api"
      response.status = 201
      response['Content-Type'] = 'application/json'
      response.body = '{"foo":"bar"}'
    else
      response.status = 404
      response['Content-Type'] = 'text/plain'
      response.body = 'Not Found'
    end
  end
end
```

This code will send a 200 response with the body “Hello, World” for the / URL, a 201 response with a JSON body for the /api URL, and a 404 for all other URLs. How would you manually test this code? The typical answer is to add a bit of code to run the web server on localhost:

```
# This will only run if this script was called directly from the CLI, but
# not if it was required from another file
if __FILE__ == $0
  # Run the server on localhost at port 8000
  server = WEBrick::HTTPServer.new :Port => 8000
  server.mount '/', WebServer

  # Shut down the server on CTRL+C
```

```

trap 'INT' do server.shutdown end

# Start the server
server.start
end

```

When you run this file from the CLI, it will start the web server on port 8000:

```

$ ruby web-server.rb
[2019-05-25 14:11:52] INFO  WEBrick 1.3.1
[2019-05-25 14:11:52] INFO  ruby 2.3.7 (2018-03-28) [universal.x86_64-darwin17]
[2019-05-25 14:11:52] INFO  WEBrick::HTTPServer#start: pid=19767 port=8000

```

You can test this server using a web browser or `curl`:

```

$ curl localhost:8000/
Hello, World

```

Manual Testing Basics

What is the equivalent of this sort of manual testing with Terraform code? For example, from the previous chapters, you already have Terraform code for deploying an ALB. Here's a snippet from *modules/networking/alb/main.tf*:

```

resource "aws_lb" "example" {
  name           = var.alb_name
  load_balancer_type = "application"
  subnets       = var.subnet_ids
  security_groups = [aws_security_group.alb.id]
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.example.arn
  port              = local.http_port
  protocol          = "HTTP"

  # By default, return a simple 404 page
  default_action {
    type = "fixed-response"

    fixed_response {
      content_type = "text/plain"
      message_body = "404: page not found"
      status_code  = 404
    }
  }
}

resource "aws_security_group" "alb" {
  name = var.alb_name
}

# (...)

```

If you compare this code to the Ruby code, one difference should be fairly obvious: you can't deploy AWS ALBs, target groups, listeners, security groups, and all the other infrastructure on your own computer.

This brings us to *key testing takeaway #1*: when testing Terraform code, you can't use localhost. This applies to most IaC tools, not just Terraform. The only practical way to do manual testing with Terraform is to deploy to a real environment (i.e., deploy to AWS). In other words, the way you've been manually running `terraform apply` and `terraform destroy` throughout the book is how you do manual testing with Terraform.

This is one of the reasons why it's essential to have easy-to-deploy examples in the *examples* folder for each module, as described in [Chapter 8](#). The easiest way to manually test the `alb` module is to use the example code you created for it in *examples/alb*:

```
provider "aws" {  
  region = "us-east-2"  
}  
  
module "alb" {  
  source = "../../modules/networking/alb"  
  
  alb_name     = "terraform-up-and-running"  
  subnet_ids = data.aws_subnets.default.ids  
}
```

As you've done many times throughout the book, you deploy this example code using `terraform apply`:

```
$ terraform apply  
  
(...)  
  
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.  
  
Outputs:  
  
alb_dns_name = "hello-world-stage-477699288.us-east-2.elb.amazonaws.com"
```

When the deployment is done, you can use a tool such as `curl` to test, for example, that the default action of the ALB is to return a 404:

```
$ curl \  
  -s \  
  -o /dev/null \  
  -w "%{http_code}" \  
  hello-world-stage-477699288.us-east-2.elb.amazonaws.com  
  
404
```



Validating Infrastructure

The examples in this chapter use `curl` and HTTP requests to validate that the infrastructure is working, because the infrastructure you're testing includes a load balancer that responds to HTTP requests. For other types of infrastructure, you'll need to replace `curl` and HTTP requests with a different form of validation. For example, if your infrastructure code deploys a MySQL database, you'll need to use a MySQL client to validate it; if your infrastructure code deploys a VPN server, you'll need to use a VPN client to validate it; if your infrastructure code deploys a server that isn't listening for requests at all, you might need to SSH to the server and execute some commands locally to test it; and so on. So although you can use the same basic test structure described in this chapter with any type of infrastructure, the validation steps will change depending on what you're testing.

In short, when working with Terraform, every developer needs good example code to test and a real deployment environment (e.g., an AWS account) to use as an equivalent to `localhost` for running those tests. In the process of manual testing, you're likely to bring up and tear down a lot of infrastructure, and likely make lots of mistakes along the way, so this environment should be completely isolated from your other, more stable environments, such as staging, and especially production.

Therefore, I strongly recommend that every team sets up an isolated *sandbox environment*, in which developers can bring up and tear down any infrastructure they want without worrying about affecting others. In fact, to reduce the chances of conflicts between multiple developers (e.g., two developers trying to create a load balancer with the same name), the gold standard is that each developer gets their own completely isolated sandbox environment. For example, if you're using Terraform with AWS, the gold standard is for each developer to have their own AWS account that they can use to test anything they want.¹

Cleaning Up After Tests

Having many sandbox environments is essential for developer productivity, but if you're not careful, you can end up with infrastructure running all over the place, cluttering up all of your environments, and costing you a lot of money.

To keep costs from spiraling out of control, *key testing takeaway #2* is: regularly clean up your sandbox environments.

¹ AWS doesn't charge anything extra for additional AWS accounts, and if you use AWS Organizations, you can create multiple "child" accounts that all roll up their billing to a single root account, as you saw in [Chapter 7](#).

At a minimum, you should create a culture in which developers clean up whatever they deploy when they are done testing by running `terraform destroy`. Depending on your deployment environment, you might also be able to find tools that you can run on a regular schedule (e.g., a cron job) to automatically clean up unused or old resources, such as `cloud-nuke` and `aws-nuke`.

For example, a common pattern is to run `cloud-nuke` as a cron job once per day in each sandbox environment to delete all resources that are more than 48 hours old, based on the assumption that any infrastructure a developer fired up for manual testing is no longer necessary after a couple of days:

```
$ cloud-nuke aws --older-than 48h
```



Warning: Lots of Coding Ahead

Writing automated tests for infrastructure code is not for the faint of heart. This automated testing section is arguably the most complicated part of the book and does not make for light reading. If you're just skimming, feel free to skip this part. On the other hand, if you really want to learn how to test your infrastructure code, roll up your sleeves and get ready to write some code! You don't need to run any of the Ruby code (it's just there to help build up your mental model), but you'll want to write and run as much Go code as you can.

Automated Tests

The idea with automated testing is to write test code that validates that your real code behaves the way it should. As you'll see in [Chapter 10](#), you can set up a CI server to run these tests after every single commit and then immediately revert or fix any commits that cause the tests to fail, thereby always keeping your code in a working state.

Broadly speaking, there are three types of automated tests:

Unit tests

Unit tests verify the functionality of a single, small unit of code. The definition of *unit* varies, but in a general-purpose programming language, it's typically a single function or class. Usually, any external dependencies—for example, databases, web services, even the filesystem—are replaced with *test doubles* or *mocks* that allow you to finely control the behavior of those dependencies (e.g., by returning a hardcoded response from a database mock) to test that your code handles a variety of scenarios.

Integration tests

Integration tests verify that multiple units work together correctly. In a general-purpose programming language, an integration test consists of code that validates that several functions or classes work together correctly. Integration tests typically use a mix of real dependencies and mocks: for example, if you're testing the part of your app that communicates with the database, you might want to test it with a real database, but mock out other dependencies, such as the app's authentication system.

End-to-end tests

End-to-end tests involve running your entire architecture—for example, your apps, your data stores, your load balancers—and validating that your system works as a whole. Usually, these tests are done from the end-user's perspective, such as using Selenium to automate interacting with your product via a web browser. End-to-end tests typically use real systems everywhere, without any mocks, in an architecture that mirrors production (albeit with fewer/smaller servers to save money).

Each type of test serves a different purpose, and can catch different types of bugs, so you'll likely want to use a mix of all three types. The purpose of unit tests is to have tests that run quickly so that you can get fast feedback on your changes and validate a variety of different permutations to build up confidence that the basic building blocks of your code (the individual units) work as expected. But just because individual units work correctly in isolation doesn't mean that they will work correctly when combined, so you need integration tests to ensure the basic building blocks fit together correctly. And just because different parts of your system work correctly doesn't mean they will work correctly when deployed in the real world, so you need end-to-end tests to validate that your code behaves as expected in conditions similar to production.

Let's now go through how to write each type of test for Terraform code.

Unit Tests

To understand how to write unit tests for Terraform code, it's helpful to first look at what it takes to write unit tests for a general-purpose programming language such as Ruby. Take a look again at the Ruby web server code:

```
class WebServer < WEBrick::HTTPServlet::AbstractServlet
  def do_GET(request, response)
    case request.path
    when "/"
      response.status = 200
      response['Content-Type'] = 'text/plain'
      response.body = 'Hello, World'
    when "/api"
```

```

    response.status = 201
    response['Content-Type'] = 'application/json'
    response.body = '{"foo":"bar"}'
  else
    response.status = 404
    response['Content-Type'] = 'text/plain'
    response.body = 'Not Found'
  end
end
end

```

Writing a unit test that calls the `do_GET` method directly is tricky, as you'd have to either instantiate real `WebServer`, `request`, and `response` objects, or create test doubles of them, both of which require a fair bit of work. When you find it difficult to write unit tests, that's often a code smell and indicates that the code needs to be refactored. One way to refactor this Ruby code to make unit testing easier is to extract the “handlers”—that is, the code that handles the `/`, `/api`, and not found paths—into its own `Handlers` class:

```

class Handlers
  def handle(path)
    case path
    when "/"
      [200, 'text/plain', 'Hello, World']
    when "/api"
      [201, 'application/json', '{"foo":"bar"}']
    else
      [404, 'text/plain', 'Not Found']
    end
  end
end

```

There are two key properties to notice about this new `Handlers` class:

Simple values as inputs

The `Handlers` class does not depend on `HTTPServer`, `HTTPRequest`, or `HTTPResponse`. Instead, all of its inputs are simple values, such as the path of the URL, which is a string.

Simple values as outputs

Instead of setting values on a mutable `HTTPResponse` object (a side effect), the methods in the `Handlers` class return the HTTP response as a simple value (an array that contains the HTTP status code, content type, and body).

Code that takes in simple values as inputs and returns simple values as outputs is typically easier to understand, update, and test. Let's first update the `WebServer` class to use the new `Handlers` class to respond to requests:

```

class WebServer < WEBrick::HTTPServlet::AbstractServlet
  def do_GET(request, response)

```

```

    handlers = Handlers.new
    status_code, content_type, body = handlers.handle(request.path)

    response.status = status_code
    response['Content-Type'] = content_type
    response.body = body
  end
end

```

This code calls the `handle` method of the `Handlers` class and sends back the status code, content type, and body returned by that method as an HTTP response. As you can see, using the `Handlers` class is clean and simple. This same property will make testing easy, too. Here are three unit tests that check each endpoint in the `Handlers` class:

```

class TestWebServer < Test::Unit::TestCase
  def initialize(test_method_name)
    super(test_method_name)
    @handlers = Handlers.new
  end

  def test_unit_hello
    status_code, content_type, body = @handlers.handle("/")
    assert_equal(200, status_code)
    assert_equal('text/plain', content_type)
    assert_equal('Hello, World', body)
  end

  def test_unit_api
    status_code, content_type, body = @handlers.handle("/api")
    assert_equal(201, status_code)
    assert_equal('application/json', content_type)
    assert_equal('{"foo":"bar"}', body)
  end

  def test_unit_404
    status_code, content_type, body = @handlers.handle("/invalid-path")
    assert_equal(404, status_code)
    assert_equal('text/plain', content_type)
    assert_equal('Not Found', body)
  end
end

```

And here's how you run the tests:

```

$ ruby web-server-test.rb
Loaded suite web-server-test
Finished in 0.000572 seconds.

-----
3 tests, 9 assertions, 0 failures, 0 errors
100% passed
-----

```

In 0.0005272 seconds, you can now find out whether your web server code works as expected. That’s the power of unit testing: a fast feedback loop that helps you build confidence in your code.

Unit testing Terraform code

What is the equivalent of this sort of unit testing with Terraform code? The first step is to identify what a “unit” is in the Terraform world. The closest equivalent to a single function or class in Terraform is a single reusable module such as the `alb` module you created in [Chapter 8](#). How would you test this module?

With Ruby, to write unit tests, you needed to refactor the code so you could run it without complicated dependencies such as `HTTPServer`, `HTTPRequest`, or `HTTPResponse`. If you think about what your Terraform code is doing—making API calls to AWS to create the load balancer, listeners, target groups, and so on—you’ll realize that 99% of what this code is doing is communicating with complicated dependencies! There’s no practical way to reduce the number of external dependencies to zero, and even if you could, you’d effectively be left with no code to test.²

That brings us to *key testing takeaway #3*: you cannot do *pure* unit testing for Terraform code.

But don’t despair. You can still build confidence that your Terraform code behaves as expected by writing automated tests that use your code to deploy real infrastructure into a real environment (e.g., into a real AWS account). In other words, unit tests for Terraform are really integration tests. However, I prefer to still call them unit tests to emphasize that the goal is to test a single unit (i.e., a single reusable module) to get feedback as quickly as possible.

This means that the basic strategy for writing unit tests for Terraform is as follows:

1. Create a small, standalone module.
2. Create an easy-to-deploy example for that module.
3. Run `terraform apply` to deploy the example into a real environment.
4. Validate that what you just deployed works as expected. This step is specific to the type of infrastructure you’re testing: for example, for an ALB, you’d validate

² In limited cases, it is possible to override the endpoints Terraform uses to communicate with providers, such as [overriding the endpoints Terraform uses to talk to AWS to instead talk to a mocking tool called LocalStack](#). This works for a small number of endpoints, but most Terraform code makes *hundreds* of different API calls to the underlying provider, and mocking out all of them is impractical. Moreover, even if you do mock them all out, it’s not clear that the resulting unit test can give you much confidence that your code works correctly: e.g., if you create mock endpoints for ASGs and ALBs, your `terraform apply` might succeed, but does that tell you anything useful about whether your code would have actually deployed a working app on top of that infrastructure?

it by sending an HTTP request and checking that you receive back the expected response.

5. Run `terraform destroy` at the end of the test to clean up.

In other words, you do *exactly* the same steps as you would when doing manual testing, but you capture those steps as code. In fact, that's a good mental model for creating automated tests for your Terraform code: ask yourself, "How would I have tested this manually to be confident it works?" and then implement that test in code.

You can use any programming language you want to write the test code. In this book, all of the tests are written in the Go programming language to take advantage of an open source Go library called **Terratest**, which supports testing a wide variety of infrastructure-as-code tools (e.g., Terraform, Packer, Docker, Helm) across a wide variety of environments (e.g., AWS, Google Cloud, Kubernetes). Terratest is a bit like a Swiss Army knife, with hundreds of tools built in that make it significantly easier to test infrastructure code, including first-class support for the test strategy just described, where you `terraform apply` some code, validate that it works, and then run `terraform destroy` at the end to clean up.

To use Terratest, you need to do the following:

1. **Install Go** (minimum version 1.13).
2. Create a folder for your test code: e.g., a folder named *test*.
3. Run `go mod init <NAME>` in the folder you just created, where NAME is the name to use for this test suite, typically in the format `github.com/<ORG_NAME>/<PROJECT_NAME>` (e.g., `go mod init github.com/brikis98/terraform-up-and-running`). This should create a *go.mod* file, which is used to track the dependencies of your Go code.

As a quick sanity check that your environment is set up correctly, create *go_sanity_test.go* in your new folder with the following contents:

```
package test

import (
    "fmt"
    "testing"
)

func TestGoIsWorking(t *testing.T) {
    fmt.Println()
    fmt.Println("If you see this text, it's working!")
    fmt.Println()
}
```

Run this test using the `go test` command:

```
go test -v
```

The `-v` flag means verbose, which ensures that the test always shows all log output. You should see output that looks something like this:

```
=== RUN    TestGoIsWorking

If you see this text, it's working!

--- PASS: TestGoIsWorking (0.00s)
PASS
ok      github.com/brikis98/terraform-up-and-running-code    0.192s
```

If that's working, feel free to delete `go_sanity_test.go`, and move on to writing a unit test for the alb module. Create `alb_example_test.go` in your `test` folder with the following skeleton of a unit test:

```
package test

import (
    "testing"
)

func TestAlbExample(t *testing.T) {
}
```

The first step is to direct Terratest to where your Terraform code resides by using the `terraform.Options` type:

```
package test

import (
    "github.com/gruntwork-io/terratest/modules/terraform"
    "testing"
)

func TestAlbExample(t *testing.T) {
    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
    }
}
```

Note that to test the alb module, you actually test the example code in your `examples` folder (you should update the relative path in `TerraformDir` to point to the folder where you created that example).

The next step in the automated test is to run `terraform init` and `terraform apply` to deploy the code. Terratest has handy helpers for doing that:

```

func TestAlbExample(t *testing.T) {
    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
    }

    terraform.Init(t, opts)
    terraform.Apply(t, opts)
}

```

In fact, running `init` and `apply` is such a common operation with Terratest that there is a convenient `InitAndApply` helper method that does both in one command:

```

func TestAlbExample(t *testing.T) {
    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
    }

    // Deploy the example
    terraform.InitAndApply(t, opts)
}

```

The preceding code is already a fairly useful unit test, since it will run `terraform init` and `terraform apply` and fail the test if those commands don't complete successfully (e.g., due to a problem with your Terraform code). However, you can go even further by making HTTP requests to the deployed load balancer and checking that it returns the data you expect. To do that, you need a way to get the domain name of the deployed load balancer. Fortunately, that's available as an output variable in the `alb` example:

```

output "alb_dns_name" {
    value     = module.alb.alb_dns_name
    description = "The domain name of the load balancer"
}

```

Terratest has helpers built in to read outputs from your Terraform code:

```

func TestAlbExample(t *testing.T) {
    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
    }

    // Deploy the example
    terraform.InitAndApply(t, opts)

    // Get the URL of the ALB
    albDnsName := terraform.OutputRequired(t, opts, "alb_dns_name")
}

```

```

        url := fmt.Sprintf("http://%s", albDnsName)
    }

```

The `OutputRequired` function returns the output of the given name, or it fails the test if that output doesn't exist or is empty. The preceding code builds a URL from this output using the `fmt.Sprintf` function that's built into Go (don't forget to import the `fmt` package). The next step is to make some HTTP requests to this URL using the `http_helper` package (make sure to add `github.com/gruntwork-io/terratest/modules/http-helper` as an import):

```

func TestAlbExample(t *testing.T) {
    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
    }

    // Deploy the example
    terraform.InitAndApply(t, opts)

    // Get the URL of the ALB
    albDnsName := terraform.OutputRequired(t, opts, "alb_dns_name")
    url := fmt.Sprintf("http://%s", albDnsName)

    // Test that the ALB's default action is working and returns a 404
    expectedStatus := 404
    expectedBody := "404: page not found"
    maxRetries := 10
    timeBetweenRetries := 10 * time.Second

    http_helper.HttpGetWithRetry(
        t,
        url,
        nil,
        expectedStatus,
        expectedBody,
        maxRetries,
        timeBetweenRetries,
    )
}

```

The `http_helper.HttpGetWithRetry` method will make an HTTP GET request to the URL you pass in and check that the response has the expected status code and body. If it doesn't, the method will retry up to the specified maximum number of retries, with the specified amount of time between retries. If it eventually achieves the expected response, the test will pass; if the maximum number of retries is reached without the expected response, the test will fail. This sort of retry logic is very common in infrastructure testing, as there is usually a period of time between when `terraform apply` finishes and when the deployed infrastructure is completely ready (i.e., it takes time for health checks to pass, DNS updates to propagate, and so on),

and as you don't know exactly how long that'll take, the best option is to retry until it works or you hit a timeout.

The last thing you need to do is to run `terraform destroy` at the end of the test to clean up. As you can guess, there is a Terratest helper for this: `terraform.Destroy`. However, if you call `terraform.Destroy` at the very end of the test, if any of the code before that causes a test failure (e.g., `HttpGetWithRetry` fails because the ALB is misconfigured), the test code will exit before getting to `terraform.Destroy`, and the infrastructure deployed for the test will never be cleaned up.

Therefore, you want to ensure that you *always* run `terraform.Destroy`, even if the test fails. In many programming languages, this is done with a `try / finally` or `try / ensure` construct, but in Go, this is done by using the `defer` statement, which will guarantee that the code you pass to it will be executed when the surrounding function returns (no matter how that return happens):

```
func TestAlbExample(t *testing.T) {
    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
    }

    // Clean up everything at the end of the test
    defer terraform.Destroy(t, opts)

    // Deploy the example
    terraform.InitAndApply(t, opts)

    // Get the URL of the ALB
    albDnsName := terraform.OutputRequired(t, opts, "alb_dns_name")
    url := fmt.Sprintf("http://%s", albDnsName)

    // Test that the ALB's default action is working and returns a 404
    expectedStatus := 404
    expectedBody := "404: page not found"
    maxRetries := 10
    timeBetweenRetries := 10 * time.Second

    http_helper.HttpGetWithRetry(
        t,
        url,
        nil,
        expectedStatus,
        expectedBody,
        maxRetries,
        timeBetweenRetries,
    )
}
```

Note that the `defer` is added early in the code, even before the call to `terraform .InitAndApply`, to ensure that nothing can cause the test to fail before getting to the `defer` statement and preventing it from queueing up the call to `terraform.Destroy`.

OK, this unit test is finally ready to run!



Terratest Version

The test code in this book was written with Terratest v0.39.0. Terratest is still a pre-1.0.0 tool, so newer releases may contain backward-incompatible changes. To ensure the test examples in this book work as written, I recommend installing Terratest specifically at version v0.39.0, and not the latest version. To do that, go into *go.mod* and add the following to the end of the file:

```
require github.com/gruntwork-io/terratest v0.39.0
```

Since this is a brand-new Go project, as a one-time action, you need to tell Go to download dependencies (including Terratest). The easiest way to do that at this stage is to run the following:

```
go mod tidy
```

This will download all your dependencies and create a *go.sum* file to lock the exact versions you used.

Next, since this test deploys infrastructure to AWS, before running the test, you need to authenticate to your AWS account as usual (see “[Other AWS Authentication Options](#)” on page 44). You saw earlier in this chapter that you should do manual testing in a sandbox account; for automated testing, this is even more important, so I recommend authenticating to a totally separate account. As your automated test suite grows, you might be spinning up hundreds or thousands of resources in every test suite, so keeping them isolated from everything else is essential.

I typically recommend that teams have a completely separate environment (e.g., completely separate AWS account) just for automated testing—separate even from the sandbox environments you use for manual testing. That way, you can safely delete all resources that are more than a few hours old in the testing environment, based on the assumption that no test will run that long.

After you’ve authenticated to an AWS account that you can safely use for testing, you can run the test, as follows:

```
$ go test -v -timeout 30m
```

```
TestAlbExample 2019-05-26T13:29:32+01:00 command.go:53:  
Running command terraform with args [init -upgrade=false]
```

```
(...)
```

```

TestAlbExample 2019-05-26T13:29:33+01:00 command.go:53:
Running command terraform with args [apply -input=false -lock=false]

(...)

TestAlbExample 2019-05-26T13:32:06+01:00 command.go:121:
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

(...)

TestAlbExample 2019-05-26T13:32:06+01:00 command.go:53:
Running command terraform with args [output -no-color alb_dns_name]

(...)

TestAlbExample 2019-05-26T13:38:32+01:00 http_helper.go:27:
Making an HTTP GET call to URL
http://terraform-up-and-running-1892693519.us-east-2.elb.amazonaws.com

(...)

TestAlbExample 2019-05-26T13:38:32+01:00 command.go:53:
Running command terraform with args
[destroy -auto-approve -input=false -lock=false]

(...)

TestAlbExample 2019-05-26T13:39:16+01:00 command.go:121:
Destroy complete! Resources: 5 destroyed.

(...)

PASS
ok      terraform-up-and-running      229.492s

```

Note the use of the `-timeout 30m` argument with `go test`. By default, Go imposes a time limit of 10 minutes for tests, after which it forcibly kills the test run, not only causing the tests to fail but also preventing the cleanup code (i.e., `terraform destroy`) from running. This ALB test should take closer to five minutes, but whenever running a Go test that deploys real infrastructure, it's safer to set an extra-long timeout to avoid the test being killed partway through and leaving all sorts of infrastructure still running.

The test will produce a lot of log output, but if you read through it carefully, you should be able to spot all of the key stages of the test:

1. Running `terraform init`
2. Running `terraform apply`

3. Reading output variables using `terraform output`
4. Repeatedly making HTTP requests to the ALB
5. Running `terraform destroy`

It's nowhere near as fast as the Ruby unit tests, but in less than five minutes, you can now automatically find out whether your `alb` module works as expected. This is about as fast of a feedback loop as you can get with infrastructure in AWS, and it should give you a lot of confidence that your code works as expected.

Dependency injection

Let's now see what it would take to add a unit test for some slightly more complicated code. Going back to the Ruby web server example once more, consider what would happen if you needed to add a new `/web-service` endpoint that made HTTP calls to an external dependency:

```
class Handlers
  def handle(path)
    case path
    when "/"
      [200, 'text/plain', 'Hello, World']
    when "/api"
      [201, 'application/json', '{"foo":"bar"}']
    when "/web-service"
      # New endpoint that calls a web service
      uri = URI("http://www.example.org")
      response = Net::HTTP.get_response(uri)
      [response.code.to_i, response['Content-Type'], response.body]
    else
      [404, 'text/plain', 'Not Found']
    end
  end
end
```

The updated `Handlers` class now handles the `/web-service` URL by making an HTTP GET to `example.org` and proxying the response. When you `curl` this endpoint, you get the following:

```
$ curl localhost:8000/web-service

<!doctype html>
<html>
<head>
  <title>Example Domain</title>
  <-- (...) -->
</head>
<body>
<div>
  <h1>Example Domain</h1>
```

```

    <p>
      This domain is established to be used for illustrative
      examples in documents. You may use this domain in
      examples without prior coordination or asking for permission.
    </p>
    <!-- (...) -->
  </div>
</body>
</html>

```

How would you add a unit test for this new method? If you tried to test the code as is, your unit tests would be subject to the behavior of an external dependency (in this case, `example.org`). This has a number of downsides:

- If that dependency has an outage, your tests will fail, even though there's nothing wrong with your code.
- If that dependency changed its behavior from time to time (e.g., returned a different response body), your tests would fail from time to time, and you'd need to constantly keep updating the test code, even though there's nothing wrong with the implementation.
- If that dependency were slow, your tests would be slow, which negates one of the main benefits of unit tests, the fast feedback loop.
- If you wanted to test that your code handles various corner cases based on how that dependency behaves (e.g., does the code handle redirects?), you'd have no way to do it without control of that external dependency.

Although working with real dependencies might make sense for integration and end-to-end tests, with unit tests, you should try to minimize external dependencies as much as possible. The typical strategy for doing this is *dependency injection*, in which you make it possible to pass in (or “inject”) external dependencies from outside your code, rather than hardcoding them within your code.

For example, the `Handlers` class shouldn't need to deal with all of the details of how to call a web service. Instead, you can extract that logic into a separate `WebService` class:

```

class WebService
  def initialize(url)
    @uri = URI(url)
  end

  def proxy
    response = Net::HTTP.get_response(@uri)
    [response.code.to_i, response['Content-Type'], response.body]
  end
end

```

This class takes a URL as an input and exposes a proxy method to proxy the HTTP GET response from that URL. You can then update the `Handlers` class to take a `WebService` instance as an input and use that instance in the `web_service` method:

```
class Handlers
  def initialize(web_service)
    @web_service = web_service
  end

  def handle(path)
    case path
    when "/"
      [200, 'text/plain', 'Hello, World']
    when "/api"
      [201, 'application/json', '{"foo":"bar"}']
    when "/web-service"
      # New endpoint that calls a web service
      @web_service.proxy
    else
      [404, 'text/plain', 'Not Found']
    end
  end
end
```

Now, in your implementation code, you can inject a real `WebService` instance that makes HTTP calls to `example.org`:

```
class WebServer < WEBrick::HTTPServlet::AbstractServlet
  def do_GET(request, response)
    web_service = WebService.new("http://www.example.org")
    handlers = Handlers.new(web_service)

    status_code, content_type, body = handlers.handle(request.path)

    response.status = status_code
    response['Content-Type'] = content_type
    response.body = body
  end
end
```

In your test code, you can create a mock version of the `WebService` class that allows you to specify a mock response to return:

```
class MockWebService
  def initialize(response)
    @response = response
  end

  def proxy
    @response
  end
end
```

And now you can create an instance of this `MockWebService` class and inject it into the `Handlers` class in your unit tests:

```
def test_unit_web_service
  expected_status = 200
  expected_content_type = 'text/html'
  expected_body = 'mock example.org'
  mock_response = [expected_status, expected_content_type, expected_body]

  mock_web_service = MockWebService.new(mock_response)
  handlers = Handlers.new(mock_web_service)

  status_code, content_type, body = handlers.handle("/web-service")
  assert_equal(expected_status, status_code)
  assert_equal(expected_content_type, content_type)
  assert_equal(expected_body, body)
end
```

Rerun the tests to make sure it all still works:

```
$ ruby web-server-test.rb
Loaded suite web-server-test
Started
...

Finished in 0.000645 seconds.
-----
4 tests, 12 assertions, 0 failures, 0 errors
100% passed
-----
```

Fantastic. Using dependency injection to minimize external dependencies allows you to write fast, reliable tests and check all the various corner cases. And since the three test cases you added earlier are still passing, you can be confident that your refactoring hasn't broken anything.

Let's now turn our attention back to Terraform and see what dependency injection looks like with Terraform modules, starting with the `hello-world-app` module. If you haven't already, the first step is to create an easy-to-deploy example for it in the *examples* folder:

```
provider "aws" {
  region = "us-east-2"
}

module "hello_world_app" {
  source = ".././../modules/services/hello-world-app"

  server_text = "Hello, World"
  environment = "example"

  db_remote_state_bucket = "(YOUR_BUCKET_NAME)"
}
```

```

db_remote_state_key    = "examples/terraform.tfstate"

instance_type          = "t2.micro"
min_size               = 2
max_size               = 2
enable_autoscaling     = false
ami                   = data.aws_ami.ubuntu.id
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-focal-20.04-amd64-server-*"]
  }
}

```

The dependency problem becomes apparent when you spot the parameters `db_remote_state_bucket` and `db_remote_state_key`: the `hello-world-app` module assumes that you've already deployed the `mysql` module and requires that you pass in the details of the S3 bucket where the `mysql` module is storing state using these two parameters. The goal here is to create a unit test for the `hello-world-app` module, and although a pure unit test with 0 external dependencies isn't possible with Terraform, it's still a good idea to minimize external dependencies whenever possible.

One of the first steps with minimizing dependencies is to make it clearer what dependencies your module has. A file-naming convention you might want to adopt is to move all of the data sources and resources that represent external dependencies into a separate *dependencies.tf* file. For example, here's what *modules/services/hello-world-app/dependencies.tf* would look like:

```

data "terraform_remote_state" "db" {
  backend = "s3"

  config = {
    bucket = var.db_remote_state_bucket
    key    = var.db_remote_state_key
    region = "us-east-2"
  }
}

data "aws_vpc" "default" {
  default = true
}

data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

```



```
}  
}
```

This convention makes it easier for users of your code to know, at a glance, what this code depends on in the outside world. In the case of the `hello-world-app` module, you can quickly see that it depends on a database, VPC, and subnets. So, how can you inject these dependencies from outside the module so that you can replace them at test time? You already know the answer to this: input variables.

For each of these dependencies, you should add a new input variable in `modules/services/hello-world-app/variables.tf`:

```
variable "vpc_id" {  
  description = "The ID of the VPC to deploy into"  
  type        = string  
  default     = null  
}  
  
variable "subnet_ids" {  
  description = "The IDs of the subnets to deploy into"  
  type        = list(string)  
  default     = null  
}  
  
variable "mysql_config" {  
  description = "The config for the MySQL DB"  
  type        = object({  
    address = string  
    port    = number  
  })  
  default     = null  
}
```

There's now an input variable for the VPC ID, subnet IDs, and MySQL config. Each variable specifies a default, so they are *optional variables* that the user can set to something custom or omit to get the default value. The default for each variable is `null`.

Note that the `mysql_config` variable uses the object type constructor to create a nested type with `address` and `port` keys. This type is intentionally designed to match the output types of the `mysql` module:

```
output "address" {  
  value        = aws_db_instance.example.address  
  description = "Connect to the database at this endpoint"  
}  
  
output "port" {  
  value        = aws_db_instance.example.port  
  description = "The port the database is listening on"  
}
```

One of the advantages of doing this is that, as soon as the refactor is complete, one of the ways you'll be able to use the `hello-world-app` and `mysql` modules together is as follows:

```
module "hello_world_app" {
  source = "../../modules/services/hello-world-app"

  server_text      = "Hello, World"
  environment      = "example"

  # Pass all the outputs from the mysql module straight through!
  mysql_config = module.mysql

  instance_type    = "t2.micro"
  min_size         = 2
  max_size         = 2
  enable_autoscaling = false
  ami              = data.aws_ami.ubuntu.id
}

module "mysql" {
  source = "../../modules/data-stores/mysql"

  db_name      = var.db_name
  db_username  = var.db_username
  db_password  = var.db_password
}
```

Because the type of `mysql_config` matches the type of the `mysql` module outputs, you can pass them all straight through in one line. And if the types are ever changed and no longer match, Terraform will give you an error right away so that you know to update them. This is not only function composition at work but also type-safe function composition.

But before that can work, you'll need to finish refactoring the code. Because the MySQL configuration can be passed in as an input, this means that the `db_remote_state_bucket` and `db_remote_state_key` variables should now be optional, so set their default values to `null`:

```
variable "db_remote_state_bucket" {
  description = "The name of the S3 bucket for the DB's Terraform state"
  type        = string
  default     = null
}

variable "db_remote_state_key" {
  description = "The path in the S3 bucket for the DB's Terraform state"
  type        = string
  default     = null
}
```

Next, use the `count` parameter to optionally create the three data sources in *modules/services/hello-world-app/dependencies.tf* based on whether the corresponding input variable is set to `null`:

```
data "terraform_remote_state" "db" {
  count = var.mysql_config == null ? 1 : 0

  backend = "s3"

  config = {
    bucket = var.db_remote_state_bucket
    key    = var.db_remote_state_key
    region = "us-east-2"
  }
}

data "aws_vpc" "default" {
  count = var.vpc_id == null ? 1 : 0
  default = true
}

data "aws_subnets" "default" {
  count = var.subnet_ids == null ? 1 : 0
  filter {
    name     = "vpc-id"
    values   = [data.aws_vpc.default.id]
  }
}
```

Now you need to update any references to these data sources to conditionally use either the input variable or the data source. Let's capture these as local values:

```
locals {
  mysql_config = (
    var.mysql_config == null
    ? data.terraform_remote_state.db[0].outputs
    : var.mysql_config
  )

  vpc_id = (
    var.vpc_id == null
    ? data.aws_vpc.default[0].id
    : var.vpc_id
  )

  subnet_ids = (
    var.subnet_ids == null
    ? data.aws_subnets.default[0].ids
    : var.subnet_ids
  )
}
```

Note that because the data sources use the count parameters, they are now arrays, so any time you reference them, you need to use array lookup syntax (i.e., [0]). Go through the code, and anywhere you find a reference to one of these data sources, replace it with a reference to one of the local variables you just added instead. Start by updating the `aws_subnets` data source to use `local.vpc_id`:

```
data "aws_subnets" "default" {
  count = var.subnet_ids == null ? 1 : 0
  filter {
    name   = "vpc-id"
    values = [local.vpc_id]
  }
}
```

Then, set the `subnet_ids` parameter of the `alb` module to `local.subnet_ids`:

```
module "alb" {
  source = "../../networking/alb"

  alb_name   = "hello-world-${var.environment}"
  subnet_ids = local.subnet_ids
}
```

In the `asg` module, make the following updates: set the `subnet_ids` parameter to `local.subnet_ids`, and in the `user_data` variables, update `db_address` and `db_port` to read data from `local.mysql_config`.

```
module "asg" {
  source = "../../cluster/asg-rolling-deploy"

  cluster_name = "hello-world-${var.environment}"
  ami          = var.ami
  instance_type = var.instance_type

  user_data = templatefile("${path.module}/user-data.sh", {
    server_port = var.server_port
    db_address  = local.mysql_config.address
    db_port     = local.mysql_config.port
    server_text = var.server_text
  })

  min_size      = var.min_size
  max_size      = var.max_size
  enable_autoscaling = var.enable_autoscaling

  subnet_ids      = local.subnet_ids
  target_group_arns = [aws_lb_target_group.asg.arn]
  health_check_type = "ELB"

  custom_tags = var.custom_tags
}
```

Finally, update the `vpc_id` parameter of the `aws_lb_target_group` to use `local.vpc_id`:

```
resource "aws_lb_target_group" "asg" {
  name      = "hello-world-${var.environment}"
  port      = var.server_port
  protocol  = "HTTP"
  vpc_id    = local.vpc_id

  health_check {
    path            = "/"
    protocol        = "HTTP"
    matcher         = "200"
    interval        = 15
    timeout         = 3
    healthy_threshold = 2
    unhealthy_threshold = 2
  }
}
```

With these updates, you can now choose to inject the VPC ID, subnet IDs, and/or MySQL config parameters into the `hello-world-app` module, or omit any of those parameters, and the module will use an appropriate data source to fetch those values by itself. Let's update the “Hello, World” app example to allow the MySQL config to be injected but omit the VPC ID and subnet ID parameters because using the default VPC is good enough for testing. Add a new input variable to *examples/hello-world-app/variables.tf*:

```
variable "mysql_config" {
  description = "The config for the MySQL DB"

  type = object({
    address = string
    port    = number
  })

  default = {
    address = "mock-mysql-address"
    port    = 12345
  }
}
```

Pass this variable through to the `hello-world-app` module in *examples/hello-world-app/main.tf*:

```
module "hello_world_app" {
  source = "../../modules/services/hello-world-app"

  server_text = "Hello, World"
  environment = "example"

  mysql_config = var.mysql_config
}
```

```

instance_type    = "t2.micro"
min_size        = 2
max_size        = 2
enable_autoscaling = false
ami             = data.aws_ami.ubuntu.id
}

```

You can now set this `mysql_config` variable in a unit test to any value you want. Create a unit test in `test/hello_world_app_example_test.go` with the following contents:

```

func TestHelloWorldAppExample(t *testing.T) {
    opts := &terraform.Options{
        // You should update this relative path to point at your
        // hello-world-app example directory!
        TerraformDir: "../examples/hello-world-app/standalone",
    }

    // Clean up everything at the end of the test
    defer terraform.Destroy(t, opts)
    terraform.InitAndApply(t, opts)

    albDnsName := terraform.OutputRequired(t, opts, "alb_dns_name")
    url := fmt.Sprintf("http://%s", albDnsName)

    maxRetries := 10
    timeBetweenRetries := 10 * time.Second

    http_helper.HttpGetWithRetryWithCustomValidation(
        t,
        url,
        nil,
        maxRetries,
        timeBetweenRetries,
        func(status int, body string) bool {
            return status == 200 &&
                strings.Contains(body, "Hello, World")
        },
    )
}

```

This code is nearly identical to the unit test for the `alb` example, with only two differences:

- The `TerraformDir` setting is pointing to the `hello-world-app` example instead of the `alb` example (be sure to update the path as necessary for your filesystem).
- Instead of using `http_helper.HttpGetWithRetry` to check for a 404 response, the test is using the `http_helper.HttpGetWithRetryWithCustomValidation` method to check for a 200 response and a body that contains the text “Hello, World.” That’s because the `User Data` script of the `hello-world-app` module

returns a 200 OK response that includes not only the server text but also other text, including HTML.

There's just one new thing you'll need to add to this test—set the `mysql_config` variable:

```
opts := &terraform.Options{
    // You should update this relative path to point at your
    // hello-world-app example directory!
    TerraformDir: "../examples/hello-world-app/standalone",

    Vars: map[string]interface{}{
        "mysql_config": map[string]interface{}{
            "address": "mock-value-for-test",
            "port":    3306,
        },
    },
}
```

The `Vars` parameter in `terraform.Options` allows you to set variables in your Terraform code. This code is passing in some mock data for the `mysql_config` variable. Alternatively, you could set this value to anything you want: for example, you could fire up a small, in-memory database at test time and set the address to that database's IP.

Run this new test using `go test`, specifying the `-run` argument to run *just* this test (otherwise, Go's default behavior is to run all tests in the current folder, including the ALB example test you created earlier):

```
$ go test -v -timeout 30m -run TestHelloWorldAppExample
```

```
(...)
```

```
PASS
```

```
ok      terraform-up-and-running      204.113s
```

If all goes well, the test will run `terraform apply`, make repeated HTTP requests to the load balancer, and, as soon as it gets back the expected response, run `terraform destroy` to clean everything up. All told, it should take only a few minutes, and you now have a reasonable unit test for the “Hello, World” app.

Running tests in parallel

In the previous section, you ran just a single test using the `-run` argument of the `go test` command. If you had omitted that argument, Go would've run all of your tests—sequentially. Although four to five minutes to run a single test isn't too bad for testing infrastructure code, if you have dozens of tests, and each one runs sequentially, it could take hours to run your entire test suite. To shorten the feedback loop, you want to run as many tests in parallel as you can.

To instruct Go to run your tests in parallel, the only change you need to make is to add `t.Parallel()` to the top of each test. Here it is in `test/hello_world_app_example_test.go`:

```
func TestHelloWorldAppExample(t *testing.T) {
    t.Parallel()

    opts := &terraform.Options{
        // You should update this relative path to point at your
        // hello-world-app example directory!
        TerraformDir: "../examples/hello-world-app/standalone",

        Vars: map[string]interface{}{
            "mysql_config": map[string]interface{}{
                "address": "mock-value-for-test",
                "port":    3306,
            },
        },
    }

    // (...)
}
```

And similarly in `test/alb_example_test.go`:

```
func TestAlbExample(t *testing.T) {
    t.Parallel()

    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
    }

    // (...)
}
```

If you run `go test` now, both of those tests will execute in parallel. However, there's one gotcha: some of the resources created by those tests—for example, the ASG, security group, and ALB—use the same name, which will cause the tests to fail due to the name clashes. Even if you weren't using `t.Parallel()` in your tests, if multiple people on your team were running the same tests or if you had tests running in a CI environment, these sorts of name clashes would be inevitable.

This leads to *key testing takeaway #4*: you must namespace all of your resources.

That is, design modules and examples so that the name of every resource is (optionally) configurable. With the `alb` example, this means that you need to make the name of the ALB configurable. Add a new input variable in `examples/alb/variables.tf` with a reasonable default:


```

variable "alb_name" {
  description = "The name of the ALB and all its resources"
  type        = string
  default     = "terraform-up-and-running"
}

```

Next, pass this value through to the `alb` module in `examples/alb/main.tf`:

```

module "alb" {
  source = "../modules/networking/alb"

  alb_name = var.alb_name
  subnet_ids = data.aws_subnets.default.ids
}

```

Now, set this variable to a unique value in `test/alb_example_test.go`:

```

package test

import (
    "fmt"
    "github.com/stretchr/testify/require"

    "github.com/gruntwork-io/terratest/modules/http-helper"
    "github.com/gruntwork-io/terratest/modules/random"
    "github.com/gruntwork-io/terratest/modules/terraform"
    "testing"
    "time"
)

func TestAlbExample(t *testing.T) {
    t.Parallel()

    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",

        Vars: map[string]interface{}{
            "alb_name": fmt.Sprintf("test-%s", random.UniqueId()),
        },
    }

    // (...)
}

```

This code sets the `alb_name` var to `test- $\langle$ RANDOM_ID $\rangle$` , where `RANDOM_ID` is a random unique ID returned by the `random.UniqueId()` helper in Terratest. This helper returns a randomized, six-character base-62 string. The idea is that it's a short identifier you can add to the names of most resources without hitting length-limit issues but random enough to make conflicts very unlikely ($62^6 = 56+$ billion

combinations). This ensures that you can run a huge number of ALB tests in parallel with no concern of having a name conflict.

Make a similar change to the “Hello, World” app example, first by adding a new input variable in *examples/hello-world-app/variables.tf*:

```
variable "environment" {  
    description = "The name of the environment we're deploying to"  
    type        = string  
    default     = "example"  
}
```

Then by passing that variable through to the hello-world-app module:

```
module "hello_world_app" {  
    source = "../../modules/services/hello-world-app"  
  
    server_text = "Hello, World"  
  
    environment = var.environment  
  
    mysql_config = var.mysql_config  
  
    instance_type    = "t2.micro"  
    min_size         = 2  
    max_size         = 2  
    enable_autoscaling = false  
    ami              = data.aws_ami.ubuntu.id  
}
```

Finally, setting `environment` to a value that includes `random.UniqueId()` in *test/hello_world_app_example_test.go*:

```
func TestHelloWorldAppExample(t *testing.T) {  
    t.Parallel()  
  
    opts := &terraform.Options{  
        // You should update this relative path to point at your  
        // hello-world-app example directory!  
        TerraformDir: "../examples/hello-world-app/standalone",  
  
        Vars: map[string]interface{}{  
            "mysql_config": map[string]interface{}{  
                "address": "mock-value-for-test",  
                "port":    3306,  
            },  
            "environment": fmt.Sprintf("test-%s", random.UniqueId()),  
        },  
    }  
  
    // (...)  
}
```

With these changes complete, it should now be safe to run all your tests in parallel:

```
$ go test -v -timeout 30m

TestAlbExample 2019-05-26T17:57:21+01:00 (...)
TestHelloWorldAppExample 2019-05-26T17:57:21+01:00 (...)
TestAlbExample 2019-05-26T17:57:21+01:00 (...)
TestHelloWorldAppExample 2019-05-26T17:57:21+01:00 (...)
TestHelloWorldAppExample 2019-05-26T17:57:21+01:00 (...)

(...)

PASS
ok      terraform-up-and-running      216.090s
```

You should see both tests running at the same time so that the entire test suite takes roughly as long as the slowest of the tests, rather than the combined time of all the tests running back to back.

Note that, by default, the number of tests Go will run in parallel is equal to how many CPUs you have on your computer. So if you only have one CPU, then by default, the tests will still run serially, rather than in parallel. You can override this setting by setting the `GOMAXPROCS` environment variable or by passing the `-parallel` argument to the `go test` command. For example, to force Go to run up to two tests in parallel, you would run the following:

```
$ go test -v -timeout 30m -parallel 2
```



Running Tests in Parallel in the Same Folder

One other type of parallelism to take into account is what happens if you try to run multiple automated tests in parallel against the same Terraform folder. For example, perhaps you'd want to run several different tests against *examples/hello-world-app*, where each test sets different values for the input variables before running `terraform apply`. If you try this, you'll hit a problem: the tests will end up clashing because they all try to run `terraform init` and end up overwriting one another's *.terraform* folder and Terraform state files.

If you want to run multiple tests against the same folder in parallel, the easiest solution is to have each test copy that folder to a unique temporary folder, and run Terraform in the temporary folder to avoid conflicts. Terratest, of course, has a built-in helper to do this for you, and it even does it in a way that ensures that relative file paths within those Terraform modules work correctly: check out the `test_structure.CopyTerraformFolderToTemp` method and its documentation for details.

Integration Tests

Now that you've got some unit tests in place, let's move on to integration tests. Again, it's helpful to start with the Ruby web server example to build up some intuition that you can later apply to the Terraform code. To do an integration test of the Ruby web server code, you need to do the following:

1. Run the web server on localhost so that it listens on a port.
2. Send HTTP requests to the web server.
3. Validate you get back the responses you expect.

Let's create a helper method in *web-server-test.rb* that implements these steps:

```
def do_integration_test(path, check_response)
  port = 8000
  server = WEBrick::HTTPServer.new :Port => port
  server.mount '/', WebServer

  begin
    # Start the web server in a separate thread so it
    # doesn't block the test
    thread = Thread.new do
      server.start
    end

    # Make an HTTP request to the web server at the
    # specified path
    uri = URI("http://localhost:#{port}#{path}")
    response = Net::HTTP.get_response(uri)

    # Use the specified check_response lambda to validate
    # the response
    check_response.call(response)
  ensure
    # Shut the server and thread down at the end of the
    # test
    server.shutdown
    thread.join
  end
end
```

The `do_integration_test` method configures the web server on port 8000, starts it in a background thread (so the web server doesn't block the test from running), sends an HTTP GET to the path specified, passes the HTTP response to the specified `check_response` function for validation, and at the end of the test, shuts down the web server. Here's how you can use this method to write an integration test for the / endpoint of the web server:

```
def test_integration_hello
  do_integration_test('/', lambda { |response|
    assert_equal(200, response.code.to_i)
    assert_equal('text/plain', response['Content-Type'])
    assert_equal('Hello, World', response.body)
  })
end
```

This method calls the `do_integration_test` method with the `/` path and passes it a lambda (essentially, an inline function) that checks the response was a 200 OK with the body “Hello, World.” The integration tests for the other endpoints are analogous. Let’s run all of the tests:

```
$ ruby web-server-test.rb

(...)

Finished in 0.221561 seconds.
-----
8 tests, 24 assertions, 0 failures, 0 errors
100% passed
-----
```

Note that before, with solely unit tests, the test suite took 0.000572 seconds to run, but now, with integration tests, it takes 0.221561 seconds, a slowdown of roughly 387 times. Of course, 0.221561 seconds is still blazing fast, but that’s only because the Ruby web server code is intentionally a minimal example that doesn’t do much. The important thing here is not the absolute numbers but the relative trend: integration tests are typically slower than unit tests. I’ll come back to this point later.

Let’s now turn our attention to integration tests for Terraform code. If a “unit” in Terraform is a single module, an integration test that validates how several units work together would need to deploy several modules and see that they work correctly. In the previous section, you deployed the “Hello, World” app example with mock data instead of a real MySQL DB. For an integration test, let’s deploy the MySQL module for real and make sure the “Hello, World” app integrates with it correctly. You should already have just such code under *live/stage/data-stores/mysql* and *live/stage/services/hello-world-app*. That is, you can create an integration test for (parts of) your staging environment.

Of course, as mentioned earlier in the chapter, all automated tests should run in an isolated AWS account. So while you’re testing the code that is meant for staging, you should authenticate to an isolated testing account and run the tests there. If your modules have anything in them hardcoded for the staging environment, this is the time to make those values configurable so you can inject test-friendly values. In particular, in *live/stage/data-stores/mysql/variables.tf*, expose the database name via a new `db_name` input variable:

```

variable "db_name" {
  description = "The name to use for the database"
  type        = string
  default     = "example_database_stage"
}

```

Pass that value through to the `mysql` module in `live/stage/data-stores/mysql/main.tf`:

```

module "mysql" {
  source = "../../../../../modules/data-stores/mysql"

  db_name      = var.db_name
  db_username  = var.db_username
  db_password  = var.db_password
}

```

Let's now create the skeleton of the integration test in `test/hello_world_integration_test.go` and fill in the implementation details later:

```

// Replace these with the proper paths to your modules
const dbDirStage = "../../live/stage/data-stores/mysql"
const appDirStage = "../../live/stage/services/hello-world-app"

func TestHelloWorldAppStage(t *testing.T) {
    t.Parallel()

    // Deploy the MySQL DB
    dbOpts := createDbOpts(t, dbDirStage)
    defer terraform.Destroy(t, dbOpts)
    terraform.InitAndApply(t, dbOpts)

    // Deploy the hello-world-app
    helloOpts := createHelloOpts(dbOpts, appDirStage)
    defer terraform.Destroy(t, helloOpts)
    terraform.InitAndApply(t, helloOpts)

    // Validate the hello-world-app works
    validateHelloApp(t, helloOpts)
}

```

The test is structured as follows: deploy `mysql`, deploy the `hello-world-app`, validate the app, undeploy the `hello-world-app` (runs at the end due to `defer`), and, finally, undeploy `mysql` (runs at the end due to `defer`). The `createDbOpts`, `createHelloOpts`, and `validateHelloApp` methods don't exist yet, so let's implement them one at a time, starting with the `createDbOpts` method:

```

func createDbOpts(t *testing.T, terraformDir string) *terraform.Options {
    uniqueId := random.UniqueId()

    return &terraform.Options{
        TerraformDir: terraformDir,

        Vars: map[string]interface{}{

```

```

        "db_name":      fmt.Sprintf("test%s", uniqueId),
        "db_username":  "admin",
        "db_password":  "password",
    },
}
}

```

Not much new so far: the code points `terraform.Options` at the passed-in directory and sets the `db_name`, `db_username`, and `db_password` variables.

The next step is to deal with where this `mysql` module will store its state. Up to now, the backend configuration has been set to hardcoded values:

```

backend "s3" {
    # Replace this with your bucket name!
    bucket      = "terraform-up-and-running-state"
    key         = "stage/data-stores/mysql/terraform.tfstate"
    region      = "us-east-2"

    # Replace this with your DynamoDB table name!
    dynamodb_table = "terraform-up-and-running-locks"
    encrypt        = true
}

```

These hardcoded values are a big problem for testing, because if you don't change them, you'll end up overwriting the real state file for staging! One option is to use Terraform workspaces (as discussed in [“Isolation via Workspaces” on page 94](#)), but that would still require access to the S3 bucket in the staging account, whereas you should be running tests in a totally separate AWS account. The better option is to use partial configuration, as introduced in [“Limitations with Terraform's Backends” on page 91](#). Move the entire backend configuration into an external file, such as *backend.hcl*:

```

bucket      = "terraform-up-and-running-state"
key         = "stage/data-stores/mysql/terraform.tfstate"
region      = "us-east-2"
dynamodb_table = "terraform-up-and-running-locks"
encrypt      = true

```

leaving the backend configuration in *live/stage/data-stores/mysql/main.tf* empty:

```

backend "s3" {
}

```

When you're deploying the `mysql` module to the real staging environment, you tell Terraform to use the backend configuration in *backend.hcl* via the `-backend-config` argument:

```
$ terraform init -backend-config=backend.hcl
```

When you're running tests on the `mysql` module, you can tell Terratest to pass in test-time-friendly values using the `BackendConfig` parameter of `terraform.Options`:

```
func createDbOpts(t *testing.T, terraformDir string) *terraform.Options {
    uniqueId := random.UniqueId()

    bucketForTesting := "YOUR_S3_BUCKET_FOR_TESTING"
    bucketRegionForTesting := "YOUR_S3_BUCKET_REGION_FOR_TESTING"
    dbStateKey := fmt.Sprintf("%s/%s/terraform.tfstate", t.Name(), uniqueId)

    return &terraform.Options{
        TerraformDir: terraformDir,

        Vars: map[string]interface{}{
            "db_name":      fmt.Sprintf("test%s", uniqueId),
            "db_username":  "admin",
            "db_password":  "password",
        },

        BackendConfig: map[string]interface{}{
            "bucket": bucketForTesting,
            "region": bucketRegionForTesting,
            "key":    dbStateKey,
            "encrypt": true,
        },
    }
}
```

You'll need to update the `bucketForTesting` and `bucketRegionForTesting` variables with your own values. You can create a single S3 bucket in your test AWS account to use as a backend, as the key configuration (the path within the bucket) includes the `uniqueId`, which should be unique enough to have a different value for each test.

The next step is to make some updates to the `hello-world-app` module in the staging environment. Open `live/stage/services/hello-world-app/variables.tf`, and expose variables for `db_remote_state_bucket`, `db_remote_state_key`, and `environment`:

```
variable "db_remote_state_bucket" {
    description = "The name of the S3 bucket for the database's remote state"
    type        = string
}

variable "db_remote_state_key" {
    description = "The path for the database's remote state in S3"
    type        = string
}

variable "environment" {
    description = "The name of the environment we're deploying to"
    type        = string
}
```



```

    default      = "stage"
}

```

Pass those values through to the `hello-world-app` module in `live/stage/services/hello-world-app/main.tf`:

```

module "hello_world_app" {
    source = "../../../../../modules/services/hello-world-app"

    server_text      = "Hello, World"

    environment      = var.environment
    db_remote_state_bucket = var.db_remote_state_bucket
    db_remote_state_key   = var.db_remote_state_key

    instance_type    = "t2.micro"
    min_size         = 2
    max_size         = 2
    enable_autoscaling = false
    ami              = data.aws_ami.ubuntu.id
}

```

Now you can implement the `createHelloOpts` method:

```

func createHelloOpts(
    dbOpts *terraform.Options,
    terraformDir string) *terraform.Options {

    return &terraform.Options{
        TerraformDir: terraformDir,

        Vars: map[string]interface{}{
            "db_remote_state_bucket": dbOpts.BackendConfig["bucket"],
            "db_remote_state_key":   dbOpts.BackendConfig["key"],
            "environment":           dbOpts.Vars["db_name"],
        },
    }
}

```

Note that `db_remote_state_bucket` and `db_remote_state_key` are set to the values used in the `BackendConfig` for the `mysql` module to ensure that the `hello-world-app` module is reading from the exact same state to which the `mysql` module just wrote. The `environment` variable is set to the `db_name` just so all the resources are name-spaced the same way.

Finally, you can implement the `validateHelloApp` method:

```

func validateHelloApp(t *testing.T, helloOpts *terraform.Options) {
    albDnsName := terraform.OutputRequired(t, helloOpts, "alb_dns_name")
    url := fmt.Sprintf("http://%s", albDnsName)

    maxRetries := 10
    timeBetweenRetries := 10 * time.Second
}

```

```

        http_helper.HttpGetWithRetryWithCustomValidation(
            t,
            url,
            nil,
            maxRetries,
            timeBetweenRetries,
            func(status int, body string) bool {
                return status == 200 &&
                    strings.Contains(body, "Hello, World")
            },
        )
    }
}

```

This method uses the `http_helper` package, just as with the unit tests, except this time, it's with the `http_helper.HttpGetWithRetryWithCustomValidation` method that allows you to specify custom validation rules for the HTTP response status code and body. This is necessary to check that the HTTP response *contains* the string “Hello, World,” rather than equals that string exactly, as the User Data script in the `hello-world-app` module returns an HTML response with other text in it as well.

Alright, run the integration test to see whether it worked:

```
$ go test -v -timeout 30m -run "TestHelloWorldAppStage"
```

```
(...)
```

```
PASS
```

```
ok      terraform-up-and-running      795.63s
```

Excellent, you now have an integration test that you can use to check that several of your modules work correctly together. This integration test is more complicated than the unit test, and it takes more than twice as long (10–15 minutes rather than 4–5 minutes). In general, there's not much that you can do to make things *faster*—the bottleneck here is how long AWS takes to deploy and undeploy RDS, ASGs, ALBs, etc.—but in certain circumstances, you might be able to make the test code do *less* using *test stages*.

Test stages

If you look at the code for your integration test, you may notice that it consists of five distinct “stages”:

1. Run `terraform apply` on the `mysql` module.
2. Run `terraform apply` on the `hello-world-app` module.
3. Run validations to make sure everything is working.
4. Run `terraform destroy` on the `hello-world-app` module.

5. Run `terraform destroy` on the `mysql` module.

When you run these tests in a CI environment, you'll want to run all of the stages, from start to finish. However, if you're running these tests in your local dev environment while iteratively making changes to the code, running all of these stages is unnecessary. For example, if you're making changes only to the `hello-world-app` module, rerunning this entire test after every change means you're paying the price of deploying and undeploying the `mysql` module, even though none of your changes affect it. That adds 5 to 10 minutes of pure overhead to every test run.

Ideally, the workflow would look more like this:

1. Run `terraform apply` on the `mysql` module.
2. Run `terraform apply` on the `hello-world-app` module.
3. Now, you start doing iterative development:
 - a. Make a change to the `hello-world-app` module.
 - b. Rerun `terraform apply` on the `hello-world-app` module to deploy your updates.
 - c. Run validations to make sure everything is working.
 - d. If everything works, move on to the next step. If not, go back to step 3a.
4. Run `terraform destroy` on the `hello-world-app` module.
5. Run `terraform destroy` on the `mysql` module.

Having the ability to quickly do that inner loop in step 3 is the key to fast, iterative development with Terraform. To support this, you need to break your test code into *stages*, in which you can choose the stages to execute and those that you can skip.

Terratest supports this natively with the `test_structure` package. The idea is that you wrap each stage of your test in a function with a name, and you can then direct Terratest to skip some of those names by setting environment variables. Each test stage stores test data on disk so that it can be read back from disk on subsequent test runs. Let's try this out on `test/hello_world_integration_test.go`, writing the skeleton of the test first and then filling in the underlying methods later:

```
func TestHelloWorldAppStageWithStages(t *testing.T) {
    t.Parallel()

    // Store the function in a short variable name solely to make the
    // code examples fit better in the book.
    stage := test_structure.RunTestStage

    // Deploy the MySQL DB
    defer stage(t, "teardown_db", func() { teardownDb(t, dbDirStage) })
}
```

```

stage(t, "deploy_db", func() { deployDb(t, dbDirStage) })

// Deploy the hello-world-app
defer stage(t, "teardown_app", func() { teardownApp(t, appDirStage) })
stage(t, "deploy_app", func() { deployApp(t, dbDirStage, appDirStage) })

// Validate the hello-world-app works
stage(t, "validate_app", func() { validateApp(t, appDirStage) })
}

```

The structure is the same as before—deploy mysql, deploy hello-world-app, validate hello-world-app, undeploy hello-world-app (runs at the end due to defer), undeploy mysql (runs at the end due to defer)—except now, each stage is wrapped in `test_structure.RunTestStage`. The `RunTestStage` method takes three arguments:

`t`

The first argument is the `t` value that Go passes as an argument to every automated test. You can use this value to manage test state. For example, you can fail the test by calling `t.Fail()`.

Stage name

The second argument allows you to specify the name for this test stage. You'll see an example shortly of how to use this name to skip test stages.

The code to execute

The third argument is the code to execute for this test stage. This can be any function.

Let's now implement the functions for each test stage, starting with `deployDb`:

```

func deployDb(t *testing.T, dbDir string) {
    dbOpts := createDbOpts(t, dbDir)

    // Save data to disk so that other test stages executed at a later
    // time can read the data back in
    test_structure.SaveTerraformOptions(t, dbDir, dbOpts)

    terraform.InitAndApply(t, dbOpts)
}

```

Just as before, to deploy mysql, the code calls `createDbOpts` and `terraform.InitAndApply`. The only new thing is that, in between those two steps, there is a call to `test_structure.SaveTerraformOptions`. This writes the data in `dbOpts` to disk so that other test stages can read it later on. For example, here's the implementation of the `teardownDb` function:

```

func teardownDb(t *testing.T, dbDir string) {
    dbOpts := test_structure.LoadTerraformOptions(t, dbDir)
    defer terraform.Destroy(t, dbOpts)
}

```

This function uses `test_structure.LoadTerraformOptions` to load the `dbOpts` data from disk that was earlier saved by the `deployDb` function. The reason you need to pass this data via the hard drive rather than passing it in memory is that you can run each test stage as part of a different test run—and therefore, as part of a different process. As you’ll see a little later in this chapter, on the first few runs of `go test`, you might want to run `deployDb` but skip `teardownDb`, and then in later runs do the opposite, running `teardownDb` but skipping `deployDb`. To ensure that you’re using the same database across all those test runs, you must store that database’s information on disk.

Let’s now implement the `deployHelloApp` function:

```
func deployApp(t *testing.T, dbDir string, helloAppDir string) {
    dbOpts := test_structure.LoadTerraformOptions(t, dbDir)
    helloOpts := createHelloOpts(dbOpts, helloAppDir)

    // Save data to disk so that other test stages executed at a later
    // time can read the data back in
    test_structure.SaveTerraformOptions(t, helloAppDir, helloOpts)

    terraform.InitAndApply(t, helloOpts)
}
```

This function reuses the `createHelloOpts` function from before and calls `terraform.InitAndApply` on it. Again, the only new behavior is the use of `test_structure.LoadTerraformOptions` to load `dbOpts` from disk and the use of `test_structure.SaveTerraformOptions` to save `helloOpts` to disk. At this point, you can probably guess what the implementation of the `teardownApp` method looks like:

```
func teardownApp(t *testing.T, helloAppDir string) {
    helloOpts := test_structure.LoadTerraformOptions(t, helloAppDir)
    defer terraform.Destroy(t, helloOpts)
}
```

And the implementation of the `validateApp` method:

```
func validateApp(t *testing.T, helloAppDir string) {
    helloOpts := test_structure.LoadTerraformOptions(t, helloAppDir)
    validateHelloApp(t, helloOpts)
}
```

So, overall, the test code is identical to the original integration test, except each stage is wrapped in a call to `test_structure.RunTestStage`, and you need to do a little work to save and load data to and from disk. These simple changes unlock an important ability: you can instruct Terratest to skip any test stage called `foo` by setting the environment variable `SKIP_foo=true`. Let’s go through a typical coding workflow to see how this works.

Your first step will be to run the test but to skip both of the teardown stages so that the `mysql` and `hello-world-app` modules stay deployed at the end of the test. Because the teardown stages are called `teardown_db` and `teardown_app`, you need to set the `SKIP_teardown_db` and `SKIP_teardown_app` environment variables, respectively, to direct Terratest to skip those two stages:

```
$ SKIP_teardown_db=true \  
  SKIP_teardown_app=true \  
  go test -timeout 30m -run 'TestHelloWorldAppStageWithStages'
```

(...)

The 'SKIP_deploy_db' environment variable is not set,
so executing stage 'deploy_db'.

(...)

The 'deploy_app' environment variable is not set,
so executing stage 'deploy_db'.

(...)

The 'validate_app' environment variable is not set,
so executing stage 'deploy_db'.

(...)

The 'teardown_app' environment variable is set,
so skipping stage 'deploy_db'.

(...)

The 'teardown_db' environment variable is set,
so skipping stage 'deploy_db'.

(...)

```
PASS  
ok      terraform-up-and-running    423.650s
```

Now you can start iterating on the `hello-world-app` module, and each time you make a change, you can rerun the tests, but this time, direct them to skip not only the teardown stages but also the `mysql` module deploy stage (because `mysql` is still running) so that the only things that execute are `deploy app` and the validations for the `hello-world-app` module:

```
$ SKIP_teardown_db=true \  
  SKIP_teardown_app=true \  
  SKIP_deploy_db=true \  
  go test -timeout 30m -run 'TestHelloWorldAppStageWithStages'
```

```
(...)

The 'SKIP_deploy_db' environment variable is set,
so skipping stage 'deploy_db'.

(...)

The 'deploy_app' environment variable is not set,
so executing stage 'deploy_db'.

(...)

The 'validate_app' environment variable is not set,
so executing stage 'deploy_db'.

(...)

The 'teardown_app' environment variable is set,
so skipping stage 'deploy_db'.

(...)

The 'teardown_db' environment variable is set,
so skipping stage 'deploy_db'.

(...)

PASS
ok      terraform-up-and-running      13.824s
```

Notice how fast each of these test runs is now: instead of waiting 10 to 15 minutes after every change, you can try out new changes in 10 to 60 seconds (depending on the change). Given that you're likely to rerun these stages dozens or even hundreds of times during development, the time savings can be massive.

Once the `hello-world-app` module changes are working the way you expect, it's time to clean everything up. Run the tests once more, this time skipping the deploy and validation stages so that only the teardown stages are executed:

```
$ SKIP_deploy_db=true \
  SKIP_deploy_app=true \
  SKIP_validate_app=true \
  go test -timeout 30m -run 'TestHelloWorldAppStageWithStages'

(...)

The 'SKIP_deploy_db' environment variable is set,
so skipping stage 'deploy_db'.

(...)

The 'SKIP_deploy_app' environment variable is set,
```

```

so skipping stage 'deploy_app'.

(...)

The 'SKIP_validate_app' environment variable is set,
so skipping stage 'validate_app'.

(...)

The 'SKIP_teardown_app' environment variable is not set,
so executing stage 'teardown_app'.

(...)

The 'SKIP_teardown_db' environment variable is not set,
so executing stage 'teardown_db'.

(...)

PASS
ok      terraform-up-and-running      340.02s

```

Using test stages lets you get rapid feedback from your automated tests, dramatically increasing the speed and quality of iterative development. It won't make any difference in how long tests take in your CI environment, but the impact on the development environment is huge.

Retries

After you start running automated tests for your infrastructure code on a regular basis, you're likely to run into a problem: flaky tests. That is, tests occasionally will fail for transient reasons, such as an EC2 Instance occasionally failing to launch, or a Terraform eventual consistency bug, or a TLS handshake error talking to S3. The infrastructure world is a messy place, so you should expect intermittent failures in your tests and handle them accordingly.

To make your tests a bit more resilient, you can add retries for known errors. For example, while writing this book, I'd occasionally get the following type of error, especially when running many tests in parallel:

```

* error loading the remote state: RequestError: send request failed
Post https://xxx.amazonaws.com/: dial tcp xx.xx.xx.xx:443:
connect: connection refused

```

To make tests more reliable in the face of such errors, you can enable retries in `Teratest` using the `MaxRetries`, `TimeBetweenRetries`, and `RetryableTerraformErrors` arguments of `terraform.Options`:

```

func createHelloOpts(
    dbOpts *terraform.Options,
    terraformDir string) *terraform.Options {

```



```

return &terraform.Options{
    TerraformDir: terraformDir,

    Vars: map[string]interface{}{
        "db_remote_state_bucket": dbOpts.BackendConfig["bucket"],
        "db_remote_state_key":    dbOpts.BackendConfig["key"],
        "environment":            dbOpts.Vars["db_name"],
    },

    // Retry up to 3 times, with 5 seconds between retries,
    // on known errors
    MaxRetries:      3,
    TimeBetweenRetries: 5 * time.Second,
    RetryableTerraformErrors: map[string]string{
        "RequestError: send request failed": "Throttling issue?",
    },
}
}

```

In the `RetryableTerraformErrors` argument, you can specify a map of known errors that warrant a retry: the keys of the map are the error messages to look for in the logs (you can use regular expressions here), and the values are additional information to display in the logs when Terratest matches one of these errors and kicks off a retry. Now, whenever your test code hits one of these known errors, you should see a message in your logs, followed by a sleep of `TimeBetweenRetries`, and then your command will rerun:

```

$ go test -v -timeout 30m

(...)

Running command terraform with args [apply -input=false -lock=false
-auto-approve]

(...)

* error loading the remote state: RequestError: send request failed
Post https://s3.amazonaws.com/: dial tcp 11.22.33.44:443:
connect: connection refused

(...)

'terraform [apply]' failed with the error 'exit status code 1'
but this error was expected and warrants a retry. Further details:
Intermittent error, possibly due to throttling?

(...)

Running command terraform with args [apply -input=false -lock=false
-auto-approve]

```

End-to-End Tests

Now that you have unit tests and integration tests in place, the final type of tests that you might want to add are *end-to-end* tests. With the Ruby web server example, end-to-end tests might consist of deploying the web server and any data stores it depends on and testing it from the web browser using a tool such as Selenium. The end-to-end tests for Terraform infrastructure will look similar: deploy everything into an environment that mimics production, and test it from the end-user's perspective.

Although you could write your end-to-end tests using the exact same strategy as the integration tests—that is, create a few dozen test stages to run `terraform apply`, do some validations, and then run `terraform destroy`—this is rarely done in practice. The reason for this has to do with the *test pyramid*, which you can see in [Figure 9-1](#).

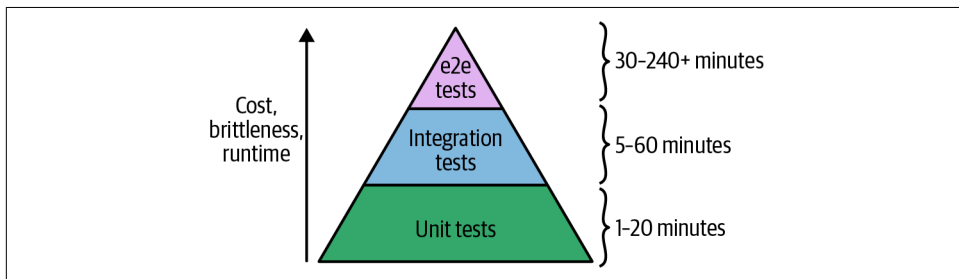


Figure 9-1. The test pyramid.

The idea of the test pyramid is that you should typically be aiming for a large number of unit tests (the bottom of the pyramid), a smaller number of integration tests (the middle of the pyramid), and an even smaller number of end-to-end tests (the top of the pyramid). This is because, as you go up the pyramid, the cost and complexity of writing the tests, the brittleness of the tests, and the runtime of the tests all increase.

That gives us *key testing takeaway #5*: smaller modules are easier and faster to test.

You saw in the previous sections that it required a fair amount of work with name-spacing, dependency injection, retries, error handling, and test stages to test even a relatively simple `hello-world-app` module. With larger and more complicated infrastructure, this only becomes more difficult. Therefore, you want to do as much of your testing as low in the pyramid as you can because the bottom of the pyramid offers the fastest, most reliable feedback loop.

In fact, by the time you get to the top of the test pyramid, running tests to deploy a complicated architecture from scratch becomes untenable for two main reasons:

Too slow

Deploying your entire architecture from scratch and then undeploying it all again can take a very long time: on the order of several hours. Test suites that

take that long provide relatively little value because the feedback loop is simply too slow. You'd probably run such a test suite only overnight, which means in the morning you'll get a report about a test failure, you'll investigate for a while, submit a fix, and then wait for the next day to see whether it worked. That limits you to roughly one bug fix attempt per day. In these sorts of situations, what actually happens is developers begin blaming others for test failures, convince management to deploy despite the test failures, and eventually ignore the test failures entirely.

Too brittle

As mentioned in the previous section, the infrastructure world is messy. As the amount of infrastructure you're deploying goes up, the odds of hitting an intermittent, flaky issue goes up as well. For example, suppose that a single resource (such as an EC2 Instance) has a one-in-a-thousand chance (0.1%) of failing due to an intermittent error (actual failure rates in the DevOps world are likely higher). This means that the probability that a test that deploys a single resource runs without any intermittent errors is 99.9%. So what about a test that deploys two resources? For that test to succeed, you need both resources to deploy without intermittent errors, and to calculate those odds, you multiply the probabilities: $99.9\% \times 99.9\% = 99.8\%$. With three resources, the odds are $99.9\% \times 99.9\% \times 99.9\% = 99.7\%$. With N resources, the formula is $99.9\%^N$.

So now let's consider different types of automated tests. If you had a unit test of a single module that deployed, say, 20 resources, the odds of success are $99.9\%^{20} = 98.0\%$. This means that 2 test runs out of 100 will fail; if you add a few retries, you can typically make these tests fairly reliable. Now, suppose that you had an integration test of 3 modules that deployed 60 resources. Now the odds of success are $99.9\%^{60} = 94.1\%$. Again, with enough retry logic, you can typically make these tests stable enough to be useful. So what happens if you want to write an end-to-end test that deploys your entire infrastructure, which consists of 30 modules, or about 600 resources? The odds of success are $99.9\%^{600} = 54.9\%$. This means that nearly half of your test runs will fail for transient reasons!

You'll be able to handle some of these errors with retries, but it quickly turns into a never-ending game of whack-a-mole. You add a retry for a TLS handshake timeout, only to be hit by an APT repo downtime in your Packer template; you add retries to the Packer build, only to have the build fail due to a Terraform eventual-consistency bug; just as you are applying the Band-Aid to that, the build fails due to a brief GitHub outage. And because end-to-end tests take so long, you get only one attempt, maybe two, per day to fix these issues.

In practice, very few companies with complicated infrastructure run end-to-end tests that deploy everything *from scratch*. Instead, the more common test strategy for end-to-end tests works as follows:

1. One time, you pay the cost of deploying a persistent, production-like environment called “test,” and you leave that environment running.
2. Every time someone makes a change to your infrastructure code, the end-to-end test does the following:
 - a. Applies the infrastructure change to the test environment.
 - b. Runs validations against the test environment (e.g., uses Selenium to test your code from the end-user’s perspective) to make sure everything is working.

By changing your end-to-end test strategy to applying only incremental changes, you’re reducing the number of resources that are being deployed at test time from several hundred to just a handful so that these tests will be faster and less brittle.

Moreover, this approach to end-to-end testing more closely mimics how you’ll be deploying those changes in production. After all, it’s not like you tear down and bring up your production environment from scratch to roll out each change. Instead, you apply each change incrementally, so this style of end-to-end testing offers a huge advantage: you can test not only that your infrastructure works correctly but also that the *deployment process* for that infrastructure works correctly, too.

Other Testing Approaches

Most of this chapter has focused on testing your Terraform code by doing a full `apply` and `destroy` cycle. This is the gold standard of testing, but there are three other types of automated tests you can use:

- Static analysis
- Plan testing
- Server testing

Just as unit, integration, and end-to-end tests each catch different types of bugs, each of the testing approaches just mentioned will catch different types of bugs as well, so you’ll most likely want to use several of these techniques together to get the best results. Let’s go through these new categories one at a time.

Static analysis

Static analysis is the most basic way to test your Terraform code: you parse the code and analyze it without actually executing it in any way. [Table 9-1](#) shows some of the tools in this space that work with Terraform and how they compare in terms of popularity and maturity, based on stats I gathered from GitHub in February 2022.

Table 9-1. A comparison of popular static analysis tools for Terraform

	terraform validate	tfsec	tfLint	Terrascan
Brief description	Built-in Terraform command	Spot potential security issues	Pluggable Terraform linter	Detect compliance and security violations
License	(same as Terraform)	MIT	MPL 2.0	Apache 2.0
Backing company	(same as Terraform)	Aqua Security	(none)	Accurics
Stars	(same as Terraform)	3,874	2,853	2,768
Contributors	(same as Terraform)	96	77	63
First release	(same as Terraform)	2019	2016	2017
Latest release	(same as Terraform)	v1.1.2	v0.34.1	v1.13.0
Built-in checks	Syntax checks only	AWS, Azure, GCP, Kubernetes, DigitalOcean, etc.	AWS, Azure, and GCP	AWS, Azure, GCP, Kubernetes, etc.
Custom checks	Not supported	Defined in YAML or JSON	Defined in a Go plugin	Defined in Rego

The simplest of these tools is `terraform validate`, which is built into Terraform itself, which can catch syntax issues. For example, if you forgot to set the `alb_name` parameter in *examples/alb*, and you ran `validate`, you would get output similar to the following:

```
$ terraform validate
| Error: Missing required argument
|
|   on main.tf line 20, in module "alb":
|   20: module "alb" {
|
|   The argument "alb_name" is required, but no definition was found.
```

Note that `validate` is limited solely to syntactic checks, whereas the other tools allow you to enforce other types of policies. For example, you can use tools such as `tfsec` and `tfLint` to enforce policies, such as:

- Security groups cannot be too open: e.g., block inbound rules that allow access from all IPs (CIDR block `0.0.0.0/0`).
- All EC2 Instances must follow a specific tagging convention.

The idea here is to *define your policies as code*, so you can enforce your security, compliance, and reliability requirements as code. In the next few sections, you'll see several other policy as code tools.

Strengths of static analysis tools

- They run fast.
- They are easy to use.
- They are stable (no flaky tests).
- You don't need to authenticate to a real provider (e.g., to a real AWS account).
- You don't have to deploy/undeploy real resources.

Weaknesses of static analysis tools

- They are very limited in the types of errors they can catch. Namely, they can only catch errors that can be determined from statically reading the code, without executing it: e.g., syntax errors, type errors, and a small subset of business logic errors. For example, you can detect a policy violation for static values, such as a security group hardcoded to allow inbound access from CIDR block `0.0.0.0/0`, but you can't detect policy violations from dynamic values, such as the same security group but with the inbound CIDR block being read in from a variable or file.
- These tests aren't checking functionality, so it's possible for all the checks to pass and the infrastructure still doesn't work!

Plan testing

Another way to test your code is to run `terraform plan` and to analyze the plan output. Since you're executing the code, this is more than static analysis, but it's less than a unit or integration test, as you're not executing the code fully: in particular, `plan` executes the read steps (e.g., fetching state, executing data sources) but not the write steps (e.g., creating or modifying resources). [Table 9-2](#) shows some of the tools that do `plan` testing and how they compare in terms of popularity and maturity, based on stats I gathered from GitHub in February 2022.

Table 9-2. A comparison of popular plan testing tools for Terraform

	Terratest	Open Policy Agent (OPA)	HashiCorp Sentinel	Checkov	terraform-compliance
Brief description	Go library for IaC testing	General-purpose policy engine	Policy-as-code for HashiCorp enterprise products	Policy-as-code for everyone	BDD test framework for Terraform
License	Apache 2.0	Apache 2.0	Commercial / proprietary license	Apache 2.0	MIT
Backing company	Gruntwork	Styra	HashiCorp	Bridgecrew	(none)
Stars	5,888	6,207	(not open source)	3,758	1,104
Contributors	157	237	(not open source)	199	36
First release	2016	2016	2017	2019	2018

	Terratest	Open Policy Agent (OPA)	HashiCorp Sentinel	Checkov	terraform-compliance
Latest release	v0.40.0	v0.37.1	v0.18.5	2.0.810	1.3.31
Built-in checks	None	None	None	AWS, Azure, GCP, Kubernetes, etc.	None
Custom checks	Defined in Go	Defined in Rego	Defined in Sentinel	Defined in Python or YAML	Defined in BDD

Since you're already familiar with Terratest, let's take a quick look at how you can use it to do plan testing on the code in *examples/alb*. If you ran `terraform plan` manually, here's a snippet of the output you'd get:

Terraform will perform the following actions:

```
# module.alb.aws_lb.example will be created
+ resource "aws_lb" "example" {
+   arn                        = (known after apply)
+   load_balancer_type        = "application"
+   name                      = "test-4Ti6CP"
+   (...)
}
```

(...)

Plan: 5 to add, 0 to change, 0 to destroy.

How can you test this output programmatically? Here's the basic structure of a test that uses Terratest's `InitAndPlan` helper to run `init` and `plan` automatically:

```
func TestAlbExamplePlan(t *testing.T) {
    t.Parallel()

    albName := fmt.Sprintf("test-%s", random.UniqueId())

    opts := &terraform.Options{
        // You should update this relative path to point at your alb
        // example directory!
        TerraformDir: "../examples/alb",
        Vars: map[string]interface{}{
            "alb_name": albName,
        },
    }

    planString := terraform.InitAndPlan(t, opts)
}
```

Even this minimal test offers some value, in that it validates that your code can successfully run `plan`, which checks that the syntax is valid and that all the read API calls work. But you can go even further. One small improvement is to check that you

get the expected counts at the end of the plan: “5 to add, 0 to change, 0 to destroy.” You can do this using the `GetResourceCount` helper

```
// An example of how to check the plan output's add/change/destroy counts
resourceCounts := terraform.GetResourceCount(t, planString)
require.Equal(t, 5, resourceCounts.Add)
require.Equal(t, 0, resourceCounts.Change)
require.Equal(t, 0, resourceCounts.Destroy)
```

You can do an even more thorough check by using the `InitAndPlanAndShowWithStructNoLogTempPlanFile` helper to parse the plan output into a struct, which gives you programmatic access to all the values and changes in that plan output. For example, you could check that the plan output includes the `aws_lb` resource at address `module.alb.aws_lb.example` and that the `name` attribute of this resource is set to the expected value, as follows:

```
// An example of how to check specific values in the plan output
planStruct :=
    terraform.InitAndPlanAndShowWithStructNoLogTempPlanFile(t, opts)

alb, exists :=
    planStruct.ResourcePlannedValuesMap["module.alb.aws_lb.example"]
require.True(t, exists, "aws_lb resource must exist")

name, exists := alb.AttributeValues["name"]
require.True(t, exists, "missing name parameter")
require.Equal(t, albName, name)
```

The strength of Terratest’s approach to plan testing is that it’s extremely flexible, as you can write arbitrary Go code to check whatever you want. But this very same factor is also, in some ways, a weakness, as it makes it harder to get started.

Some teams prefer a more declarative language for defining their policies as code. In the last few years, Open Policy Agent (OPA) has become a popular *policy-as-code* tool, as it allows you to capture your company’s policies as code in a declarative language called Rego.

For example, many companies have tagging policies they want to enforce. A common one with Terraform code is to ensure that every resource that is managed by Terraform has a `ManagedBy = terraform` tag. Here is a simple policy called *enforce_tagging.rego* you could use to check for this tag:

```
package terraform

allow {
    resource_change := input.resource_changes[_]
    resource_change.change.after.tags["ManagedBy"]
}
```


This policy will look through the changes in a `terraform plan` output, extract the tag `ManagedBy`, and set an OPA variable called `allow` to `true` if that tag is set or `undefined` otherwise.

Now, consider the following Terraform module:

```
resource "aws_instance" "example" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t2.micro"  
}
```

This module is not setting the required `ManagedBy` tag. How can we catch that with OPA?

The first step is to run `terraform plan` and to save the output to a plan file:

```
$ terraform plan -out tfplan.binary
```

OPA only operates on JSON, so the next step is to convert the plan file to JSON using the `terraform show` command:

```
$ terraform show -json tfplan.binary > tfplan.json
```

Finally, you can run the `opa eval` command to check this plan file against the *enforce_tagging.rego* policy:

```
$ opa eval \  
--data enforce_tagging.rego \  
--input tfplan.json \  
--format pretty \  
data.terraform.allow
```

```
undefined
```

Since the `ManagedBy` tag was not set, the output from OPA is `undefined`. Now, try setting the `ManagedBy` tag:

```
resource "aws_instance" "example" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t2.micro"  
  
  tags = {  
    ManagedBy = "terraform"  
  }  
}
```

Rerun `terraform plan`, `terraform show`, and `opa eval`:

```
$ terraform plan -out tfplan.binary
```

```
$ terraform show -json tfplan.binary > tfplan.json
```

```
$ opa eval \  
--data enforce_tagging.rego \  
--input tfplan.json
```

```
--input tfplan.json \  
--format pretty \  
data.terraform.allow  
  
true
```

This time, the output is `true`, which means the policy has passed.

Using tools like OPA, you can enforce your company’s requirements by creating a library of such policies and setting up a CI/CD pipeline that runs these policies against your Terraform modules after every commit.

Strengths of plan testing tools

- They run fast—not quite as fast as pure static analysis but much faster than unit or integration tests.
- They are somewhat easy to use—not quite as easy as pure static analysis but much easier than unit or integration tests.
- They are stable (few flaky tests)—not quite as stable as pure static analysis but much more stable than unit or integration tests.
- You don’t have to deploy/undeploy real resources.

Weaknesses of plan testing tools

- They are limited in the types of errors they can catch. They can catch more than pure static analysis but nowhere near as many errors as unit and integration testing.
- You have to authenticate to a real provider (e.g., to a real AWS account). This is required for `plan` to work.
- These tests aren’t checking functionality, so it’s possible for all the checks to pass and the infrastructure still doesn’t work!

Server testing

There are a set of testing tools that are focused on testing that your servers (including virtual servers) have been properly configured. I’m not aware of any common name for these sorts of tools, so I’ll call it *server testing*. These are not general-purpose tools for testing all aspects of your Terraform code. In fact, most of these tools were originally built to be used with configuration management tools, such as Chef and Puppet, which were entirely focused on launching servers. However, as Terraform has grown in popularity, it’s now very common to use it to launch servers, and these tools can be helpful for validating that the servers you launched are working. [Table 9-3](#) shows some of the tools that do server testing and how they compare in terms of popularity and maturity, based on stats I gathered from GitHub in February 2022.

Table 9-3. A comparison of popular server testing tools

	InSpec	Serverspec	Goss
Brief description	Auditing and testing framework	RSpec tests for your servers	Quick and easy server testing/validation
License	Apache 2.0	MIT	Apache 2.0
Backing company	Chef	(none)	(none)
Stars	2,472	2,426	4,607
Contributors	279	128	89
First release	2016	2013	2015
Latest release	v4.52.9	v2.42.0	v0.3.16
Built-in checks	None	None	None
Custom checks	Defined in a Ruby-based DSL	Defined in a Ruby-based DSL	Defined in YAML

Most of these tools provide a simple *domain-specific language* (DSL) for checking that the servers you’ve deployed conform to some sort of specification. For example, if you were testing a Terraform module that deployed an EC2 Instance, you could use the following inspec code to validate that the Instance has proper permissions on specific files, has certain dependencies installed, and is listening on a specific port:

```
describe file('/etc/myapp.conf') do
  it { should exist }
  its('mode') { should cmp 0644 }
end

describe apache_conf do
  its('Listen') { should cmp 8080 }
end

describe port(8080) do
  it { should be_listening }
end
```

Strengths of server testing tools

- They make it easy to validate specific properties of servers. The DSLs these tools offer are much easier to use for common checks than doing it all from scratch.
- You can build up a library of policy checks. Because each individual check is quick to write, per the previous bullet point, these tools tend to be a good way to validate a checklist of requirements, especially around compliance (e.g., PCI compliance, HIPAA compliance, etc.).
- They can catch many types of errors. Since you actually have to run `apply` and you validate a real, running server, these types of tests catch far more types of errors than pure static analysis or plan testing.

Weaknesses of server testing tools

- They are not as fast. These tests only work on servers that are deployed, so you have to run the full `apply` (and perhaps `destroy`) cycle, which can take a long time.
- They are not as stable (some flaky tests). Since you have to run `apply` and wait for real servers to deploy, you will hit various intermittent issues and occasionally have flaky tests.
- You have to authenticate to a real provider (e.g., to a real AWS account). This is required for the `apply` to work to deploy the servers, plus, these server testing tools all require additional authentication methods—e.g., SSH—to connect to the servers you’re testing.
- You have to deploy/undeploy real resources. This takes time and costs money.
- They only thoroughly check that servers work and not other types of infrastructure.
- These tests aren’t checking functionality, so it’s possible for all the checks to pass and the infrastructure still doesn’t work!

Conclusion

Everything in the infrastructure world is continuously changing: Terraform, Packer, Docker, Kubernetes, AWS, Google Cloud, Azure, and so on are all moving targets. This means that infrastructure code rots very quickly. Or to put it another way:

Infrastructure code without automated tests is broken.

I mean this both as an aphorism and as a literal statement. Every single time I’ve gone to write infrastructure code, no matter how much effort I’ve put into keeping the code clean, testing it manually, and doing code reviews, as soon as I’ve taken the time to write automated tests, I’ve found numerous nontrivial bugs. Something magical happens when you take the time to automate the testing process and, almost without exception, it flushes out problems that you otherwise would’ve never found yourself (but your customers would’ve). And not only do you find these bugs when you first add automated tests, but if you run your tests after every commit, you’ll keep finding bugs over time, especially as the DevOps world changes all around you.

The automated tests I’ve added to my infrastructure code have caught bugs not only in my own code but also in the tools I was using, including nontrivial bugs in Terraform, Packer, Elasticsearch, Kafka, AWS, and so on. Writing automated tests as shown in this chapter is *not* easy: it takes considerable effort to write these tests, it takes even more effort to maintain them and add enough retry logic to make them reliable, and it takes still more effort to keep your test environment clean to keep costs in check. But it’s all worth it.

When I build a module to deploy a data store, for example, after every commit to that repo, my tests fire up a dozen copies of that data store in various configurations, write data, read data, and then tear everything back down. Each time those tests pass, that gives me huge confidence that my code still works. If nothing else, the automated tests let me sleep better. Those hours I spent dealing with retry logic and eventual consistency pay off in the hours I won't be spending at 3 a.m. dealing with an outage.



This Book Has Tests, Too!

All of the code examples in this book have tests, too. You can find all of the code examples, and all of their corresponding tests, at [GitHub](#).

Throughout this chapter, you saw the basic process of testing Terraform code, including the following key takeaways:

When testing Terraform code, you can't use localhost

Therefore, you need to do all of your manual testing by deploying real resources into one or more isolated sandbox environments.

You cannot do pure unit testing for Terraform code

Therefore, you have to do all of your automated testing by writing code that deploys real resources into one or more isolated sandbox environments.

Regularly clean up your sandbox environments

Otherwise, the environments will become unmanageable, and costs will spiral out of control.

You must namespace all of your resources

This ensures that multiple tests running in parallel do not conflict with one another.

Smaller modules are easier and faster to test

This was one of the key takeaways in [Chapter 8](#), and it's worth repeating in this chapter, too: smaller modules are easier to create, maintain, use, and test.

You also saw a number of different testing approaches throughout this chapter: unit testing, integration testing, end-to-end testing, static analysis, and so on. [Table 9-4](#) shows the trade-offs between these different types of tests.

Table 9-4. A comparison of testing approaches (more black squares is better)

	Static analysis	Plan testing	Server testing	Unit tests	Integration tests	End-to-end tests
Fast to run	■■■■■	■■■■□	■■■■□	■□□□	■□□□	□□□□
Cheap to run	■■■■■	■■■■□	■■■■□	■□□□	■□□□	□□□□
Stable and reliable	■■■■■	■■■■□	■■■■□	■□□□	■□□□	□□□□
Easy to use	■■■■■	■■■■□	■■■■□	■□□□	■□□□	□□□□
Check syntax	■■■■■	■■■■■	■■■■■	■■■■■	■■■■■	■■■■■
Check policies	■□□□	■■■■□	■■■■□	■■■■■	■■■■■	■■■■■
Check servers work	□□□□	□□□□	■■■■■	■■■■■	■■■■■	■■■■■
Check other infrastructure works	□□□□	□□□□	■□□□	■■■■□	■■■■■	■■■■■
Check all the infrastructure works together	□□□□	□□□□	□□□□	■□□□	■■■□□	■■■■■

So which testing approach should you use? The answer is: a mix of all of them! Each type of test has strengths and weaknesses, so you have to combine multiple types of tests to be confident your code works as expected. That doesn't mean that you use all the different types of tests in equal proportion: recall the test pyramid and how, in general, you'll typically want lots of unit tests, fewer integration tests, and only a small number of high-value end-to-end tests. Moreover, you don't have to add all the different types of tests at once. Instead, pick the ones that give you the best bang for your buck and add those first. Almost any testing is better than none, so if all you can add for now is static analysis, then use that as a starting point, and build on top of it incrementally.

Let's now move on to [Chapter 10](#), where you'll see how to incorporate Terraform code and your automated test code into your team's workflow, including how to manage environments, how to configure a CI/CD pipeline, and more.

How to Use Terraform as a Team

As you’ve been reading this book and working through the code samples, you’ve most likely been working by yourself. In the real world, you’ll most likely be working as part of a team, which introduces a number of new challenges. You may need to find a way to convince your team to use Terraform and other infrastructure-as-code (IaC) tools. You may need to deal with multiple people concurrently trying to understand, use, and modify the Terraform code you write. And you may need to figure out how to fit Terraform into the rest of your tech stack and make it a part of your company’s workflow.

In this chapter, I’ll dive into the key processes you need to put in place to make Terraform and IaC work for your team:

- Adopting infrastructure as code in your team
- A workflow for deploying application code
- A workflow for deploying infrastructure code
- Putting it all together

Let’s go through these topics one at a time.



Example Code

As a reminder, you can find all of the code examples in the book on [GitHub](#).

Adopting IaC in Your Team

If your team is used to managing all of your infrastructure by hand, switching to infrastructure as code requires more than just introducing a new tool or technology. It also requires changing the culture and processes of the team. Changing culture and process is a significant undertaking, especially at larger companies. Because every team's culture and process is a little different, there's no one-size-fits-all way to do it, but here are a few tips that will be useful in most situations:

- Convince your boss
- Work incrementally
- Give your team the time to learn

Convince Your Boss

I've seen this story play out many times at many companies: a developer discovers Terraform, becomes inspired by what it can do, shows up to work full of enthusiasm and excitement, shows Terraform to everyone... and the boss says "no." The developer, of course, becomes frustrated and discouraged. Why doesn't everyone else see the benefits of this? We could automate everything! We could avoid so many bugs! How else can we pay down all this tech debt? How can you all be so blind??

The problem is that although this developer sees all the benefits of adopting an IaC tool such as Terraform, they aren't seeing all the costs. Here are just a few of the costs of adopting IaC:

Skills gap

The move to IaC means that your Ops team will need to spend most of its time writing large amounts of code: Terraform modules, Go tests, Chef recipes, and so on. Whereas some Ops engineers are comfortable with coding all day and will love the change, others will find this a tough transition. Many Ops engineers and sysadmins are used to making changes manually, with perhaps an occasional short script here or there, and the move to doing software engineering nearly full time might require learning a number of new skills or hiring new people.

New tools

Software developers can become attached to the tools they use; some are nearly religious about it. Every time you introduce a new tool, some developers will be thrilled at the opportunity to learn something new, but others will prefer to stick to what they know and may resist having to invest lots of time and energy learning new languages and techniques.

Change in mindset

If your team members are used to managing infrastructure manually, they are used to making all of their changes *directly*: for example, by SSHing to a server and executing a few commands. The move to IaC requires a shift in mindset where you make all of your changes *indirectly*, first by editing code, then checking it in, and then letting some automated process apply the changes. This layer of indirection can be frustrating; for simple tasks, it'll feel slower than the direct option, especially when you're still learning a new IaC tool and are not efficient with it.

Opportunity cost

If you choose to invest your time and resources in one project, you are implicitly choosing not to invest that time and resources in other projects. What projects will have to be put on hold so that you can migrate to IaC? How important are those projects?

Some developers on your team will look at this list and become excited. But many others will groan—including your boss. Learning new skills, mastering new tools, and adopting new mindsets may or may not be beneficial, but one thing is certain: it is not free. Adopting IaC is a significant investment, and as with any investment, you need to consider not only the potential upside but also the potential downsides.

Your boss in particular will be sensitive to the opportunity cost. One of the key responsibilities of any manager is to make sure the team is working on the highest-priority projects. When you show up and excitedly start talking about Terraform, what your boss might really be hearing is, “Oh no, this sounds like a massive undertaking. How much time is it going to take?” It's not that your boss is blind to what Terraform can do, but if you are spending time on that, you might not have time to deploy the new app the search team has been asking about for months, or to prepare for the Payment Card Industry (PCI) audit, or to dig into the outage from last week. So, if you want to convince your boss that your team should adopt IaC, your goal is not to prove that it has value but that it will bring more value to your team than anything else you could work on during that time.

One of the least effective ways to do this is to just list the features of your favorite IaC tool: for example, Terraform is declarative, it's popular, it's open source. This is one of many areas where developers would do well to learn from salespeople. Most salespeople know that focusing on features is typically an ineffective way to sell products. A slightly better technique is to focus on benefits: that is, instead of talking about what a product can do (“product X can do Y!”), you should talk about what the customer can do by using that product (“you can do Y by using product X!”). In other words, show the customer what new superpowers your product can give them.

For example, instead of telling your boss that Terraform is declarative, talk about how your infrastructure will be far easier to maintain. Instead of talking about the fact that Terraform is popular, talk about how you'll be able to leverage lots of existing modules and plugins to get things done faster. And instead of explaining to your boss that Terraform is open source, help your boss see how much easier it will be to hire new developers for the team from a large, active open source community.

Focusing on benefits is a great start, but the best salespeople know an even more effective strategy: focus on the problems. If you watch a great salesperson talking to a customer, you'll notice that it's actually the customer that does most of the talking. The salesperson spends most of their time listening and looking for one specific thing: What is the key problem that customer is trying to solve? What's the biggest pain point? Instead of trying to sell some sort of features or benefits, the best salespeople try to solve their customer's problems. If that solution happens to include the product they are selling, all the better, but the real focus is on problem solving, not selling.

Talk to your boss and try to understand the most important problems they are working on that quarter or that year. You might find that those problems would not be solved by IaC. And that's OK! It might be slightly heretical for the author of a book on Terraform to say this, but not every team needs IaC. Adopting IaC has a relatively high cost, and although it will pay off in the long term for some scenarios, it won't for others; for example, if you're at a tiny startup with just one Ops person, or you're working on a prototype that might be thrown away in a few months, or you're just working on a side project for fun, managing infrastructure by hand is often the right choice. Sometimes, even if IaC would be a great fit for your team, it won't be the highest priority, and given limited resources, working on other projects might still be the right choice.

If you do find that one of the key problems your boss is focused on can be solved with IaC, then your goal is to show your boss what that world looks like. For example, perhaps the biggest issue your boss is focused on this quarter is improving uptime. You've had numerous outages the last few months, many hours of downtime, customers are complaining, and the CEO is breathing down your manager's neck, checking in daily to see how things are going. You dig in and find out that more than half of these outages were caused by a manual error during deployment: e.g., someone accidentally skipped an important step during the rollout process, or a server was misconfigured, or the infrastructure in staging didn't match what you had in production.

Now, when you talk to your boss, instead of talking about Terraform features or benefits, lead with the following: "I have an idea for how to reduce our outages by half." I guarantee this will get your boss's attention. Use this opportunity to paint a picture for your boss of a world in which your deployment process is fully automated,

reliable, and repeatable so that the manual errors that caused half of your previous outages are no longer possible. Not only that, but if deployment is automated, you can also add automated tests, reducing outages further and allowing the whole company to deploy twice as often. Let your boss dream of being the one to tell the CEO that they've managed to cut outages in half and double deployments. And then mention that, based on your research, you believe you can deliver this future world using Terraform.

There's no guarantee that your boss will say yes, but your odds are quite a bit higher with this approach. And your odds get even better if you work incrementally.

Work Incrementally

One of the most important lessons I've learned in my career is that most large software projects fail. Whereas roughly 3 out of 4 small IT projects (less than \$1 million) are completed successfully, only 1 out of 10 large projects (greater than \$10 million) are completed on time and on budget, and more than one-third of large projects are never completed at all.¹

This is why I always get worried when I see a team try to not only adopt IaC but to do so all at once, across a huge amount of infrastructure, across every team, and often as part of an even bigger initiative. I can't help but shake my head when I see the CEO or CTO of a large company give marching orders that everything must be migrated to the cloud, the old datacenters must be shut down, and that everyone will "do DevOps" (whatever that means), all within six months. I'm not exaggerating when I say that I've seen this pattern several dozen times, and without exception, every single one of these initiatives has failed. Inevitably, two to three years later, every one of these companies is still working on the migration, the old datacenter is still running, and no one can tell whether they are really doing DevOps.

If you want to successfully adopt IaC, or if you want to succeed at any other type of migration project, the only sane way to do it is incrementally. The key to *incrementalism* is not just splitting up the work into a series of small steps but splitting up the work in such a way that every step brings its own value—even if the later steps never happen.

To understand why this is so important, consider the opposite, *false incrementalism*.² Suppose that you do a huge migration project, broken up into several small steps, but the project doesn't offer any real value until the very final step is completed. For example, the first step is to rewrite the frontend, but you don't launch it, because it

1 The Standish Group, "CHAOS Manifesto 2013: Think Big, Act Small," 2013, <https://oreil.ly/ydaWQ>.

2 Dan Milstein, "How to Survive a Ground-Up Rewrite Without Losing Your Sanity," OnStartups.com, April 8, 2013, <https://oreil.ly/nOGrU>.

relies on a new backend. Then, you rewrite the backend, but you don't launch that either, because it doesn't work until data is migrated to a new data store. And then, finally, the last step is to do the data migration. Only after this last step do you finally launch everything and begin realizing any value from doing all this work. Waiting until the very end of a project to get any value is a big risk. If that project is canceled or put on hold or significantly changed partway through, you might get zero value out of it, despite a lot of investment.

In fact, this is exactly what happens with many large migration projects. The project is big to begin with, and like most software projects, it takes much longer than expected. During that time, market conditions change, or the original stakeholders lose patience (e.g., the CEO was OK with spending three months to clean up tech debt, but after 12 months, it's time to begin shipping new products), and the project ends up getting canceled before completion. With false incrementalism, this gives you the worst possible outcome: you've paid a huge cost and received absolutely nothing in return.

Therefore, incrementalism is essential. You want each part of the project to deliver some value so that even if the project doesn't finish, no matter what step you got to, it was still worth doing. The best way to accomplish this is to focus on solving one, small, concrete problem at a time. For example, instead of trying to do a "big bang" migration to the cloud, try to identify one, small, specific app or team that is struggling, and work to migrate just them. Or instead of trying to do a "big bang" move to "DevOps," try to identify a single, small, concrete problem (e.g., outages during deployment) and put in place a solution for that specific problem (e.g., automate the most problematic deployment with Terraform).

If you can get a quick win by fixing one real, concrete problem right away, and making one team successful, you'll begin to build momentum. That team can become your cheerleader and help convince other teams to migrate, too. Fixing the specific deployment issue can make the CEO happy and get you support to use IaC for more projects. This will allow you to go for another quick win, and another one after that. And if you can keep repeating this process—delivering value early and often—you'll be far more likely to succeed at the larger migration effort. But even if the larger migration doesn't work out, at least one team is more successful now and one deployment process works better, so it was still worth the investment.

Give Your Team the Time to Learn

I hope that, at this point, it's clear that adopting IaC can be a significant investment. It's not something that will happen overnight. It's not something that will happen magically, just because the manager gives you a nod. It will happen only through a deliberate effort of getting everyone on board, making learning resources (e.g.,

documentation, video tutorials, and, of course, this book!) available, and providing dedicated time for team members to ramp up.

If your team doesn't get the time and resources that it needs, then your IaC migration is unlikely to be successful. No matter how nice your code is, if your entire team isn't on board with it, here's how it will play out:

1. One developer on the team is passionate about IaC and spends a few months writing beautiful Terraform code and using it to deploy lots of infrastructure.
2. The developer is happy and productive, but unfortunately, the rest of the team did not get the time to learn and adopt Terraform.
3. Then, the inevitable happens: an outage. One of your team members needs to deal with it, and they have two options: either (A) fix the outage the way they've always done it, by making changes manually, which takes a few minutes, or (B) fix the outage by using Terraform, but they aren't familiar with it, so this could take hours or days. Your team members are probably reasonable, rational people and will almost always choose option A.
4. Now, as a result of the manual change, the Terraform code no longer matches what's actually deployed. Therefore, next time someone on your team tries to use Terraform, there's a chance that they will get a weird error. If they do, they will lose trust in the Terraform code and once again fall back to option A, making more manual changes. This makes the code even more out of sync with reality, so the odds of the next person getting a weird Terraform error are even higher, and you quickly get into a cycle in which team members make more and more manual changes.
5. In a remarkably short time, everyone is back to doing everything manually, the Terraform code is completely unusable, and the months spent writing it are a total waste.

This scenario isn't hypothetical but something I've seen happen at many different companies. They have large, expensive codebases full of beautiful Terraform code that are just gathering dust. To avoid this scenario, you need to not only convince your boss that you should use Terraform but also give everyone on the team the time they need to learn the tool and internalize how to use it so that when the next outage happens, it's easier to fix it in code than it is to do it by hand.

One thing that can help teams adopt IaC faster is to have a well-defined process for using it. When you're learning or using IaC on a small team, running it ad hoc on a developer's computer is good enough. But as your company and IaC usage grows, you'll want to define a more systematic, repeatable, automated workflow for how deployments happen.

A Workflow for Deploying Application Code

In this section, I'll introduce a typical workflow for taking application code (e.g., a Ruby on Rails or Java/Spring app) from development all the way to production. This workflow is reasonably well understood in the DevOps industry, so you'll probably be familiar with parts of it. Later in this chapter, I'll talk about a workflow for taking infrastructure code (e.g., Terraform modules) from development to production. This workflow is not nearly as well known in the industry, so it will be helpful to compare that workflow side by side with the application workflow to understand how to translate each application code step to an analogous infrastructure code step.

Here's what the application code workflow looks like:

1. Use version control.
2. Run the code locally.
3. Make code changes.
4. Submit changes for review.
5. Run automated tests.
6. Merge and release.
7. Deploy.

Let's go through these steps one at a time.

Use Version Control

All of your code should be in version control. No exceptions. It was the #1 item on the classic **Joel Test** when Joel Spolsky created it more than 20 years ago, and the only things that have changed since then are that (a) with tools like GitHub, it's easier than ever to use version control and (b) you can represent more and more things as code. This includes documentation (e.g., a README written in Markdown), application configuration (e.g., a config file written in YAML), specifications (e.g., test code written with RSpec), tests (e.g., automated tests written with JUnit), databases (e.g., schema migrations written in ActiveRecord), and of course, infrastructure.

As in the rest of this book, I'm going to assume that you're using Git for version control. For example, here is how you can check out the code repo for this book:

```
$ git clone https://github.com/brikis98/terraform-up-and-running-code.git
```

By default, this checks out the `main` branch of your repo, but you'll most likely do all of your work in a separate branch. Here's how you can create a branch called `example-feature` and switch to it by using the `git checkout` command:

```
$ cd terraform-up-and-running-code
$ git checkout -b example-feature
Switched to a new branch 'example-feature'
```

Run the Code Locally

Now that the code is on your computer, you can run it locally. You may recall the Ruby web server example from [Chapter 9](#), which you can run as follows:

```
$ cd code/ruby/10-terraform/team
$ ruby web-server.rb

[2019-06-15 15:43:17] INFO WEBrick 1.3.1
[2019-06-15 15:43:17] INFO ruby 2.3.7 (2018-03-28) [universal.x86_64-darwin17]
[2019-06-15 15:43:17] INFO WEBrick::HTTPServer#start: pid=28618 port=8000
```

Now you can manually test it with `curl`:

```
$ curl http://localhost:8000
Hello, World
```

Alternatively, you can run the automated tests:

```
$ ruby web-server-test.rb

(...)

Finished in 0.633175 seconds.
-----
8 tests, 24 assertions, 0 failures, 0 errors
100% passed
-----
```

The key thing to notice is that both manual and automated tests for application code can run completely locally on your own computer. You'll see later in this chapter that this is not true for the same part of the workflow for infrastructure changes.

Make Code Changes

Now that you can run the application code, you can begin making changes. This is an iterative process in which you make a change, rerun your manual or automated tests to see whether the change worked, make another change, rerun the tests, and so on.

For example, you can change the output of *web-server.rb* to “Hello, World v2,” restart the server, and see the result:

```
$ curl http://localhost:8000
Hello, World v2
```

You might also update and rerun the automated tests. The idea in this part of the workflow is to optimize the feedback loop so that the time between making a change and seeing whether it worked is minimized.

As you work, you should regularly be committing your code, with clear commit messages explaining the changes you've made:

```
$ git commit -m "Updated Hello, World text"
```

Submit Changes for Review

Eventually, the code and tests will work the way you want them to, so it's time to submit your changes for a code review. You can do this with a separate code review tool (e.g., Phabricator or Review Board) or, if you're using GitHub, you can create a *pull request*. There are several different ways to create a pull request. One of the easiest is to `git push` your `example-feature` branch back to `origin` (that is, back to GitHub itself), and GitHub will automatically print out a pull request URL in the log output:

```
$ git push origin example-feature
```

```
(...)
```

```
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
```

```
remote:
```

```
remote: Create a pull request for 'example-feature' on GitHub by visiting:
```

```
remote:      https://github.com/<OWNER>/<REPO>/pull/new/example-feature
```

```
remote:
```

Open that URL in your browser, fill out the pull request title and description, and then click Create. Your team members will now be able to review the changes, as shown in [Figure 10-1](#).

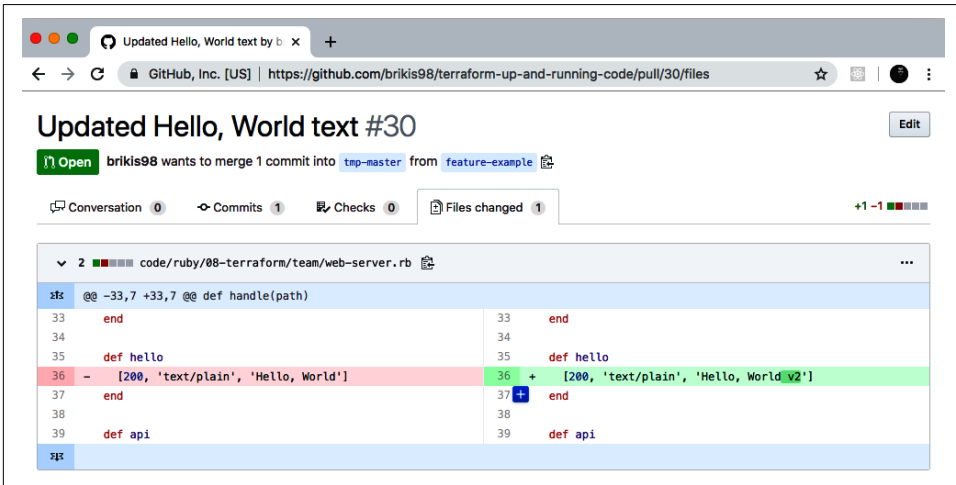


Figure 10-1. Your team members can review your code changes in a GitHub pull request.

Run Automated Tests

You should set up commit hooks to run automated tests for every commit you push to your version control system. The most common way to do this is to use a *continuous integration* (CI) server, such as Jenkins, CircleCI, or GitHub Actions. Most popular CI servers have integrations built in specifically for GitHub, so not only does every commit automatically run tests, but the output of those tests shows up in the pull request itself, as shown in [Figure 10-2](#).

You can see in [Figure 10-2](#) that CircleCI has run unit tests, integration tests, end-to-end tests, and some static analysis checks (in the form of security vulnerability scanning using a tool called snyk) against the code in the branch, and everything passed.

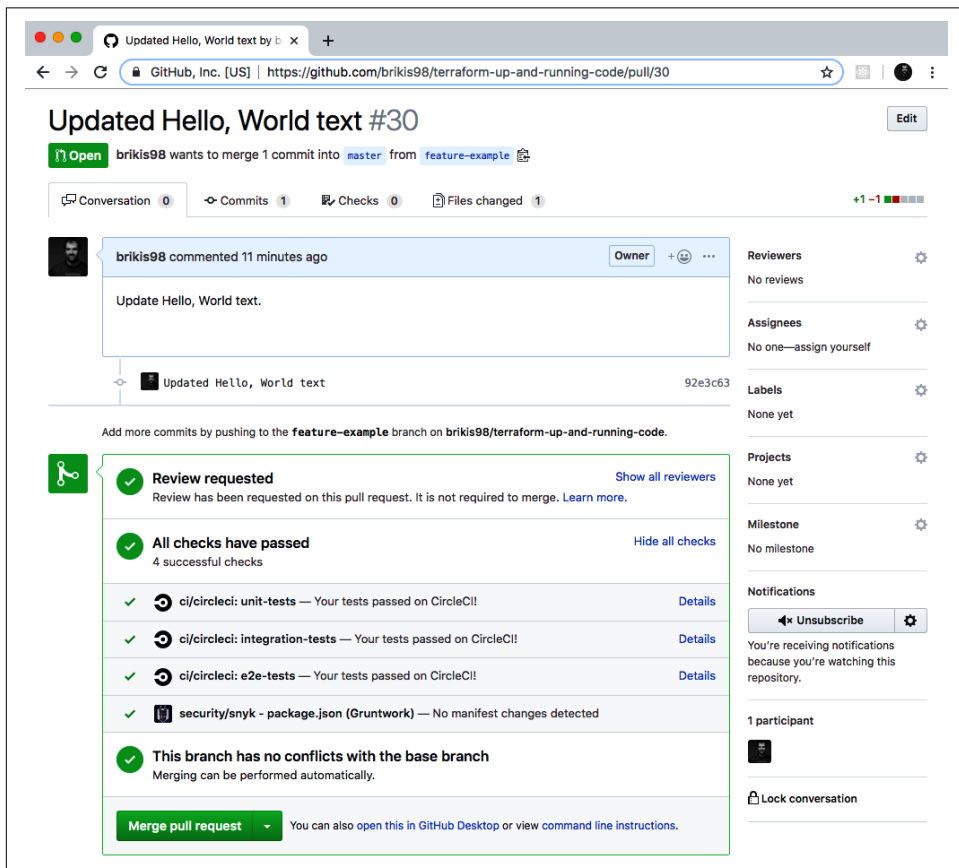


Figure 10-2. GitHub pull request showing automated test results from CircleCI.

Merge and Release

Your team members should review your code changes, looking for potential bugs, enforcing coding guidelines (more on this later in the chapter), checking that the existing tests passed, and ensuring that you’ve added tests for any new behavior. If everything looks good, your code can be merged into the `main` branch.

The next step is to release the code. If you’re using immutable infrastructure practices (as discussed in [“Server Templating Tools” on page 7](#)), releasing application code means packaging that code into a new, immutable, versioned artifact. Depending on how you want to package and deploy your application, the artifact can be a new Docker image, a new virtual machine image (e.g., new AMI), a new `.jar` file, a new `.tar` file, etc. Whatever format you pick, make sure the artifact is immutable (i.e., you never change it) and that it has a unique version number (so you can distinguish this artifact from all of the others).

For example, if you are packaging your application using Docker, you can store the version number in a Docker tag. You could use the ID of the commit (the sha1 hash) as the tag so that you can map the Docker image you’re deploying back to the exact code it contains:

```
$ commit_id=$(git rev-parse HEAD)
$ docker build -t brikis98/ruby-web-server:$commit_id .
```

The preceding code will build a new Docker image called `brikis98/ruby-web-server` and tag it with the ID of the most recent commit, which will look something like `92e3c6380ba6d1e8c9134452ab6e26154e6ad849`. Later on, if you’re debugging an issue in a Docker image, you can see the exact code it contains by checking out the commit ID the Docker image has as a tag:

```
$ git checkout 92e3c6380ba6d1e8c9134452ab6e26154e6ad849
HEAD is now at 92e3c63 Updated Hello, World text
```

One downside to commit IDs is that they aren’t very readable or memorable. An alternative is to create a Git tag:

```
$ git tag -a "v0.0.4" -m "Update Hello, World text"
$ git push --follow-tags
```

A tag is a pointer to a specific Git commit but with a friendlier name. You can use this Git tag on your Docker images:

```
$ git_tag=$(git describe --tags)
$ docker build -t brikis98/ruby-web-server:$git_tag .
```

Thus, when you’re debugging, check out the code at a specific tag:

```
$ git checkout v0.0.4
Note: checking out 'v0.0.4'.
```

```
(...)  
HEAD is now at 92e3c63 Updated Hello, World text
```

Deploy

Now that you have a versioned artifact, it's time to deploy it. There are many different ways to deploy application code, depending on the type of application, how you package it, how you want to run it, your architecture, what tools you're using, and so on. Here are a few of the key considerations:

- Deployment tooling
- Deployment strategies
- Deployment server
- Promotion across environments

Deployment tooling

There are many different tools that you can use to deploy your application, depending on how you package it and how you want to run it. Here are a few examples:

Terraform

As you've seen in this book, you can use Terraform to deploy certain types of applications. For example, in earlier chapters, you created a module called `asg-rolling-deploy` that could do a zero-downtime rolling deployment across an ASG. If you package your application as an AMI (e.g., using Packer), you could deploy new AMI versions with the `asg-rolling-deploy` module by updating the `ami` parameter in your Terraform code and running `terraform apply`.

Orchestration tools

There are a number of orchestration tools designed to deploy and manage applications, such as Kubernetes (arguably the most popular Docker orchestration tool), Amazon ECS, HashiCorp Nomad, and Apache Mesos. In [Chapter 7](#), you saw an example of how to use Kubernetes to deploy Docker containers.

Scripts

Terraform and most orchestration tools support only a limited set of deployment strategies (discussed in the next section). If you have more complicated requirements, you may have to write custom scripts to implement these requirements.

Deployment strategies

There are a number of different strategies that you can use for application deployment, depending on your requirements. Suppose that you have five copies of the old

version of your app running, and you want to roll out a new version. Here are a few of the most common strategies you can use:

Rolling deployment with replacement

Take down one of the old copies of the app, deploy a new copy to replace it, wait for the new copy to come up and pass health checks, start sending the new copy live traffic, and then repeat the process until all of the old copies have been replaced. Rolling deployment with replacement ensures that you never have more than five copies of the app running, which can be useful if you have limited capacity (e.g., if each copy of the app runs on a physical server) or if you're dealing with a stateful system where each app has a unique identity (e.g., this is often the case with consensus systems, such as Apache ZooKeeper). Note that this deployment strategy can work with larger batch sizes (you can replace more than one copy of the app at a time if you can handle the load and won't lose data with fewer apps running) and that during deployment, you will have both the old and new versions of the app running at the same time.

Rolling deployment without replacement

Deploy one new copy of the app, wait for the new copy to come up and pass health checks, start sending the new copy live traffic, undeploy an old copy of the app, and then repeat the process until all the old copies have been replaced. Rolling deployment without replacement works only if you have flexible capacity (e.g., your apps run in the cloud, where you can spin up new virtual servers any time you want) and if your application can tolerate more than five copies of it running at the same time. The advantage is that you never have less than five copies of the app running, so you're not running at a reduced capacity during deployment. Note that this deployment strategy can also work with larger batch sizes (if you have the capacity for it, you can deploy five new copies all at once) and that during deployment, you will have both the old and new versions of the app running at the same time.

Blue-green deployment

Deploy five new copies of the app, wait for all of them to come up and pass health checks, shift all live traffic to the new copies at the same time, and then undeploy the old copies. Blue-green deployment works only if you have flexible capacity (e.g., your apps run in the cloud, where you can spin up new virtual servers any time you want) and if your application can tolerate more than five copies of it running at the same time. The advantage is that only one version of your app is visible to users at any given time and that you never have less than five copies of the app running, so you're not running at a reduced capacity during deployment.

Canary deployment

Deploy one new copy of the app, wait for it to come up and pass health checks, start sending live traffic to it, and then pause the deployment. During the pause, compare the new copy of the app, called the “canary,” to one of the old copies, called the “control.” You can compare the canary and control across a variety of dimensions: CPU usage, memory usage, latency, throughput, error rates in the logs, HTTP response codes, and so on. Ideally, there’s no way to tell the two servers apart, which should give you confidence that the new code works just fine. In that case, you unpause the deployment and use one of the rolling deployment strategies to complete it. On the other hand, if you spot any differences, then that may be a sign of problems in the new code, and you can cancel the deployment and undeploy the canary before the problem becomes worse.

The name comes from the “canary in a coal mine” concept, where miners would take canary birds with them down into the tunnels, and if the tunnels filled with dangerous gases (e.g., carbon monoxide), those gases would affect the canary before the miners, thus providing an early warning to the miners that something was wrong and that they needed to exit immediately, before more damage was done. The canary deployment offers similar benefits, giving you a systematic way to test new code in production in a way that, if something goes wrong, you get a warning early on, when it has affected only a small portion of your users and you still have enough time to react and prevent further damage.

Canary deployments are often combined with *feature toggles*, in which you wrap all new features in an if-statement. By default, the if-statement defaults to false, so the new feature is toggled off when you initially deploy the code. Because all new functionality is off, when you deploy the canary server, it should behave identically to the control, and any differences can be automatically flagged as a problem and trigger a rollback. If there were no problems, later on you can enable the feature toggle for a portion of your users via an internal web interface. For example, you might initially enable the new feature only for employees; if that works well, you can enable it for 1% of users; if that’s still working well, you can ramp it up to 10%; and so on. If at any point there’s a problem, you can use the feature toggle to ramp the feature back down. This process allows you to separate *deployment* of new code from *release* of new features.

Deployment server

You should run the deployment from a CI server and not from a developer’s computer. This has the following benefits:

Fully automated

To run deployments from a CI server, you’ll be forced to fully automate all deployment steps. This ensures that your deployment process is captured as

code, that you don't miss any steps accidentally due to manual error, and that the deployment is fast and repeatable.

Consistent environment

If developers run deployments from their own computers, you'll run into bugs due to differences in how their computer is configured: for example, different operating systems, different dependency versions (different versions of Terraform), different configurations, and differences in what's actually being deployed (e.g., the developer accidentally deploys a change that wasn't committed to version control). You can eliminate all of these issues by deploying everything from the same CI server.

Better permissions management

Instead of giving every developer permissions to deploy, you can give solely the CI server those permissions (especially for the production environment). It's a lot easier to enforce good security practices for a single server than it is to do for numerous developers with production access.

Promotion across environments

If you're using immutable infrastructure practices, the way to roll out new changes is to promote the exact same versioned artifact from one environment to another. For example, if you have dev, staging, and production environments, to roll out `v0.0.4` of your app, you would do the following:

1. Deploy `v0.0.4` of the app to dev.
2. Run your manual and automated tests in dev.
3. If `v0.0.4` works well in dev, repeat steps 1 and 2 to deploy `v0.0.4` to staging (this is known as *promoting* the artifact).
4. If `v0.0.4` works well in staging, repeat steps 1 and 2 again to promote `v0.0.4` to prod.

Because you're running the exact same artifact everywhere, there's a good chance that if it works in one environment, it will work in another. And if you do hit any issues, you can roll back anytime by deploying an older artifact version.

A Workflow for Deploying Infrastructure Code

Now that you've seen the workflow for deploying application code, it's time to dive into the workflow for deploying infrastructure code. In this section, when I say "infrastructure code," I mean code written with any IaC tool (including, of course, Terraform) that you can use to deploy arbitrary infrastructure changes beyond a

single application: for example, deploying databases, load balancers, network configurations, DNS settings, and so on.

Here's what the infrastructure code workflow looks like:

1. Use version control
2. Run the code locally
3. Make code changes
4. Submit changes for review
5. Run automated tests
6. Merge and release
7. Deploy

On the surface, it looks identical to the application workflow, but under the hood, there are important differences. Deploying infrastructure code changes is more complicated, and the techniques are not as well understood, so being able to relate each step back to the analogous step from the application code workflow should make it easier to follow along. Let's dive in.

Use Version Control

Just as with your application code, all of your infrastructure code should be in version control. This means that you'll use `git clone` to check out your code, just as before. However, version control for infrastructure code has a few extra requirements:

- *Live* repo and *modules* repo
- Golden Rule of Terraform
- The trouble with branches

Live repo and modules repo

As discussed in [Chapter 4](#), you will typically want at least two separate version control repositories for your Terraform code: one repo for modules and one repo for live infrastructure. The repository for modules is where you create your reusable, versioned modules, such as all the modules you built in the previous chapters of this book (`cluster/asg-rolling-deploy`, `data-stores/mysql`, `networking/alb`, and `services/hello-world-app`). The repository for live infrastructure defines the live infrastructure you've deployed in each environment (`dev`, `stage`, `prod`, etc.).

One pattern that works well is to have one infrastructure team in your company that specializes in creating reusable, robust, production-grade modules. This team

can create remarkable leverage for your company by building a library of modules that implement the ideas from [Chapter 8](#); that is, each module has a composable API, is thoroughly documented (including executable documentation in the *examples* folder), has a comprehensive suite of automated tests, is versioned, and implements all of your company’s requirements from the production-grade infrastructure checklist (i.e., security, compliance, scalability, high availability, monitoring, and so on).

If you build such a library (or you buy one off the shelf³), all the other teams at your company will be able to consume these modules, a bit like a service catalog, to deploy and manage their own infrastructure, without (a) each team having to spend months assembling that infrastructure from scratch or (b) the Ops team becoming a bottleneck because it must deploy and manage the infrastructure for every team. Instead, the Ops team can spend most of its time writing infrastructure code, and all of the other teams will be able to work independently, using these modules to get themselves up and running. And because every team is using the same canonical modules under the hood, as the company grows and requirements change, the Ops team can push out new versions of the modules to all teams, ensuring everything stays consistent and maintainable.

Or it will be maintainable, as long as you follow the Golden Rule of Terraform.

The Golden Rule of Terraform

Here’s a quick way to check the health of your Terraform code: go into your *live* repository, pick several folders at random, and run `terraform plan` in each one. If the output is always “no changes,” that’s great, because it means that your infrastructure code matches what’s actually deployed. If the output sometimes shows a small diff, and you hear the occasional excuse from your team members (“Oh, right, I tweaked that one thing by hand and forgot to update the code”), your code doesn’t match reality, and you might soon be in trouble. If `terraform plan` fails completely with weird errors, or every `plan` shows a gigantic diff, your Terraform code has no relation at all to reality and is likely useless.

The gold standard, or what you’re really aiming for, is what I call *The Golden Rule of Terraform*:

The main branch of the *live* repository should be a 1:1 representation of what’s actually deployed in production.

Let’s break this sentence down, starting at the end and working our way back:

³ See the [Gruntwork Infrastructure as Code Library](#).

“...what’s actually deployed”

The only way to ensure that the Terraform code in the *live* repository is an up-to-date representation of what’s actually deployed is to *never make out-of-band changes*. After you begin using Terraform, do not make changes via a web UI, or manual API calls, or any other mechanism. As you saw in [Chapter 5](#), out-of-band changes not only lead to complicated bugs, but they also void many of the benefits you get from using IaC in the first place.

“...a 1:1 representation...”

If I browse your *live* repository, I should be able to see, from a quick scan, what resources have been deployed in what environments. That is, every resource should have a 1:1 match with some line of code checked into the *live* repo. This seems obvious at first glance, but it’s surprisingly easy to get it wrong. One way to get it wrong, as I just mentioned, is to make out-of-band changes so that the code is there, but the live infrastructure is different. A more subtle way to get it wrong is to use Terraform workspaces to manage environments so that the live infrastructure is there, but the code isn’t. That is, if you use workspaces, your *live* repo will have only one copy of the code, even though you may have 3 or 30 environments deployed with it. From merely looking at the code, there will be no way to know what’s actually deployed, which will lead to mistakes and make maintenance complicated. Therefore, as described in [“Isolation via Workspaces” on page 94](#), instead of using workspaces to manage environments, you want each environment defined in a separate folder, using separate files, so that you can see exactly what environments have been deployed just by browsing the *live* repository. Later in this chapter, you’ll see how to do this with minimal copying and pasting.

“The main branch...”

You should have to look at only a single branch to understand what’s actually deployed in production. Typically, that branch will be `main`. This means that all changes that affect the production environment should go directly into `main` (you can create a separate branch but only to create a pull request with the intention of merging that branch into `main`), and you should run `terraform apply` only for the production environment against the `main` branch. In the next section, I’ll explain why.

The trouble with branches

In [Chapter 3](#), you saw that you can use the locking mechanisms built into Terraform backends to ensure that if two team members are running `terraform apply` at the same time on the same set of Terraform configurations, their changes do not overwrite each other. Unfortunately, this only solves part of the problem. Even though Terraform backends provide locking for Terraform state, they cannot help you with locking at the level of the Terraform code itself. In particular, if two team members

are deploying the same code to the same environment but from different branches, you'll run into conflicts that locking can't prevent.

For example, suppose that one of your team members, Anna, makes some changes to the Terraform configurations for an app called “foo” that consists of a single EC2 Instance:

```
resource "aws_instance" "foo" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t2.micro"  
}
```

The app is getting a lot of traffic, so Anna decides to change the `instance_type` from `t2.micro` to `t2.medium`:

```
resource "aws_instance" "foo" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t2.medium"  
}
```

Here's what Anna sees when she runs `terraform plan`:

```
$ terraform plan
```

```
(...)
```

Terraform will perform the following actions:

```
# aws_instance.foo will be updated in-place  
~ resource "aws_instance" "foo" {  
  ami           = "ami-0fb653ca2d3203ac1"  
  id            = "i-096430d595c80cb53"  
  instance_state = "running"  
  ~ instance_type = "t2.micro" -> "t2.medium"  
  (...)           
}
```

```
Plan: 0 to add, 1 to change, 0 to destroy.
```

Those changes look good, so she deploys them to staging.

In the meantime, Bill comes along and also starts making changes to the Terraform configurations for the same app but on a different branch. All Bill wants to do is to add a tag to the app:

```
resource "aws_instance" "foo" {  
  ami           = data.aws_ami.ubuntu.id  
  instance_type = "t2.micro"  
  
  tags = {  
    Name = "foo"  
  }  
}
```

Note that Anna's changes are already deployed in staging, but because they are on a different branch, Bill's code still has the `instance_type` set to the old value of `t2.micro`. Here's what Bill sees when he runs the `plan` command (the following log output is truncated for readability):

```
$ terraform plan

(...)

Terraform will perform the following actions:

# aws_instance.foo will be updated in-place
~ resource "aws_instance" "foo" {
    ami                = "ami-0fb653ca2d3203ac1"
    id                 = "i-096430d595c80cb53"
    instance_state     = "running"
    ~ instance_type    = "t2.medium" -> "t2.micro"
    + tags              = {
        + "Name" = "foo"
    }
    (...)
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

Uh oh, he's about to undo Anna's `instance_type` change! If Anna is still testing in staging, she'll be very confused when the server suddenly redeploys and starts behaving differently. The good news is that if Bill diligently reads the `plan` output, he can spot the error before it affects Anna. Nevertheless, the point of the example is to highlight what happens when you deploy changes to a shared environment from different branches.

The locking from Terraform backends doesn't help here, because the conflict has nothing to do with concurrent modifications to the state file; Bill and Anna might be applying their changes weeks apart, and the problem would be the same. The underlying cause is that branching and Terraform are a bad combination. Terraform is implicitly a mapping from Terraform code to infrastructure deployed in the real world. Because there's only one real world, it doesn't make much sense to have multiple branches of your Terraform code. So for any shared environment (e.g., stage, prod), always deploy from a single branch.

Run the Code Locally

Now that you've got the code checked out onto your computer, the next step is to run it. The gotcha with Terraform is that, unlike application code, you don't have "localhost"; for example, you can't deploy an AWS ASG onto your own laptop. As discussed in ["Manual Testing Basics" on page 317](#), the only way to manually test

Terraform code is to run it in a sandbox environment, such as an AWS account dedicated for developers (or better yet, one AWS account for each developer).

Once you have a sandbox environment, to test manually, you run `terraform apply`:

```
$ terraform apply

(...)

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

alb_dns_name = "hello-world-stage-477699288.us-east-2.elb.amazonaws.com"
```

And you verify the deployed infrastructure works by using tools such as `curl`:

```
$ curl hello-world-stage-477699288.us-east-2.elb.amazonaws.com
Hello, World
```

To run automated tests written in Go, you use `go test` in a sandbox account dedicated to testing:

```
$ go test -v -timeout 30m

(...)

PASS
ok      terraform-up-and-running      229.492s
```

Make Code Changes

Now that you can run your Terraform code, you can iteratively begin to make changes, just as with application code. Every time you make a change, you can rerun `terraform apply` to deploy those changes and rerun `curl` to see whether those changes worked:

```
$ curl hello-world-stage-477699288.us-east-2.elb.amazonaws.com
Hello, World v2
```

Or you can rerun `go test` to make sure the tests are still passing:

```
$ go test -v -timeout 30m

(...)

PASS
ok      terraform-up-and-running      229.492s
```

The only difference from application code is that infrastructure code tests typically take longer, so you'll want to put more thought into how you can shorten the test cycle so that you can get feedback on your changes as quickly as possible. In “**Test**

stages” on page 354, you saw that you can use these test stages to rerun only specific stages of a test suite, dramatically shortening the feedback loop.

As you make changes, be sure to regularly commit your work:

```
$ git commit -m "Updated Hello, World text"
```

Submit Changes for Review

After your code is working the way you expect, you can create a pull request to get your code reviewed, just as you would with application code. Your team will review your code changes, looking for bugs as well as enforcing *coding guidelines*. Whenever you’re writing code as a team, regardless of what type of code you’re writing, you should define guidelines for everyone to follow. One of my favorite definitions of “clean code” comes from an interview I did with Nick Dellamaggiore for my earlier book, *Hello, Startup*:

If I look at a single file and it’s written by 10 different engineers, it should be almost indistinguishable which part was written by which person. To me, that is clean code.

The way you do that is through code reviews and publishing your style guide, your patterns, and your language idioms. Once you learn them, everybody is way more productive because you all know how to write code the same way. At that point, it’s more about what you’re writing and not how you write it.

—Nick Dellamaggiore, Infrastructure Lead at Coursera

The Terraform coding guidelines that make sense for each team will be different, so here, I’ll list a few of the common ones that are useful for most teams:

- Documentation
- Automated tests
- File layout
- Style guide

Documentation

In some sense, Terraform code is, in and of itself, a form of documentation. It describes in a simple language exactly what infrastructure you deployed and how that infrastructure is configured. However, there is no such thing as self-documenting code. Although well-written code can tell you *what* it does, no programming language that I’m aware of (including Terraform) can tell you *why* it does it.

This is why all software, including IaC, needs documentation beyond the code itself. There are several types of documentation that you can consider and have your team members require as part of code reviews:

Written documentation

Most Terraform modules should have a README that explains what the module does, why it exists, how to use it, and how to modify it. In fact, you may want to write the README first, before any of the actual Terraform code, because that will force you to consider *what* you’re building and *why* you’re building it before you dive into the code and get lost in the details of *how* to build it.⁴ Spending 20 minutes writing a README can often save you hours of writing code that solves the wrong problem. Beyond the basic README, you might also want to have tutorials, API documentation, wiki pages, and design documents that go deeper into how the code works and why it was built this way.

Code documentation

Within the code itself, you can use comments as a form of documentation. Terraform treats any text that begins with a hash (#) as a comment. Don’t use comments to explain what the code does; the code should do that itself. Only include comments to offer information that can’t be expressed in code, such as how the code is meant to be used or why the code uses a particular design choice. Terraform also allows every input and output variable to declare a `description` parameter, which is a great place to describe how that variable should be used.

Example code

As discussed in [Chapter 8](#), every Terraform module should include example code that shows how that module is meant to be used. This is a great way to highlight the intended usage patterns and give your users a way to try your module without having to write any code, and it’s the main way to add automated tests for the module.

Automated tests

All of [Chapter 9](#) focuses on testing Terraform code, so I won’t repeat any of that here, other than to say that infrastructure code without tests is broken. Therefore, one of the most important comments you can make in any code review is “How did you test this?”

File layout

Your team should define conventions for where Terraform code is stored and the file layout you use. Because the file layout for Terraform also determines the way Terraform state is stored, you should be especially mindful of how file layout affects your ability to provide isolation guarantees, such as ensuring that changes in a staging environment cannot accidentally cause problems in production. In a code review, you might want to enforce the file layout described in [“Isolation via File Layout”](#) on [page](#)

⁴ Writing the README first is called [Readme-Driven Development](#).

100, which provides isolation between different environments (e.g., stage and prod) and different components (e.g., a network topology for the entire environment and a single app within that environment).

Style guide

Every team should enforce a set of conventions about code style, including the use of whitespace, newlines, indentation, curly braces, variable naming, and so on. Although programmers love to debate spaces versus tabs and where the curly brace should go, the more important thing is that you are consistent throughout your codebase.

Terraform has a built-in `fmt` command that can reformat code to a consistent style automatically:

```
$ terraform fmt
```

I recommend running this command as part of a commit hook to ensure that all code committed to version control uses a consistent style.

Run Automated Tests

Just as with application code, your infrastructure code should have commit hooks that kick off automated tests in a CI server after every commit and show the results of those tests in the pull request. You already saw how to write unit tests, integration tests, and end-to-end tests for your Terraform code in [Chapter 9](#). There's one other critical type of test you should run: `terraform plan`. The rule here is simple:

Always run `plan` before `apply`.

Terraform shows the `plan` output automatically when you run `apply`, so what this rule really means is that you should always pause and read the `plan` output! You'd be amazed at the type of errors you can catch by taking 30 seconds to scan the “diff” you get as an output. A great way to encourage this behavior is by integrating `plan` into your code review flow. For example, [Atlantis](#) is an open source tool that automatically runs `terraform plan` on commits and adds the `plan` output to pull requests as a comment, as shown in [Figure 10-3](#).



Figure 10-3. Atlantis can automatically add the output of the `terraform plan` command as a comment on your pull requests.

Terraform Cloud and Terraform Enterprise, HashiCorp's paid tools, both support running `plan` automatically on pull requests as well.

Merge and Release

After your team members have had a chance to review the code changes and `plan` output and all the tests have passed, you can merge your changes into the `main` branch and release the code. Similar to application code, you can use Git tags to create a versioned release:

```
$ git tag -a "v0.0.6" -m "Updated hello-world-example text"
$ git push --follow-tags
```


Whereas with application code, you often have a separate artifact to deploy, such as a Docker image or VM image, since Terraform natively supports downloading code from Git, the repository at a specific tag is the immutable, versioned artifact you will be deploying.

Deploy

Now that you have an immutable, versioned artifact, it's time to deploy it. Here are a few of the key considerations for deploying Terraform code:

- Deployment tooling
- Deployment strategies
- Deployment server
- Promote artifacts across environments

Deployment tooling

When deploying Terraform code, Terraform itself is the main tool that you use. However, there are a few other tools that you might find useful:

Atlantis

The open source tool you saw earlier can not only add the plan output to your pull requests but also allows you to trigger a `terraform apply` when you add a special comment to your pull request. Although this provides a convenient web interface for Terraform deployments, be aware that it doesn't support versioning, which can make maintenance and debugging for larger projects more difficult.

Terraform Cloud and Terraform Enterprise

HashiCorp's paid products provide a web UI that you can use to run `terraform plan` and `terraform apply` as well as manage variables, secrets, and access permissions.

Terragrunt

This is an open source wrapper for Terraform that fills in some gaps in Terraform. You'll see how to use it a bit later in this chapter to deploy versioned Terraform code across multiple environments with minimal copying and pasting.

Scripts

As always, you can write scripts in a general-purpose programming language such as Python or Ruby or Bash to customize how you use Terraform.

Deployment strategies

For most types of infrastructure changes, Terraform doesn't offer any built-in deployment strategies: for example, there's no way to do a blue-green deployment for a VPC change, and there's no way to feature toggle a database change. You're essentially limited to `terraform apply`, which either works or it doesn't. A small subset of changes do support deployment strategies, such as the zero-downtime rolling deployment in the `asg-rolling-deploy` module you built in previous chapters, but these are the exceptions and not the norm.

Due to these limitations, it's critical to take into account what happens when a deployment goes wrong. With an application deployment, many types of errors are caught by the deployment strategy; for example, if the app fails to pass health checks, the load balancer will never send it live traffic, so users won't be affected. Moreover, the rolling deployment or blue-green deployment strategy can automatically roll back to the previous version of the app in case of errors.

Terraform, on the other hand, *does not roll back automatically in case of errors*. In part, that's because there is no reasonable way to roll back many types of infrastructure changes: for example, if an app deployment failed, it's almost always safe to roll back to an older version of the app, but if the Terraform change you were deploying failed, and that change was to delete a database or terminate a server, you can't easily roll that back!

Therefore, you should expect errors to happen and ensure you have a first-class way to deal with them:

Retries

Certain types of Terraform errors are transient and go away if you rerun `terraform apply`. The deployment tooling you use with Terraform should detect these known errors and automatically retry after a brief pause. Terragrunt has **automatic retries** on known errors as a built-in feature.

Terraform state errors

Occasionally, Terraform will fail to save state after running `terraform apply`. For example, if you lose internet connectivity partway through an `apply`, not only will the `apply` fail, but Terraform won't be able to write the updated state file to your remote backend (e.g., to Amazon S3). In these cases, Terraform will save the state file on disk in a file called `errored.tfstate`. Make sure that your CI server does not delete these files (e.g., as part of cleaning up the workspace after a build)! If you can still access this file after a failed deployment, as soon as internet connectivity is restored, you can push this file to your remote backend (e.g., to S3) using the `state push` command so that the state information isn't lost:

```
$ terraform state push errored.tfstate
```

Errors releasing locks

Occasionally, Terraform will fail to release a lock. For example, if your CI server crashes in the middle of a `terraform apply`, the state will remain permanently locked. Anyone else who tries to run `apply` on the same module will get an error message saying the state is locked and showing the ID of the lock. If you're absolutely sure this is an accidentally leftover lock, you can forcibly release it using the `force-unlock` command, passing it the ID of the lock from that error message:

```
$ terraform force-unlock <LOCK_ID>
```

Deployment server

Just as with your application code, all of your infrastructure code changes should be applied from a CI server and not from a developer's computer. You can run `terraform` from Jenkins, CircleCI, GitHub Actions, Terraform Cloud, Terraform Enterprise, Atlantis, or any other reasonably secure automated platform. This gives you the same benefits as with application code: it forces you to fully automate your deployment process, it ensures deployment always happens from a consistent environment, and it gives you better control over who has permissions to access production environments.

That said, permissions to deploy infrastructure code are quite a bit trickier than for application code. With application code, you can usually give your CI server a minimal, fixed set of permissions to deploy your apps; for example, to deploy to an ASG, the CI server typically needs only a few specific `ec2` and `autoscaling` permissions. However, to be able to deploy arbitrary infrastructure code changes (e.g., your Terraform code might try to deploy a database or a VPC or an entirely new AWS account), the CI server needs arbitrary permissions—that is, admin permissions. And that's a problem.

The reason it's a problem is that CI servers are (a) notoriously hard to secure,⁵ (b) accessible to all the developers at your company, and (c) used to execute arbitrary code. Adding permanent admin permissions to this mix is just asking for trouble! You'd effectively be giving every single person on your team admin permissions and turning your CI server into a very high-value target for attackers.

There are a few things you can do to minimize this risk:

Lock the CI server down

Make it accessible solely over HTTPs, require all users to be authenticated, and follow server-hardening practices (e.g., lock down the firewall, install fail2ban, enable audit logging, etc.).

⁵ See [10 real-world stories of how we've compromised CI/CD pipelines](#) for some eye-opening examples.

Don't expose your CI server on the public internet

That is, run the CI server in private subnets, without any public IP, so that it's accessible only over a VPN connection. That way, only users with valid network access (e.g., via a VPN certificate) can access your CI server at all. Note that this does have a drawback: webhooks from external systems won't work. For example, GitHub won't automatically be able to trigger builds in your CI server; instead, you'll need to configure your CI server to poll your version control system for updates. This is a small price to pay for a significantly more secure CI server.

Enforce an approval workflow

Configure your CI/CD pipeline to require that every deployment be approved by at least one person (other than the person who requested the deployment in the first place). During this approval step, the reviewer should be able to see both the code changes and the plan output, as one final check that things look OK before apply runs. This ensures that every deployment, code change, and plan output has had at least two sets of eyes on it.

Don't give the CI server permanent credentials

As you saw in [Chapter 6](#), instead of manually managed, permanent credentials (e.g., AWS access keys copy/pasted into your CI server), you should prefer to use authentication mechanisms that use temporary credentials, such as IAM roles and OIDC.

Don't give the CI server admin credentials

Instead, isolate the admin credentials to a totally separate, isolated *worker*: e.g., a separate server, a separate container, etc. That worker should be extremely locked down, so no developers have access to it at all, and the only thing it allows is for the CI server to trigger that worker via an extremely limited remote API. For example, that worker's API may only allow you to run specific commands (e.g., `terraform plan` and `terraform apply`), in specific repos (e.g., your live repo), in specific branches (e.g., the main branch), and so on. This way, even if an attacker gets access to your CI server, they still won't have access to the admin credentials, and all they can do is request a deployment on some code that's already in your version control system, which isn't nearly as much of a catastrophe as leaking the admin credentials fully.⁶

⁶ Check out [Gruntwork Pipelines](#) for a real-world example of this worker pattern.

Promote artifacts across environments

Just as with application artifacts, you'll want to promote your immutable, versioned infrastructure artifacts from environment to environment: for example, promote `v0.0.6` from dev to stage to prod.⁷ The rule here is also simple:

Always test Terraform changes in pre-prod before prod.

Because everything is automated with Terraform anyway, it doesn't cost you much extra effort to try a change in staging before production, but it will catch a huge number of errors. Testing in pre-prod is especially important because, as mentioned earlier in this chapter, Terraform does not roll back changes in case of errors. If you run `terraform apply` and something goes wrong, you must fix it yourself. This is easier and less stressful to do if you catch the error in a pre-prod environment rather than prod.

The process for promoting Terraform code across environments is similar to the process of promoting application artifacts, except there is an extra approval step, as mentioned in the previous section, where you run `terraform plan` and have someone manually review the output and approve the deployment. This step isn't usually necessary for application deployments, as most application deployments are similar and relatively low risk. However, every infrastructure deployment can be completely different, and mistakes can be very costly (e.g., deleting a database), so having one last chance to look at the `plan` output and review it is well worth the time.

Here's what the process looks like for promoting, for instance, `v0.0.6` of a Terraform module across the dev, stage, and prod environments:

1. Update the dev environment to `v0.0.6`, and run `terraform plan`.
2. Prompt someone to review and approve the plan; for example, send an automated message via Slack.
3. If the plan is approved, deploy `v0.0.6` to dev by running `terraform apply`.
4. Run your manual and automated tests in dev.
5. If `v0.0.6` works well in dev, repeat steps 1–4 to promote `v0.0.6` to staging.
6. If `v0.0.6` works well in staging, repeat steps 1–4 again to promote `v0.0.6` to production.

⁷ Credit for how to promote Terraform code across environments goes to Kief Morris: [Using Pipelines to Manage Environments with Infrastructure as Code](#).

One important issue to deal with is all the code duplication between environments in the *live* repo. For example, consider the *live* repo shown in [Figure 10-4](#).

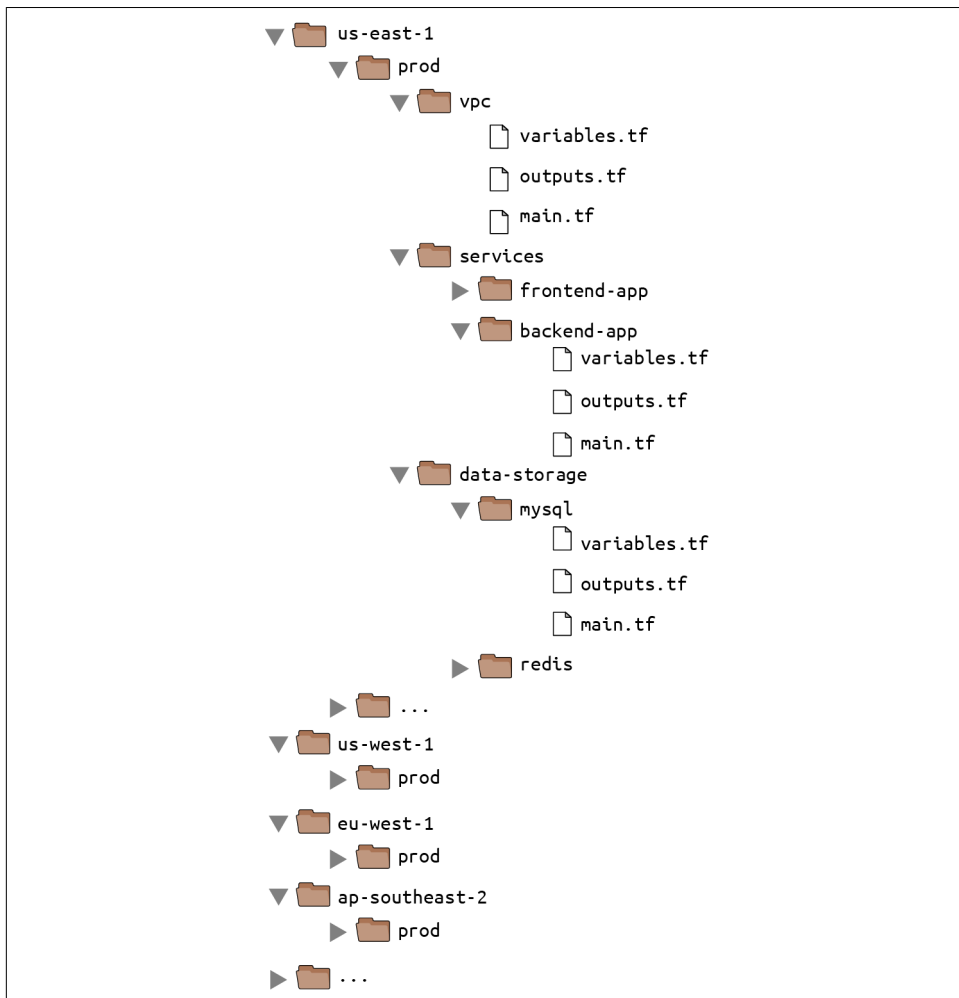


Figure 10-4. File layout with a large number of copy/pasted environments and modules within each environment.

This *live* repo has a large number of regions, and within each region, a large number of modules, most of which are copied and pasted. Sure, each module has a *main.tf* that references a module in your *modules* repo, so it's not as much copying and pasting as it could be, but even if all you're doing is instantiating a single module, there is still a large amount of boilerplate that needs to be duplicated between each environment:

- The provider configuration
- The backend configuration
- The input variables to pass to the module
- The output variables to proxy from the module

This can add up to dozens or hundreds of lines of mostly identical code in each module, copied and pasted into each environment. To make this code more DRY, and to make it easier to promote Terraform code across environments, you can use the open source tool I've mentioned earlier called Terragrunt. Terragrunt is a thin wrapper for Terraform, which means that you run all of the standard terraform commands, except you use terragrunt as the binary:

```
$ terragrunt plan
$ terragrunt apply
$ terragrunt output
```

Terragrunt will run Terraform with the command you specify, but based on configuration you specify in a *terragrunt.hcl* file, you can get some extra behavior. In particular, Terragrunt allows you to define all of your Terraform code exactly once in the *modules* repo, whereas in the *live* repo, you will have solely *terragrunt.hcl* files that provide a DRY way to configure and deploy each module in each environment. This will result in a *live* repo with far fewer files and lines of code, as shown in [Figure 10-5](#).

To get started, install Terragrunt by following the [instructions on the Terragrunt website](#). Next, add a provider configuration to *modules/data-stores/mysql/main.tf* and *modules/services/hello-world-app/main.tf*:

```
provider "aws" {
  region = "us-east-2"
}
```

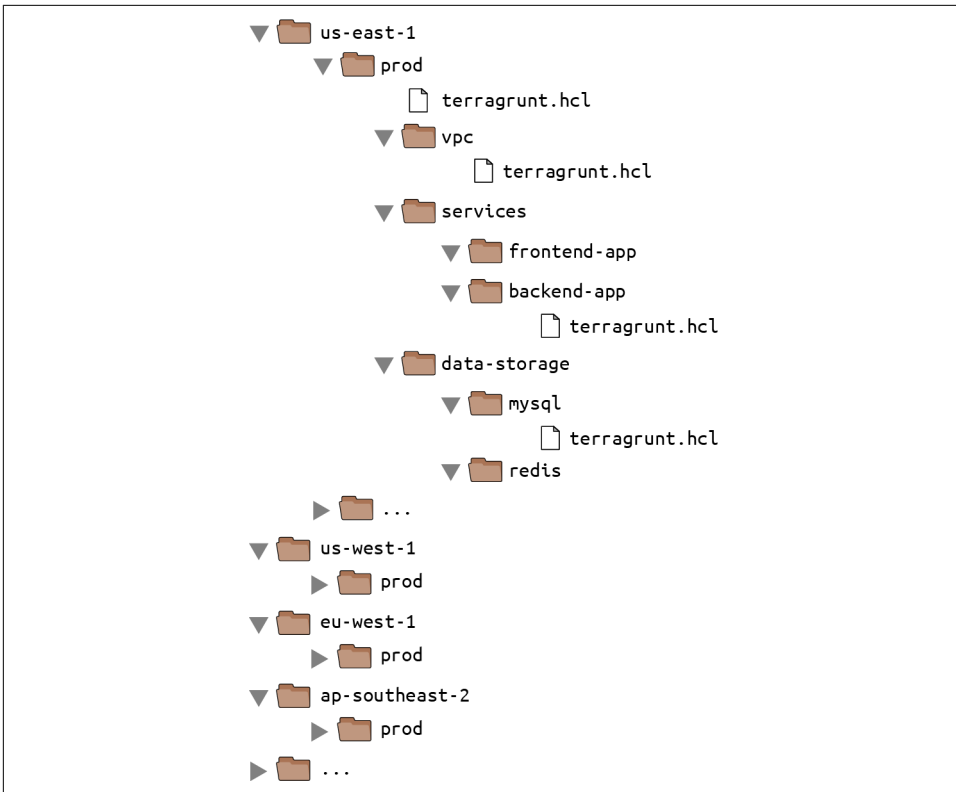


Figure 10-5. Use Terragrunt in your live repos to reduce the amount of code duplication.

Commit these changes and release a new version of your *modules* repo:

```

$ git add modules/data-stores/mysql/main.tf
$ git add modules/services/hello-world-app/main.tf
$ git commit -m "Update mysql and hello-world-app for Terragrunt"
$ git tag -a "v0.0.7" -m "Update Hello, World text"
$ git push --follow-tags

```

Now, head over to the *live* repo, and delete all the *.tf* files. You're going to replace all that copied and pasted Terraform code with a single *terragrunt.hcl* file for each module. For example, here's *terragrunt.hcl* for *live/stage/data-stores/mysql/terragrunt.hcl*:

```

terraform {
  source = "github.com/<OWNER>/modules//data-stores/mysql?ref=v0.0.7"
}

inputs = {
  db_name = "example_stage"

  # Set the username using the TF_VAR_db_username environment variable

```



```

    # Set the password using the TF_VAR_db_password environment variable
}

```

As you can see, *terragrunt.hcl* files use the same HashiCorp Configuration Language (HCL) syntax as Terraform itself. When you run `terragrunt apply` and it finds the source parameter in a *terragrunt.hcl* file, Terragrunt will do the following:

1. Check out the URL specified in `source` to a temporary folder. This supports the same URL syntax as the `source` parameter of Terraform modules, so you can use local file paths, Git URLs, versioned Git URLs (with a `ref` parameter, as in the preceding example), and so on.
2. Run `terraform apply` in the temporary folder, passing it the input variables that you've specified in the `inputs = { ... }` block.

The benefit of this approach is that the code in the *live* repo is reduced to just a single *terragrunt.hcl* file per module, which contains only a pointer to the module to use (at a specific version), plus the input variables to set for that specific environment. That's about as DRY as you can get.

Terragrunt also helps you keep your backend configuration DRY. Instead of having to define the bucket, key, `dynamodb_table`, and so on in every single module, you can define it in a single *terragrunt.hcl* file per environment. For example, create the following in *live/stage/terragrunt.hcl*:

```

remote_state {
  backend = "s3"

  generate = {
    path      = "backend.tf"
    if_exists = "overwrite"
  }

  config = {
    bucket      = "<YOUR_BUCKET>"
    key         = "${path_relative_to_include()}/terraform.tfstate"
    region     = "us-east-2"
    encrypt     = true
    dynamodb_table = "<YOUR_TABLE>"
  }
}

```

From this one `remote_state` block, Terragrunt can generate the backend configuration dynamically for each of your modules, writing the configuration in `config` to the file specified via the `generate` param. Note that the `key` value in `config` uses a Terragrunt built-in function called `path_relative_to_include()`, which will return the relative path between this root *terragrunt.hcl* file and any child module that includes

it. For example, to include this root file in *live/stage/data-stores/mysql/terragrunt.hcl*, add an include block:

```
terraform {
  source = "github.com/<OWNER>/modules//data-stores/mysql?ref=v0.0.7"
}

include {
  path = find_in_parent_folders()
}

inputs = {
  db_name = "example_stage"

  # Set the username using the TF_VAR_db_username environment variable
  # Set the password using the TF_VAR_db_password environment variable
}
```

The include block finds the root *terragrunt.hcl* using the Terragrunt built-in function `find_in_parent_folders()`, automatically inheriting all the settings from that parent file, including the `remote_state` configuration. The result is that this `mysql` module will use all the same backend settings as the root file, and the key value will automatically resolve to *data-stores/mysql/terraform.tfstate*. This means that your Terraform state will be stored in the same folder structure as your *live* repo, which will make it easy to know which module produced which state files.

To deploy this module, run `terragrunt apply`:

```
$ terragrunt apply --terragrunt-log-level debug
DEBU[0001] Reading Terragrunt config file at terragrunt.hcl
DEBU[0001] Included config live/stage/terragrunt.hcl
DEBU[0001] Downloading Terraform configurations into .terragrunt-cache
DEBU[0001] Generated file backend.tf
DEBU[0013] Running command: terraform init

(...)

Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

(...)

DEBU[0024] Running command: terraform apply

(...)

Terraform will perform the following actions:

(...)
```

Plan: 5 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

(...)

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Normally, Terragrunt only shows the log output from Terraform itself, but as I included `--terragrunt-log-level debug`, the preceding output shows what Terragrunt does under the hood:

1. Read the *terragrunt.hcl* file in the *mysql* folder where you ran `apply`.
2. Pull in all the settings from the included root *terragrunt.hcl* file.
3. Download the Terraform code specified in the source URL into the *.terragrunt-cache* scratch folder.
4. Generate a *backend.tf* file with your backend configuration.
5. Detect that `init` has not been run and run it automatically (Terragrunt will even create your S3 bucket and DynamoDB table automatically if they don't already exist).
6. Run `apply` to deploy changes.

Not bad for a couple of tiny *terragrunt.hcl* files!

You can now deploy the *hello-world-app* module in staging by adding *live/stage/services/hello-world-app/terragrunt.hcl* and running `terragrunt apply`:

```
terraform {
  source = "github.com/<OWNER>/modules//services/hello-world-app?ref=v0.0.7"
}

include {
  path = find_in_parent_folders()
}

dependency "mysql" {
  config_path = "../data-stores/mysql"
}

inputs = {
  environment = "stage"
  ami         = "ami-0fb653ca2d3203ac1"
```

```

min_size = 2
max_size = 2

enable_autoscaling = false

mysql_config = dependency.mysql.outputs
}

```

This *terragrunt.hcl* file uses the source URL and inputs just as you saw before and uses `include` to pull in the settings from the root *terragrunt.hcl* file, so it will inherit the same backend settings, except for the `key`, which will be automatically set to *services/hello-world-app/terraform.tfstate*, just as you'd expect. The one new thing in this *terragrunt.hcl* file is the dependency block:

```

dependency "mysql" {
  config_path = "../data-stores/mysql"
}

```

This is a Terragrunt feature that can be used to automatically read the output variables of another Terragrunt module, so you can pass them as input variables to the current module, as follows:

```
mysql_config = dependency.mysql.outputs
```

In other words, dependency blocks are an alternative to using `terraform_remote_state` data sources to pass data between modules. While `terraform_remote_state` data sources have the advantage of being native to Terraform, the drawback is that they make your modules more tightly coupled together, as each module needs to know how other modules store state. Using Terragrunt dependency blocks allows your modules to expose generic inputs like `mysql_config` and `vpc_id`, instead of using data sources, which makes the modules less tightly coupled and easier to test and reuse.

Once you've got *hello-world-app* working in staging, create analogous *terragrunt.hcl* files in *live/prod* and promote the exact same `v0.0.7` artifact to production by running `terragrunt apply` in each module.

Putting It All Together

You've now seen how to take both application code and infrastructure code from development all the way through to production. [Table 10-1](#) shows an overview of the two workflows side by side.

Table 10-1. Application and infrastructure code workflows

	Application code	Infrastructure code
Use version control	• <code>git clone</code> • One repo per app • Use branches	• <code>git clone</code> • <i>live</i> and <i>modules</i> repos • Don't use branches
Run the code locally	• Run on localhost • <code>ruby web-server.rb</code> • <code>ruby web-server-test.rb</code>	• Run in a sandbox environment • <code>terraform apply</code> • <code>go test</code>
Make code changes	• Change the code • <code>ruby web-server.rb</code> • <code>ruby web-server-test.rb</code>	• Change the code • <code>terraform apply</code> • <code>go test</code> • Use test stages
Submit changes for review	• Submit a pull request • Enforce coding guidelines	• Submit a pull request • Enforce coding guidelines
Run automated tests	• Tests run on CI server • Unit tests • Integration tests • End-to-end tests • Static analysis	• Tests run on CI server • Unit tests • Integration tests • End-to-end tests • Static analysis • <code>terraform plan</code>
Merge and release	• <code>git tag</code> • Create versioned, immutable artifact	• <code>git tag</code> • Use repo with tag as versioned, immutable artifact
Deploy	• Deploy with Terraform, orchestration tool (e.g., Kubernetes, Mesos), scripts • Many deployment strategies: rolling deployment, blue-green, canary • Run deployment on a CI server • Give CI server limited permissions • Promote immutable, versioned artifacts across environments • Once a pull request is merged, deploy automatically	• Deploy with Terraform, Atlantis, Terraform Cloud, Terraform Enterprise, Terragrunt, scripts • Limited deployment strategies (make sure to handle errors: retries, <code>errored.tfstate!</code>) • Run deployment on a CI server • Give CI server temporary credentials solely to invoke a separate, locked-down worker that has admin permissions • Promote immutable, versioned artifacts across environments • Once a pull request is merged, go through an approval workflow where someone checks the <code>plan</code> output one last time, and then deploy automatically

If you follow this process, you will be able to run application and infrastructure code in dev, test it, review it, package it into versioned, immutable artifacts, and promote those artifacts from environment to environment, as shown in [Figure 10-6](#).

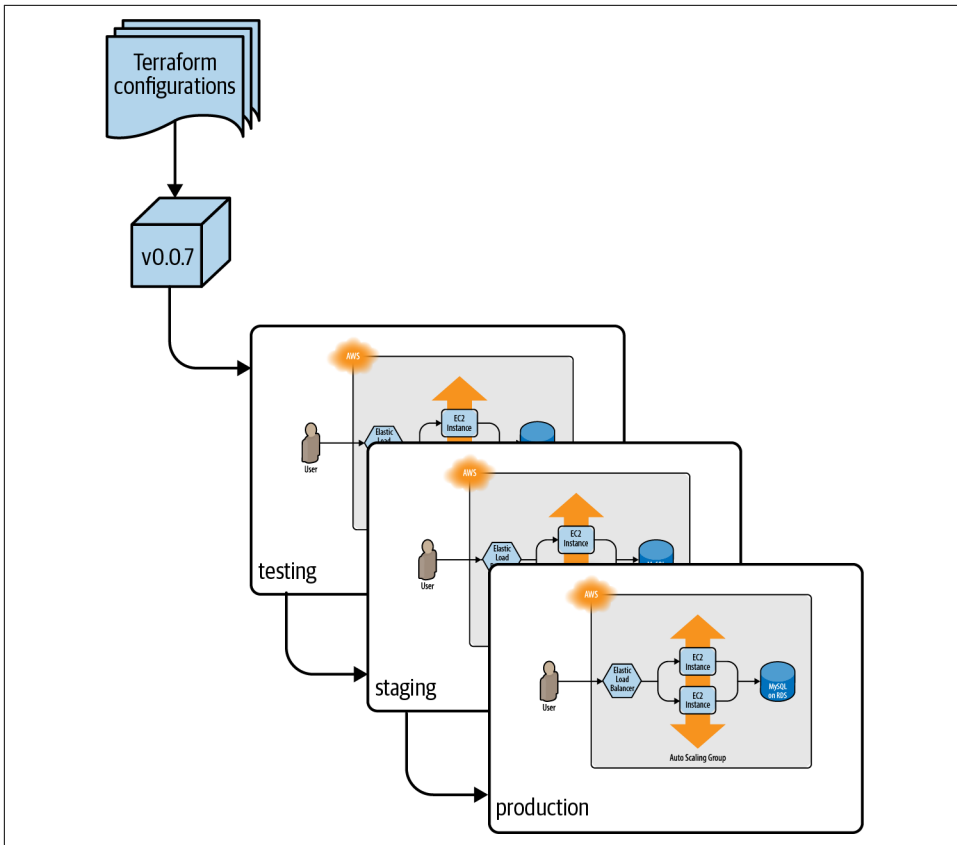


Figure 10-6. Promoting an immutable, versioned artifact of Terraform code from environment to environment.

Conclusion

If you've made it to this point in the book, you now know just about everything you need to use Terraform in the real world, including how to write Terraform code; how to manage Terraform state; how to create reusable modules with Terraform; how to do loops, if-statements, and deployments; how to manage secrets; how to work with multiple regions, accounts, and clouds; how to write production-grade Terraform code; how to test your Terraform code; and how to use Terraform as a team. You've worked through examples of deploying and managing servers, clusters of servers, load balancers, databases, scheduled actions, CloudWatch alarms, IAM users, reusable modules, zero-downtime deployment, AWS Secrets Manager, Kubernetes clusters, automated tests, and more. Phew! Just don't forget to run `terraform destroy` in each module when you're all done.

The power of Terraform, and more generally, IaC, is that you can manage all the operational concerns around an application using the same coding principles as the application itself. This allows you to apply the full power of software engineering to your infrastructure, including modules, code reviews, version control, and automated testing.

If you use Terraform correctly, your team will be able to deploy faster and respond to changes more quickly. Hopefully, deployments will become routine and boring—and in the world of operations, boring is a very good thing. And if you really do your job right, rather than spending all your time managing infrastructure by hand, your team will be able to spend more and more time improving that infrastructure, allowing you to go even faster.

This is the end of the book but just the beginning of your journey with Terraform. To learn more about Terraform, IaC, and DevOps, head over to [Appendix](#) for a list of recommended reading. And if you've got feedback or questions, I'd love to hear from you at jim@ybrikman.com. Thank you for reading!

Recommended Reading

The following are some of the best resources I've found on DevOps and infrastructure as code, including books, blog posts, newsletters, and talks.

Books

- *Infrastructure as Code: Dynamic Systems for the Cloud Age* by Kief Morris (O'Reilly)
- *Site Reliability Engineering: How Google Runs Production Systems* by Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy (O'Reilly)
- *The DevOps Handbook: How To Create World-Class Agility, Reliability, and Security in Technology Organizations* by Gene Kim, Jez Humble, Patrick Debois, and John Willis (IT Revolution Press)
- *Designing Data-Intensive Applications* by Martin Kleppmann (O'Reilly)
- *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation* by Jez Humble and David Farley (Addison-Wesley Professional)
- *Release It! Design and Deploy Production-Ready Software* by Michael T. Nygard (The Pragmatic Bookshelf)
- *Kubernetes in Action* by Marko Luksa (Manning)
- *Leading the Transformation: Applying Agile and DevOps Principles at Scale* by Gary Gruver and Tommy Mouser (IT Revolution Press)
- *Visible Ops Handbook* by Kevin Behr, Gene Kim, and George Spafford (Information Technology Process Institute)
- *Effective DevOps* by Jennifer Davis and Ryn Daniels (O'Reilly)

- *Lean Enterprise* by Jez Humble, Joanne Molesky, Barry O'Reilly (O'Reilly)
- *Hello, Startup: A Programmer's Guide to Building Products, Technologies, and Teams* by Yevgeniy Brikman (O'Reilly)

Blogs

- High Scalability
- Code as Craft
- AWS News Blog
- Kitchen Soap
- Paul Hammant's blog
- Martin Fowler's blog
- Gruntwork Blog
- Yevgeniy Brikman blog

Talks

- “Reusable, Composable, Battle-Tested Terraform Modules” by Yevgeniy Brikman
- “5 Lessons Learned from Writing Over 300,000 Lines of Infrastructure Code” by Yevgeniy Brikman
- “Automated Testing for Terraform, Docker, Packer, Kubernetes, and More” by Yevgeniy Brikman
- “Infrastructure as Code: Running Microservices on AWS using Docker, Terraform, and ECS” by Yevgeniy Brikman
- “Agility Requires Safety” by Yevgeniy Brikman
- “Adopting Continuous Delivery” by Jez Humble
- “Continuously Deploying Culture” by Michael Rembetsy and Patrick McDonnell
- “10+ Deploys Per Day: Dev and Ops Cooperation at Flickr” by John Allspaw and Paul Hammond
- “Why Google Stores Billions of Lines of Code in a Single Repository” by Rachel Potvin
- “The Language of the System” by Rich Hickey
- “Real Software Engineering” by Glenn Vanderburg

Newsletters

- [DevOps Weekly](#)
- [Gruntwork Newsletter](#)
- [Terraform: Up & Running Newsletter](#)
- [Terraform Weekly Newsletter](#)

Online Forums

- [Terraform subforum of HashiCorp Discuss](#)
- [Terraform subreddit](#)
- [DevOps subreddit](#)

A

- Access Key IDs (in AWS), 42, 44, 196-199, 200-201
- accessing secrets, interfaces for, 195
 - (see also secrets management)
- accidental complexity, 278
- accounts (in AWS)
 - configuring multiple, 238-245
 - setup, 40-42
- ad hoc scripts, 4-5
- agent versus agentless software, 28-29
- ALB (Application Load Balancer), 71-78, 283-285
- aliases, 227
 - configuration aliases, 247-248
 - for Gmail, 241
 - KMS CMK aliases, 210
 - when to use, 236-237, 245
- Amazon S3 (Simple Storage Service), as remote backend, 85-91
- Amazon Web Services (see AWS)
- AMI (Amazon Machine Image)
 - ami parameter, 46
 - Packer template, 10-11
 - zero-downtime deployment, 169-179
- AMI IDs, managing, 229-230
- ami parameter, 14, 46
- Ansible, comparison with other IaC tools, 20-36
- application code workflow
 - automated tests, 385
 - changing code, 383-384
 - comparison with infrastructure code workflow, 412
 - deployment, 387-390
 - locally run code, 383
 - merge and release, 386
 - reviewing code changes, 384
 - steps in, 382
 - version control, 382
- Application Load Balancer (ALB), 71-78, 283-285
- arguments for resources, 45
- arrays
 - lookup syntax, 144
 - of resources, 145
- ASGs (Auto Scaling Groups), 66-70
 - instance refresh, 181-184
 - scheduled actions, 127-129
 - zero-downtime deployment, 171-179
- assume role policies, 201
- Atlantis, 399-401
- auditing multiple AWS accounts, 239
- authentication (see secrets management)
- auto healing, 12
- auto scaling, 12, 127-129, 160-162
- automated tests, 398
 - in application code workflow, 385
 - end-to-end tests, 362-364
 - in infrastructure code workflow, 399-400
- integration tests
 - with multiple modules, 349-354
 - retries in, 360-361
 - Ruby comparison, 348-349
 - stages of, 354-360
- plan testing, 366-370
- purpose of, 320
- server testing, 370-372

- static analysis, 364-366
- types of, 320-321
- unit tests
 - dependency injection, 332-343
 - Ruby comparison, 321-324
 - running in parallel, 343-347
 - with Terratest, 324-332
- when to use, 297
- Availability Zones (AZs), 45, 179-180
- AWS (Amazon Web Services)
 - accounts
 - configuring multiple, 238-245
 - setup, 40-42
 - AZs (Availability Zones), 45, 179-180
 - benefits of, 39
 - configuring as provider, 45
 - regions, 45, 226-237
- aws eks update-kubeconfig command, 270
- AWS Provider, xx, 222
- AZs (Availability Zones), 45, 179-180

B

- backends
 - configuring with Terragrunt, 409-411
 - local backends, 84
 - remote backends
 - limitations of, 91-93
 - shared storage with, 84-91
- Bash scripts, 305
 - as ad hoc scripts, 4-5
 - externalizing, 111-113
- best practices for testing, 373
- blocking public access to S3 buckets, 87
- blue-green deployment, 388
- branches, reasons to avoid, 393-395
- bucket parameter, 86, 88
- built-in functions, explained, 111-113
- bus factor, 16

C

- canary deployment, 389
- centralized secret stores, 194
- changing code
 - in application code workflow, 383-384
 - in infrastructure code workflow, 396
- checklist for production-grade infrastructure, 279-280
- Chef, comparison with other IaC tools, 20-36
- child accounts, creating, 239-241

- CI servers
 - automated tests, 385
 - as deployment servers, 389, 403-404
 - machine user authentication, 199-206, 219
- CI/CD workflow
 - for application code
 - automated tests, 385
 - changing code, 383-384
 - comparison with infrastructure code workflow, 412
 - deployment, 387-390
 - locally run code, 383
 - merge and release, 386
 - reviewing code changes, 384
 - steps in, 382
 - version control, 382
 - CircleCI, 199-201
 - EC2 Instances with IAM roles, 201-204
 - GitHub Actions, 204-206
 - for infrastructure code
 - automated tests, 399-400
 - changing code, 396
 - comparison with application code workflow, 412
 - deployment, 401-412
 - locally run code, 395
 - merge and release, 400-401
 - reviewing code changes, 397-399
 - steps in, 390
 - version control, 391-395
- CIDR blocks, 54, 366
- CircleCI, 199-201, 385
- CircleCI Context, 200
- CLB (Classic Load Balancer), 71
- cleaning up manual tests, 319-320
- CLI (command-line interface), 195
- cloud providers (see providers)
- CloudFormation, comparison with other IaC tools, 20-36
- cluster of web servers
 - deploying, 66-70
 - refactoring, 283-285
- clusters (Kubernetes), 12
 - deploying Docker containers, 264-272
 - inspecting, 262-264
- CMK (Customer Managed Key), 209
- code examples
 - fair use permissions, xx-xxi
 - location of, xix-xx

- versions used for, [xx](#)
- coding guidelines, [397-399](#)
- collaboration using Git repositories, [51](#)
- combining IaC tools, [34-36](#)
- command history, avoiding writing to, [197](#)
- command-line interface (CLI), [195](#)
- comments, as documentation, [398](#)
- community size in IaC tool comparison, [31-33](#)
- comparison of IaC tools, [20-36](#)
 - agent versus agentless, [28-29](#)
 - combining tools, [34-36](#)
 - configuration management versus provisioning, [21](#)
 - general-purpose versus domain-specific language, [25-26](#)
 - large versus small community, [31-33](#)
 - master versus masterless, [26-27](#)
 - mature versus cutting edge, [33](#)
 - mutable versus immutable infrastructure, [21-22](#)
 - paid versus free, [30-31](#)
 - procedural versus declarative language, [22-25](#)
- composable modules for production-grade infrastructure, [285-291](#)
- concat built-in function, [165](#)
- conditionals
 - count parameter
 - if-else-statements with, [163-165](#)
 - if-statements with, [160-162](#)
 - for_each and for expressions, [165-167](#)
 - if string directive, [167-168](#)
 - ternary syntax, [99](#), [161](#), [165](#)
 - types and purpose of, [160](#)
- configurable web servers, deploying, [60-66](#)
- configuration aliases, [247-248](#)
- configuration drift, [2](#)
- configuration files
 - explained, [18](#)
 - naming conventions, [102-103](#)
- configuration management tools, [5-7](#)
 - combining with provisioning, [34](#)
 - provisioning tools versus, [21](#)
- container engines, [9](#)
- containers, [9](#)
 - deploying with EKS (Elastic Kubernetes Service), [264-272](#)
 - explained, [250-252](#)
 - Pods, [13](#), [257-258](#)

- control plane (Kubernetes), [253](#), [266](#)
- cost in IaC tool comparison, [30-31](#)
- count parameter
 - conditionals with
 - if-else-statements, [163-165](#)
 - if-statements, [160-162](#)
 - limitations, [147-149](#), [179-180](#)
 - loops with, [142-149](#)
- create_before_destroy lifecycle setting, [68](#), [141](#), [171](#), [181](#), [188](#)
- creation-time provisioners, [309](#)
- credentials (see secrets)
- Customer Managed Key (CMK), [209](#)
- customer secrets, [193](#)

D

- data sources
 - external, [311-312](#)
 - replicating in multiple AWS regions, [230-236](#)
 - secrets management, [206](#)
 - with encrypted files, [209-213](#)
 - with environment variables, [207-209](#)
 - with secret stores, [213-217](#)
 - for subnets, [69-70](#)
 - terraform_remote_state, [105-113](#)
- declarative language
 - procedural versus, [22-25](#)
 - Terraform as, [141](#)
- declaring variables, [60-66](#)
- default parameter, [60](#)
- Default VPC (in AWS), [42](#), [59](#), [69](#)
- default_tags block, [155](#)
- defining listeners, [72](#)
- dependencies
 - implicit, [55](#)
 - in Terragrunt, [412](#)
 - types of, [298](#)
 - versioning pinning, [298-303](#)
- dependency injection, [332-343](#)
- depends_on parameter, [65](#)
- deploying
 - application code, [387-390](#)
 - Docker with EKS (Elastic Kubernetes Service), [264-272](#)
 - infrastructure code, [401-412](#)
 - load balancers, [70-78](#)
 - modules in multiple AWS regions, [230-236](#)
 - resources in multiple AWS regions, [228-230](#)

- servers
 - cluster of web servers, 66-70
 - configurable web servers, 60-66
 - single servers, 44-52
 - web servers, 52-59
 - with zero-downtime deployment, 169-179
 - deployment servers, 389, 403-404
 - deployment strategies, 387-389, 402-403
 - deployment tools, 387, 401
 - Deployments (Kubernetes), 13, 254-259
 - description parameter, 60, 64
 - destroy-time provisioners, 309
 - DevOps, 1-3
 - project time estimates, 277-279
 - resources for information, 417-419
 - Docker
 - containers (see containers)
 - deploying with EKS (Elastic Kubernetes Service), 264-272
 - explained, 249-252
 - installing, 249
 - purpose of, 11
 - Docker Hub, 250
 - docker kill command, 263
 - docker ps command, 262
 - Docker Registry, 250
 - docker rm command, 252
 - docker run command, 249
 - docker start command, 251
 - documentation
 - IaC as, 16
 - for Terraform, xix, 46
 - types of, 397-398
 - DRY (Don't Repeat Yourself) principle, 60, 93, 407
 - DSL (domain-specific language) versus GPL (general-purpose programming language), 25-26
 - DynamoDB tables
 - limitations of remote backends, 91-93
 - locking with, 87
 - dynamodb_table parameter, 89
- ## E
- EC2 Instances, 45, 49
 - (see also servers)
 - deploying in multiple AWS regions, 228-230
 - IAM roles for, 201-204
 - EKS (Elastic Kubernetes Service), 264-272
 - ELB (Elastic Load Balancer) service, 70
 - enabling
 - server-side encryption, 87
 - versioning in Amazon S3, 86
 - encrypt parameter, 89
 - encryption
 - secrets management with, 209-213
 - server-side, enabling, 87
 - end-to-end tests, 321, 362-364
 - enforcing tagging standards, 155
 - environment variables, secrets management
 - with, 107, 197-199, 207-209
 - environments
 - folders for, 101-102
 - promotion across, 390, 405-412
 - sandbox, 319-320, 330, 396
 - errors in deployment, 402-403
 - essential complexity, 278
 - example code, as documentation, 398
 - examples
 - fair use permissions, xx-xxi
 - location of, xix-xx
 - versions used for, xx
 - executing scripts
 - for external data source, 311-312
 - with provisioners, 306-310
 - expressions, 55
 - external data source, 311-312
 - externalizing Bash scripts, 111-113
- ## F
- fair use permissions for code examples, xx-xxi
 - false incrementalism, 379
 - feature toggles, 389
 - file layout
 - conventions for, 398
 - isolating state files, 100-105
 - file paths
 - local, 138
 - path references for, 129-130
 - file-based secret stores, 194
 - folders
 - for environments, 101-102
 - running commands across multiple, 104
 - for expressions
 - conditionals with, 165-167
 - loops with, 156-158
 - for string directive, loops with, 158-159
 - format built-in function, 111

- for_each expressions
 - conditionals with, 165-167
 - limitations, 179-180
 - loops with, 149-154
- function composition, 285-291

G

- get deployments command, 262, 271
- get nodes command, 254, 270
- get pods command, 262, 271
- get services command, 263, 271
- git commit command, 52
- git push command, 52
- Git repositories
 - collaborating with teammates, 51
 - SSH authentication for, 137
- GitHub Actions, 204-206
- GitHub, pull requests, 384
- .gitignore file, 51
- Gmail aliases, 241
- Go, installing, 325
- Golden Rule of Terraform, 392-393
- GPL (general-purpose programming language)
 - versus DSL (domain-specific language), 25-26

H

- HCL (HashiCorp Configuration Language), 44
- health checks, 74
- heredoc syntax, 54, 167
- history files, avoiding writing to, 197
- Hofstadter's Law, 277
- human users, secrets management and, 197-199
- hypervisors, 8

I

- IaC (infrastructure as code), 275
 - (see also production-grade infrastructure)
 - ad hoc scripts, 4-5
 - benefits of, 16-17
 - configuration management tools, 5-7
 - importing into state files, 185
 - orchestration tools, 12-14
 - out of band resources, 185
 - provisioning tools, 14
 - purpose of, 3
 - refactoring, 186-189
 - resources for information, 417-419

- server templating tools, 7-11
- team adoption of
 - incrementalism in, 379-380
 - learning curve, 380-381
 - return on investment, 376-379
- tool comparison, 20-36
 - agent versus agentless, 28-29
 - combining tools, 34-36
 - configuration management versus provisioning, 21
 - general-purpose versus domain-specific language, 25-26
 - large versus small community, 31-33
 - master versus masterless, 26-27
 - mature versus cutting edge, 33
 - mutable versus immutable infrastructure, 21-22
 - paid versus free, 30-31
 - procedural versus declarative language, 22-25
 - validating, 319
- IAM (Identity and Access Management) service, 40
- IAM OIDC identity providers, 204
- IAM Policies, 41
- IAM roles
 - child account authentication, 243
 - cross-account authentication, 244
 - for EC2 Instances, 201-204
- IAM users
 - changing policies, 163-165
 - creating, 40, 142-152
 - errors, 184-186
- idempotence, 6, 47, 89
- if string directive, conditionals with, 167-168
- if-else-statements with count parameter, 163-165
- if-statements with count parameter, 160-162
- images (of servers), 7-11
- immutable infrastructure
 - explained, 11
 - mutable versus, 21-22
 - promotion across environments, 390, 405
 - releasing application code, 386
 - workflows, 412-413
- implicit dependencies, 55
- importing infrastructure, 185
- incrementalism in IaC (infrastructure as code)
 - adoption, 379-380

- infrastructure as code (see IaC)
- infrastructure code workflow
 - automated tests, 399-400
 - changing code, 396
 - comparison with application code workflow, 412
 - deployment, 401-412
 - locally run code, 395
 - merge and release, 400-401
 - reviewing code changes, 397-399
 - steps in, 390
 - version control, 391-395
- infrastructure secrets, 194
- inline blocks
 - creating multiple with `for_each` expressions, 152-154
 - separate resources versus, 130-132
- input variables, 60-64
 - in composable modules, 286-291
 - in modules, 121-124
- inspecting Kubernetes clusters, 262-264
- installing
 - Docker, 249
 - Go, 325
 - kubectrl, 254
 - providers, 223-225
 - Terraform, 43-44, 300
 - Terragrunt, 407
- instance metadata endpoints, 203
- instance profiles, 202
- instance refresh, 181-184
- instance_type parameter, 46
- integration tests
 - purpose of, 321
 - retries in, 360-361
 - Ruby comparison, 348-349
 - stages of, 354-360
 - with multiple modules, 349-354
- interfaces, accessing secrets, 195
- interpolation, 64
- isolating
 - multiple AWS accounts, 238
 - at network level, 133
 - state files, 83
 - purpose of, 93-94
 - via file layout, 100-105
 - via workspaces, 94-100

K

- kernel, 9
- kernel space, 9
- key parameter, 88
- key policies, 209
- KMS (key management system), 194
- kubectrl, 254, 262-263
- Kubernetes
 - clusters, 12
 - deploying Docker containers, 264-272
 - inspecting, 262-264
 - control plane, 253, 266
 - EKS (Elastic Kubernetes Service), 264-272
 - explained, 252-264
 - kubectrl, 254, 262-263
 - objects in, 254
 - Pods, 13, 257-258
 - running, 253
 - worker nodes, 253, 266
- Kubernetes Deployment, 13, 254-259
- Kubernetes Service, 255, 259-261

L

- large modules, limitations of, 281
- launch configurations, 67
- launch templates, 67
- learning curve in IaC (infrastructure as code)
 - adoption, 380-381
- length built-in function, 144
- lifecycle settings, 68, 86, 171
- list comprehensions (Python), 156
- listener rules, creating, 75
- listeners, defining, 72
- literals, 55
- live repository, 391-393, 406-412
- load balancing, 12, 70-78
- local backends, 84
- local file paths, 138
- local names, 224
- local references, 126
- local values in modules, 125-127
- locally run code
 - in application code workflow, 383
 - in infrastructure code workflow, 395
- lock files, 302
- locking
 - with DynamoDB tables, 87
 - with remote backends, 84
 - state files, 83

- version control disadvantages of, 84
- lookup syntax for arrays, 144
- loops
 - count parameter, 142-149
 - for expressions, 156-158
 - for string directive, 158-159
 - for_each expressions, 149-154
 - types and purpose of, 142

M

- machine users, secrets management and, 199-206, 219
- managed node groups, 266-268
- Managed Policies (in AWS), 41
- manual tests
 - cleanup, 319-320
 - explained, 317-319
 - Ruby comparison, 316-317
- maps of resources, 150
- master versus masterless servers, 26-27
- maturity in IaC tool comparison, 33
- merging code
 - in application code workflow, 386
 - in infrastructure code workflow, 400-401
- mocks, 320
- modules
 - authenticating multiple AWS accounts, 242-244
 - configuring for Kubernetes, 261-262
 - count parameter, 146-147
 - creating for Kubernetes, 255-261
 - creating reusable, 118-120
 - deploying in multiple AWS regions, 230-236
 - for_each expressions, 152
 - inline blocks versus separate resources, 130-132
 - input variables, 121-124
 - integration tests, 349-354
 - local values, 125-127
 - for multiple providers, 245-248
 - output variables, 127-129
 - path references, 129-130
 - for production-grade infrastructure composable modules, 285-291
 - non-Terraform code in, 305-312
 - small modules, 281-285
 - testable modules, 291-298
 - versioned modules, 298-305
 - publishing, 304-305

- purpose of, 92, 117
- repository for, 391-392
- self-validating, 293
 - precondition/postcondition blocks in, 295-297
 - validation blocks in, 294-295
- tagging standards, 155
- types of, 118, 245
- unit tests for, 324-332
- versioning, 133-138
- versioning pinning, 302-303
- multiple AWS accounts, configuring, 238-245
- multiple AWS regions, configuring, 226-237
- multiple folders, running commands across, 104
- multiple inline blocks, creating with for_each expressions, 152-154
- multiple modules, integration tests with, 349-354
- multiple providers
 - Docker, explained, 249-252
 - EKS (Elastic Kubernetes Service), 264-272
 - Kubernetes, explained, 252-264
 - multiple AWS accounts, 238-245
 - multiple AWS regions, 226-237
 - multiple copies of different providers, 248-272
 - multiple copies of same provider, 226-248
 - reusable modules for, 245-248
- mutable versus immutable infrastructure, 21-22

N

- naming
 - configuration files, 102-103
 - resources, 49
- network isolation, 133
- network security
 - CIDR blocks, 54, 366
 - data sources for subnets, 69
 - Default VPC (in AWS), 42
 - network isolation, 133
 - port numbers and, 53
 - security groups, creating, 54, 73
 - VPC subnets, 59
- NLB (Network Load Balancer), 71
- non-Terraform code, running in modules, 305-312
- null_resource, 310-311

O

- objects (Kubernetes), 254
- OIDC (Open ID Connect), 204-206
- one built-in function, 165
- OPA (Open Policy Agent), 368-370
- OpenStack Heat, comparison with other IaC tools, 20-36
- orchestration tools, 12-14
 - combining with provisioning and server templating, 35-36
 - for deployment, 387
 - purpose of, 252
- out of band resources, 185
- output variables, 64-66
 - in composable modules, 286-291
 - in modules, 127-129

P

- Packer, 7-11
- parallel unit tests, running, 343-347
- partial configurations, 92, 351-352
- passing secrets via environment variables, 107, 197-199
- path references in modules, 129-130
- permissions for code examples, xx-xxi
- personal secrets, 193
- plain text, avoiding secrets in, 192-193
- plan files, secrets management, 218-219
- plan testing, 366-370, 399
- plugins, providers as, 222-223
- Pod Template, 257-258
- Pods, 13, 257-258
- policy-as-code tools, 368-370
- port numbers, 53
- postcondition blocks, 295-297
- precondition blocks, 295-297
- preferred local names, 224
- prevent_destroy parameter, 86
- private API, state files as, 83
- private Git repositories, 137
- private keys, 194
- private subnets, 60
- Private Terraform Registry, 305
- procedural versus declarative language, 22-25
- production-grade infrastructure
 - checklist for, 279-280
 - definition of, 275
 - modules for
 - composable modules, 285-291

- non-Terraform code in, 305-312
- small modules, 281-285
- testable modules, 291-298
- versioned modules, 298-305
- time estimates for, 275-279
- promotion across environments, 390, 405-412
- providers, 249
 - (see also multiple providers)
 - configuring, 45, 225-226
 - multiple AWS accounts, 238-245
 - multiple AWS regions, 226-237
 - in reusable modules, 245-248
 - installing, 223-225
 - as plugins, 222-223
 - secrets management, 196-197
 - human users and, 197-199
 - machine users and, 199-206, 219
 - transparent portability, 19
 - versioning pinning, 301-302
- provisioners
 - executing scripts, 306-310
 - with null_resource, 310-311
 - User Data scripts versus, 310
- provisioning tools, 14
 - combining with configuration management, 34
 - combining with orchestration and server templating, 35-36
 - combining with server templating, 35-35
 - configuration management tools versus, 21
- public access to S3 buckets, blocking, 87
- public keys, 194
- public subnets, 59
- Public Terraform Registry, 304-305
- publishing modules, 304-305
- pull requests, 384
- Pulumi, comparison with other IaC tools, 20-36
- Puppet, comparison with other IaC tools, 20-36

R

- random integers, generating, 180
- RDS (Relational Database Service), 105
- README files, 398
- reconciliation loops, 254
- refactoring
 - Terraform code, 186-189
 - web server clusters, 283-285
- references, 55
- region parameter, 89

- regions (in AWS), 45, 226-237
- Rego, 368-370
- Relational Database Service (RDS), 105
- relative filepaths, 129
- releasing code
 - in application code workflow, 386
 - in infrastructure code workflow, 400-401
- remote backends
 - limitations of, 91-93
 - shared storage with, 84-91
- remote procedure calls (RPCs), 222
- removing resources, 79-80
- replicas, 13
- replicating data sources in multiple AWS regions, 230-236
- repositories
 - sharing, 51
 - SSH authentication for, 137
 - for version control, 391-392
- required_providers block, 224-225
- required_version parameter, 298
- resource attribute references, 55
- resource dependencies, 105
- resources
 - arrays of, 145
 - creating, 45
 - creating out of band, 185
 - deploying in multiple AWS regions, 228-230
 - lifecycle settings, 68
 - maps of, 150
 - naming, 49
 - null_resource, 310-311
 - removing, 79-80
 - secrets management, 206
 - with encrypted files, 209-213
 - with environment variables, 207-209
 - with secret stores, 213-217
 - separate resources versus inline blocks, 130-132
 - tagging standards, 155
- resources for information, 417-419
- retries
 - in integration tests, 360-361
 - with known errors, 402
- return on investment for IaC (infrastructure as code), 376-379
- reusability of code, 17
- reusable modules, 245
 - creating, 118-120

- for multiple providers, 245-248
- reviewing code changes
 - in application code workflow, 384
 - for infrastructure code, 397-399
- rolling deployments, 7, 388-388
- root accounts, 239
- root modules, 118, 245
- root users, 40
 - port numbers and, 53
- RPCs (remote procedure calls), 222
- running
 - Kubernetes, 253
 - parallel unit tests, 343-347

S

- S3 buckets
 - creating, 85-86
 - limitations of remote backends, 91-93
 - public access, blocking, 87
 - server-side encryption, enabling, 87
 - versioning, enabling, 86
- sandbox environments, 319-320, 330, 396
- scheduled actions, 127, 160
- scripts
 - for deployment, 387, 401
 - executing
 - for external data source, 311-312
 - with provisioners, 306-310
- Secret Access Keys (in AWS), 42, 44
- secret stores, secrets management with, 213-217
- secrets
 - access interface for, 195
 - avoiding in plain text, 192-193
 - passing via environment variables, 107
 - with remote backends, 85
 - storage methods, 194-195
 - types of, 193-194
 - version control disadvantages, 84
- secrets management
 - CI/CD workflow
 - CircleCI, 199-201
 - EC2 Instances with IAM roles, 201-204
 - GitHub Actions, 204-206
 - multiple AWS accounts, 238, 242-244
 - for plan files, 218-219
 - for providers, 196-197
 - human users and, 197-199
 - machine users and, 199-206, 219

- for resources and data sources, 206
 - with encrypted files, 209-213
 - with environment variables, 207-209
 - with secret stores, 213-217
- rules of, 192-193
- for state files, 217-218
- Terraform authentication options, 44
- tool comparison, 195-196
- security (see network security; secrets management)
- security groups, creating, 54, 73
- selector block, 259
- self-validating modules, 293
 - precondition/postcondition blocks in, 295-297
 - validation blocks in, 294-295
- semantic versioning, 136
- sensitive parameter, 61, 65
- separate resources versus inline blocks, 130-132
- server templating tools, 7-11
 - combining with orchestration and provisioning, 35-36
 - combining with provisioning, 35
- server testing, 370-372
- server-side encryption, enabling, 87
- servers, deploying
 - cluster of web servers, 66-70
 - configurable web servers, 60-66
 - single servers, 44-52
 - web servers, 52-59
- service discovery, 12
- shared storage for state files, 83-91
 - with remote backends, 84-91
 - version control disadvantages, 83-84
- sharing Git repositories with teammates, 51
- small modules for production-grade infrastructure, 281-285
- snowflake servers, 2
- software delivery, 1
- sops, 212
- splat expression, 145
- Spolsky, Joel, 382
- SSH authentication for Git repositories, 137
- stages of integration tests, 354-360
- state files
 - explained, 82-83
 - importing infrastructure, 185
 - isolating, 83
 - purpose of, 93-94
 - via file layout, 100-105
 - via workspaces, 94-100
- locking, 83
- as private API, 83
- secrets management, 217-218
- shared storage for, 83-91
 - with remote backends, 84-91
 - version control disadvantages, 83-84
- terraform_remote_state data source, 105-113
- static analysis, 364-366
- storing secrets, 194-195
- string directives, 158
- strip markers, 168
- structural types, 62
- style guidelines, 399
- subnets
 - data sources for, 69-70
 - of VPCs, 59
- subnets parameter, 72
- subnet_ids parameter, 69

T

- tagging standards, enforcing, 155
- target groups, creating, 74
- TDD (Test-Driven Development), 293
- team adoption of IaC
 - incrementalism in, 379-380
 - learning curve, 380-381
 - return on investment, 376-379
- templatefile built-in function, 111-113, 129
- ternary syntax, 99, 161, 165
- Terraform
 - authentication options, 44
 - changes, 2017-2019, xvi-xviii
 - comparison with other IaC tools, 20-36
 - agent versus agentless, 28-29
 - combining tools, 34-36
 - configuration management versus provisioning, 21
 - general-purpose versus domain-specific language, 25-26
 - large versus small community, 31-33
 - master versus masterless, 26-27
 - mature versus cutting edge, 33
 - mutable versus immutable infrastructure, 21-22
 - paid versus free, 30-31

- procedural versus declarative language,
 - 22-25
- core, 222, 298-301
- as declarative language, 141
- as deployment tool, 387
- documentation, xix, 46
- Golden Rule of Terraform, 392-393
- installing, 43-44, 300
- new features, xiv-xvi
- operational overview, 17-19
- purpose of, ix
- refactoring, 186-189
- transparent portability, 19
- version used, xx
- terraform apply command, 48
- Terraform Cloud, 401
- Terraform configurations, 18
- terraform console command, 111
- terraform destroy command, 79
- Terraform Enterprise, 401
- terraform fmt command, 399
- terraform graph command, 56
- terraform import command, 185
- terraform init command, 47, 89
- terraform output command, 65
- terraform plan command, 47, 184-186, 366-370, 399
- Terraform state files (see state files)
- terraform validate command, 365
- terraform version command, 301
- terraform workspace list command, 97
- terraform workspace new command, 96
- terraform workspace select command, 97
- terraform workspace show command, 95
- terraform_remote_state data source, 105-113
- Terragrunt, 93, 401
 - installing, 407
 - promotion across environments, 407-412
 - tgswitch, 301
- Terratest, 248
 - integration tests with, 355-360
 - plan tests with, 367-368
 - retries in integration tests, 360-361
 - unit tests with, 324-332
 - versions of, 330
- test doubles, 320
- test pyramid, 362
- Test-Driven Development (TDD), 293

- testable modules for production-grade infrastructure, 291-298
- testing
 - automated tests, 398
 - in application code workflow, 385
 - end-to-end tests, 362-364
 - in infrastructure code workflow, 399-400
 - integration tests, 348-361
 - plan testing, 366-370
 - purpose of, 320
 - server testing, 370-372
 - static analysis, 364-366
 - types of, 320-321
 - unit tests, 321-347
 - when to use, 297
 - best practices, 373
 - comparison of approaches, 373
 - manual tests
 - cleanup, 319-320
 - explained, 317-319
 - Ruby comparison, 316-317
- tfenv, 299-301
- tgswitch, 301
- time estimates for production-grade infrastructure, 275-279
- transparent portability, 19
- type constraints, 61
- type parameter, 61

U

- Ubuntu, 249
- UI (user interface), 195
- unit tests
 - dependency injection, 332-343
 - purpose of, 320
 - Ruby comparison, 321-324
 - running in parallel, 343-347
 - with Terratest, 324-332
- units, defined, 320
- User Data scripts versus provisioners, 310
- user interface (UI), 195
- user space, 9
- user_data parameter, 14, 53
- user_data_replace_on_change parameter, 54

V

- Vagrant, purpose of, 11
- validating
 - code, 17

- infrastructure, 319
- validation blocks, 294-295
- validation parameter, 61
- values built-in function, 151
- variable references, 64
- variables, 121
 - (see also input variables; output variables)
 - declaring, 60-66
- version control, 51
 - in application code workflow, 382
 - disadvantages of, 83-84
 - importance of, 17
 - in infrastructure code workflow, 391-395
 - secrets in, 192
- versioning
 - in Amazon S3, 85, 86
 - modules, 133-138
 - for production-grade infrastructure, 298-305
- versioning pinning, 298-303
- VM images, 8
- VMs (virtual machines), 8
- VPCs (virtual private clouds)

- Default VPC (in AWS), 42
- network isolation, 133
- subnets, 59

W

- web servers
 - deploying, 52-59
 - cluster of web servers, 66-70
 - configurable web servers, 60-66
 - refactoring, 283-285
- worker nodes (Kubernetes), 253, 266
- workflows (see CI/CD workflow)
- workspaces
 - isolating state files, 94-100
 - limitations of, 99

Y

- yak shaving, 277-278

Z

- zero-downtime deployment, 169-179
 - limitations, 181-184

About the Author

Yevgeniy (Jim) Brikman loves programming, writing, speaking, traveling, and lifting heavy things. He is the cofounder of Gruntwork, a company that provides DevOps as a Service. He's also the author of another book published by O'Reilly Media called *Hello, Startup: A Programmer's Guide to Building Products, Technologies, and Teams*. Previously, he worked as a software engineer at LinkedIn, TripAdvisor, Cisco Systems, and Thomson Financial and got his BS and master's degrees at Cornell University. For more info, check out ybrikman.com.

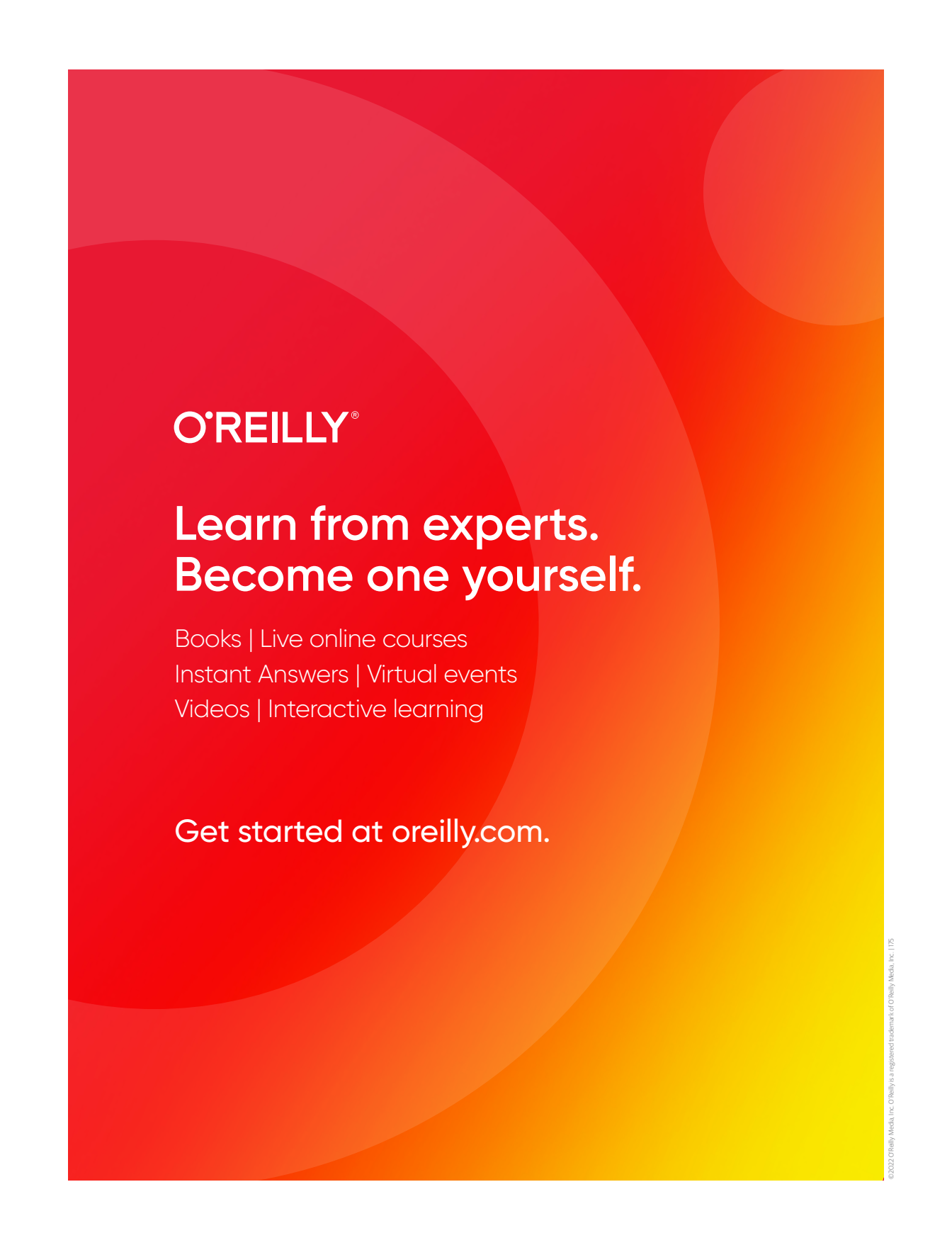
Colophon

The animal on the cover of *Terraform: Up and Running* is a flying dragon lizard (*Draco volans*), a small reptile so named for its ability to glide using winglike flaps of skin known as *patagia*. The patagia are brightly colored and allow the animal to glide for up to eight meters. The flying dragon lizard is commonly found in many Southeast Asian countries, including Indonesia, Vietnam, Thailand, the Philippines, and Singapore.

Flying dragon lizards feed on insects and can grow to more than 20 centimeters in length. They live primarily in forested regions, gliding from tree to tree to find prey and avoid predators. Females descend from the trees only to lay their eggs in hidden holes in the ground. The males are highly territorial and will chase rivals from tree to tree.

Although once thought to be poisonous, flying dragon lizards pose no threat to humans and are sometimes kept as pets. They are not currently threatened or endangered. Many of the animals on O'Reilly covers are endangered; all of them are important to the world.

The cover illustration is by Karen Montgomery, based on an antique line engraving from *Johnson's Natural History*. The cover fonts are Gilroy Semibold and Guardian Sans. The text font is Adobe Minion Pro; the heading font is Adobe Myriad Condensed; and the code font is Dalton Maag's Ubuntu Mono.

The background of the entire page is a vibrant gradient of red and orange, transitioning from a deep red on the left to a bright yellow-orange on the right. Overlaid on this gradient are several large, semi-transparent, overlapping circles in various shades of red and orange, creating a dynamic, organic feel.

O'REILLY®

**Learn from experts.
Become one yourself.**

Books | Live online courses
Instant Answers | Virtual events
Videos | Interactive learning

Get started at oreilly.com.